



python

Modul Praktikum

Dasar Kecerdasan Artificial Intelligence

S1 Informatika

Published by school of computing

Our official instagram



@informatics_lab_telu

LEMBAR PENGESAHAN

Saya yang bertanda tangan di bawah ini:

Nama : SABRINA ADINDA SARI, S.Kom., M.Kom.

NIP : 25010014

Koordinator Mata Kuliah : Dasar Kecerdasan Artifisial

Program Studi : S1 Informatika

Menerangkan dengan sesungguhnya bahwa modul ini digunakan untuk pelaksanaan praktikum di Semester Genap Tahun Ajaran 2025/2026 di Laboratorium Informatika Fakultas Informatika Universitas Telkom.



Bandung, 07 Februari 2026



Mengesahkan,

Mengetahui,

Koordinator MK Berpraktik Dasar Kecerdasan Artifisial

Kepala Program Studi S1 Informatika

SABRINA ADINDA SARI, S.Kom., M.Kom.

NIP. 25010014

Dr. MAHMUD DWI SULISTIYO, S.T., M.T.

NIP. 13880017

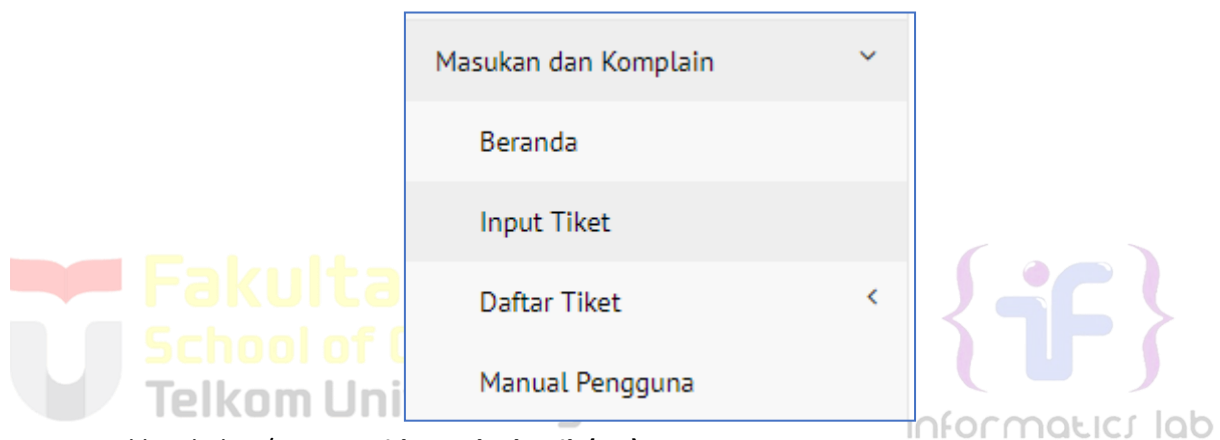
PERATURAN PRAKTIKUM LABORATORIUM INFORMATIKA 2025/2026

1. Praktikum diampu oleh dosen kelas dan dibantu oleh asisten laboratorium dan asisten praktikum.
2. Praktikum dilaksanakan di Gedung TULT (Telkom University Landmark Tower) lantai 6 dan 7 sesuai jadwal yang ditentukan.
3. Praktikan wajib membawa modul praktikum, kartu praktikum, dan alat tulis.
4. Praktikan wajib mengecek kehadiran di igracias dan sheet yang dibagikan asisten.
5. Durasi kegiatan praktikum S-1 = 2 jam (100 menit).
6. Jumlah pertemuan praktikum:
 - 16 kali pertemuan
7. Praktikan wajib hadir minimal 75% dari seluruh pertemuan praktikum di lab.
8. Praktikan yang datang terlambat :
 - ≤ 5 menit : diperbolehkan mengikuti praktikum tanpa tambahan praktikum
 - ≥ 30 menit : tidak diperbolehkan mengikuti praktikum
9. Saat praktikum berlangsung, asisten praktikum dan praktikan:
 - Wajib menggunakan seragam sesuai aturan institusi.
 - Wajib mematikan/ mengkondisikan semua alat komunikasi.
 - Dilarang membuka aplikasi yang tidak berhubungan dengan praktikum yang berlangsung.
 - Dilarang mengubah pengaturan *software* maupun *hardware* komputer tanpa ijin.
 - Dilarang membawa makanan maupun minuman di ruang praktikum.
 - Dilarang memberikan jawaban ke praktikan lain.
 - Dilarang menyebarkan soal praktikum.
 - Dilarang membuang sampah di ruangan praktikum.
 - Wajib meletakkan alas kaki dengan rapi pada tempat yang telah disediakan.
10. Setiap praktikan dapat mengikuti praktikum susulan maksimal dua modul untuk satu mata kuliah praktikum.
 - Praktikan yang dapat mengikuti praktikum susulan hanyalah praktikan yang memenuhi syarat sesuai ketentuan institusi, yaitu: sakit (dibuktikan dengan surat keterangan medis), tugas dari institusi (dibuktikan dengan surat dinas atau dispensasi dari institusi), atau mendapat musibah atau keduakaan (menunjukkan surat keterangan dari orangtua/wali mahasiswa.)
 - Persyaratan untuk praktikum susulan diserahkan sesegera mungkin kepada asisten laboratorium untuk keperluan administrasi.
 - Praktikan yang diijinkan menjadi peserta praktikum susulan ditetapkan oleh Lab Informatika dan tidak dapat diganggu gugat.
11. Ketidakhadiran pada kelas praktikum:
 - **Nilai Modul = 0**
12. Meminta, mendapatkan, dan menyebarluaskan soal dan atau kunci jawaban praktikum:
 - **Penyebarnya soal dan kunci jawaban: Pengajuan sanksi kepada Komisi Disiplin Fakultas**
 - **Penerima soal dan kunci jawaban: Nilai '0' pada (seluruh assessment) praktikum**

13. Lupa menghapus file praktikum:
 - **Pengurangan nilai modul 20%**
14. Memicu kegaduhan, sehingga membuat situasi tidak kondusif (jalan-jalan, mengganggu teman, mengobrol, dll), asisten praktikum diwajibkan menegur sebanyak 3x
 - **Pengurangan nilai modul 50%**
15. Menyalahgunakan fitur Ims
 - **Siap menerima sanksi lebih lanjut dari IFLAB**

TATA CARA KOMPLAIN PRAKTIKUM IFLAB MELALUI IGRACIAS

1. Login IGracias
2. Pilih Menu **Masukan dan Komplain**, pilih **Input Tiket**



3. Pilih Fakultas/Bagian: **Bidang Akademik (FIF)**
4. Pilih Program Studi/Urusan: **Urusan Laboratorium/Bengkel/Studio (FIF)**
5. Pilih Layanan: **Praktikum**
6. Pilih Kategori: **Pelaksanaan Praktikum**, lalu pilih **Sub Kategori**.
7. Isi **Deskripsi** sesuai komplain yang ingin disampaikan.

Input Keluhan

Fakultas / Bagian :

Program Studi / Urusan :

Pelapor : RIZQILLAH ZAHRA LESTARI

Layanan :

Kategori :

Sub Kategori :

Tipe Masukan : ☒ Komplain ☐ Masukan

Deskripsi :

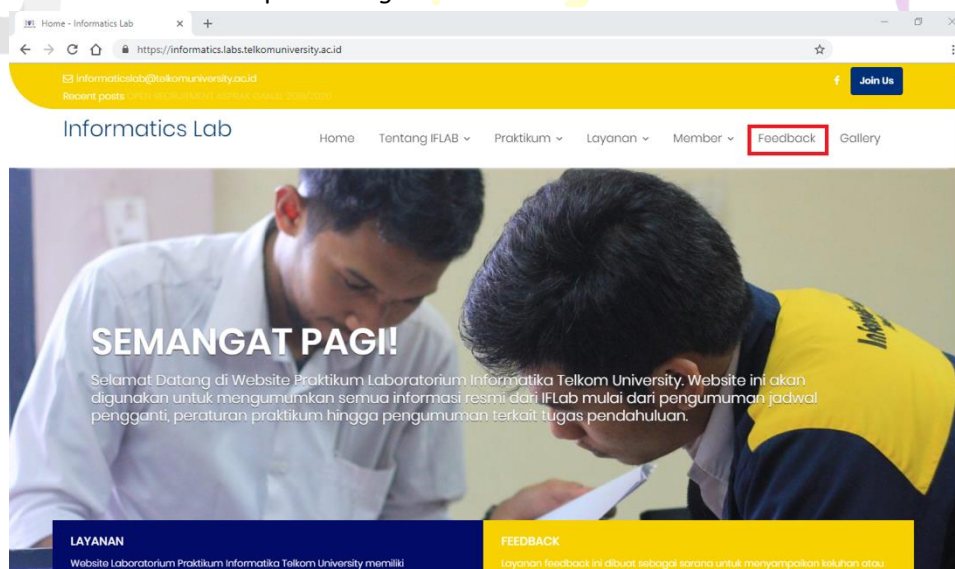
B *I* U

Paragraph Font size

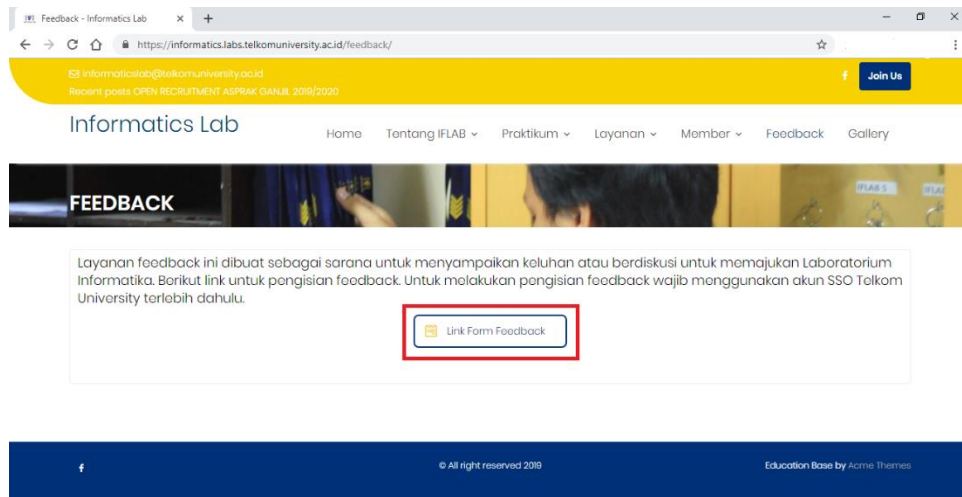
8. Lampirkan *file* jika perlu. Lalu klik Kirim.

TATA CARA KOMPLAIN PRAKTIKUM IFLAB MELALUI WEBSITE

1. Buka *website* <https://informatics.labs.telkomuniversity.ac.id/> melalui *browser*.
2. Pilih menu **Feedback** pada *navigation bar website*.



3. Pilih tombol **Link Form Feedback**.



4. Lakukan *login* menggunakan akun **SSO Telkom University** untuk mengakses *form feedback*.
5. Isi *form* sesuai dengan *feedback* yang ingin diberikan.

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
PERATURAN PRAKTIKUM LABORATORIUM INFORMATIKA 2024/2025.....	ii
TATA CARA KOMPLAIN PRAKTIKUM IFLAB MELALUI IGRACIAS	iii
TATA CARA KOMPLAIN PRAKTIKUM IFLAB MELALUI <i>WEBSITE</i>	iv
DAFTAR ISI.....	vi
DAFTAR GAMBAR	xii
DAFTAR TABEL	xiv
Modul 1 PENGENALAN PYTHON.....	1
1.1. Python dengan Anaconda.....	1
1.2. Membuat Program dengan Python	1
1.2.1. Membuka Python dengan Terminal / CMD.....	1
1.2.2. Membuka Python dengan Jupyter Notebook	2
1.2.3. Python sebagai Kalkulator	5
1.2.4. Tipe Data Pada Python	6
A. Tipe Data Numerik	6
B. Tipe Data <i>String</i>	7
C. Tipe Data <i>List</i>	8
D. Tipe Data <i>Dictionary</i>	10
E. Tipe Data <i>Set</i>	12
F. Tipe Data <i>Boolean</i>	14
1.3. <i>Input</i> dan <i>Output</i> Pada Python	15
A. <i>Input</i> Pada Python.....	15
B. <i>Output</i> Pada Python	15

1.4.	Range Statement	16
1.5.	Percabangan Pada Python.....	17
1.5.1.	Percabangan `if`	17
1.5.2.	Percabangan `if-else`	18
1.5.3.	Percabangan `if-elif-else`	18
1.6.	Perulangan <i>While</i>	19
1.7.	Perulangan <i>For</i>	20
1.8.	Fungsi.....	21
1.8.1.	Parameter dan Argumen Pada Fungsi	21
1.8.2.	Fungsi Anonim	22
1.9.	Pemrograman Modular	22
1.10.	Soal Latihan Pemrograman Python	23
Modul 2	PENGENALAN <i>LIBRARY</i> NUMPY	24
2.1.	Cara <i>Import</i> NumPy	24
2.2.	Definisi <i>Array</i> dan <i>Array</i> NumPy.....	24
2.3.	Membuat Program dengan <i>Library</i> Numpy.....	25
2.4.	Atribut <i>Array</i> NumPy	26
2.4.1.	Atribut <i>ndim</i>	26
2.4.2.	Atribut <i>shape</i>	27
2.4.3.	Atribut <i>size</i>	28
2.4.4.	Atribut <i>dtype</i>	29
2.5.	Deklarasi <i>Array</i> dengan NumPy.....	29
2.6.	Penambahan Elemen <i>Array</i>	32
2.7.	Penghapusan Elemen <i>Array</i>	34

2.8.	Pengurutan Elemen Array	35
2.9.	<i>Indexing</i> dan <i>Slicing</i> Array	35
2.10.	Penyalinan (<i>Copy</i>) Array	37
2.11.	Operasi Dasar Array.....	38
2.12.	<i>Transpose</i> dan <i>Reshape</i> Array	39
2.13.	Soal Latihan Pemrograman Numpy	41
Modul 3 PENGENALAN <i>LIBRARY</i> NETWORKX		42
3.1.	Cara <i>Import</i> NetworkX.....	42
3.2.	Membuat Graf Kosong pada NetworkX	42
3.3.	Menambahkan <i>Node</i> Pada Graf	44
3.4.	Menambahkan <i>Edge</i> Pada Graf.....	45
3.5.	Menghapus <i>Node</i> dan <i>Edge</i>	46
3.6.	Elemen Pada Graf	47
3.7.	Menambahkan <i>Weight</i> Pada <i>Graph</i>	48
3.8.	Analisis Graf	49
3.9.	Menampilkan Graf dengan Matplotlib.....	52
3.9.1.	Tata Letak <i>Node</i>	52
3.9.2.	Fungsi <i>Layout</i> Pada Graf	53
3.9.3.	Fungsi <i>show_graph</i> Untuk Visualisasi Graf	55
3.10.	Membuat Graf Secara Otomatis.....	56
3.10.1.	<i>Complete Graph</i>	56
3.10.2.	<i>Complete Bipartite Graph</i>	57
3.10.3.	<i>Petersen Graph</i>	58
3.10.4.	<i>Tetrahedral Graph</i>	58

3.10.5.	<i>Edros-Renyi Graph</i>	59
3.10.6.	<i>Barabasi-Albert Graph</i>	60
3.11.	Soal Latihan Pemrograman NetworkX	60
Modul 4 PENGENALAN <i>LIBRARY</i> MATPLOTLIB		62
4.1.	Cara <i>Import</i> Matplotlib	62
4.2.	Elemen <i>Figure</i> Pada Matplotlib	62
4.3.	Labeling Plot	64
4.4.	Plot <i>Pairwise</i> Pada Matplotlib	67
4.5.	Plot Distribusi Statistik Pada Matplotlib	72
4.6.	Soal Latihan <i>Library</i> Matplotlib	74
Modul 5 IMPLEMENTASI <i>BREADTH-FIRST SEARCH</i>		76
5.1.	Implementasi BFS Pada NetworkX	76
5.1.1.	Implementasi BFS Pada <i>Tree</i>	76
5.1.2.	Implementasi BFS Pada <i>Directed Graph</i>	80
5.2.	Soal Latihan	83
Modul 6 IMPLEMENTASI <i>DEPTH-FIRST SEARCH</i>		84
6.1.	Implementasi DFS Pada <i>Tree</i>	84
6.2.	Implementasi DFS Pada <i>Directed Graph</i>	88
6.3.	Soal Latihan	91
Modul 7 IMPLEMENTASI <i>UNIFORM COST SEARCH</i>		92
7.1.	Implementasi UCS Pada <i>Tree</i>	92
7.2.	Implementasi UCS Pada <i>Directed Graph</i>	95
7.3.	Soal Latihan	98
Modul 8 IMPLEMENTASI <i>HILL CLIMBING</i>		100

8.1.	Contoh Implementasi <i>Hill Climbing</i>	100
8.2.	Soal Latihan	102
Modul 9 IMPLEMENTASI <i>SIMULATED ANNEALING</i>		104
9.1.	Cara Kerja Algoritma <i>Simulated Annealing</i>	104
9.2.	Contoh Implementasi	105
9.3.	Soal Latihan	108
Modul 10 IMPLEMENTASI <i>GREEDY BEST-FIRST SEARCH</i>		110
10.1.	Implementasi <i>Greedy</i> BFS pada <i>Directed Graph</i>	110
10.2.	Soal Latihan	113
Modul 11 IMPLEMENTASI <i>A*</i>		115
11.1.	Implementasi <i>A*</i> Pada <i>Directed Graph</i>	116
11.2.	Soal Latihan	119
Modul 12 PENGENALAN <i>LIBRARY SYMPY</i>		120
12.1.	Cara <i>Import</i> SymPy	120
12.2.	Pendefinisian Simbol Pada SymPy.....	120
12.3.	Aljabar Dasar Pada SymPy	121
12.4.	Kalkulus Dasar Pada SymPy	122
12.5.	Penyelesaian Persamaan Simbolik Pada SymPy.....	123
12.6.	Manipulasi Matriks Pada SymPy.....	125
12.7.	Soal Latihan <i>Library</i> SymPy.....	126
Modul 13 PENGENALAN <i>LIBRARY SCIKIT-FUZZY</i>		128
13.1.	Instalasi Scikit-Fuzzy	128
13.2.	Membuat Program dengan <i>Library</i> Scikit-Fuzzy.....	129
13.2.1.	Inisialisasi Variabel <i>Fuzzy</i>	129

13.2.2.	Membuat Fungsi Keanggotaan	130
13.2.3.	Penentuan Aturan <i>Fuzzy</i>	132
13.2.4.	Pemberian <i>Input</i> dan Perhitungan dari <i>Fuzzy System</i>	133
13.3.	<i>Study Case</i> Sistem <i>Fuzzy</i> dengan Scikit-Fuzzy.....	134
13.4.	Soal Latihan Pemrograman Scikit-Fuzzy	137



DAFTAR GAMBAR

Gambar 1. Tampilan terminal setelah menjalankan perintah `python`	2
Gambar 2. Tampilan awal Jupyter Notebook.	3
Gambar 3. Tampilan menu untuk membuat folder baru pada Jupyter Notebook.	3
Gambar 4. Tampilan menu untuk mengubah nama folder. (a) Menu untuk memilih folder yang akan diubah namanya. (b) Menu untuk mengisi nama baru dari folder yang dipilih.	4
Gambar 5. Tampilan menu untuk membuat file Python Notebook baru.	4
Gambar 6. Tampilan menu Python Notebook.	5
Gambar 7. Gambar contoh Complete Graph.	56
Gambar 8. Gambar contoh Bipartite Graph.	57
Gambar 9. Gambar contoh Edros-Renyi Graph.	59
Gambar 10. Contoh Implementasi Tree untuk BFS.	77
Gambar 11. Contoh directed graf untuk implementasi DFS.	80
Gambar 12. Contoh tree untuk DFS.	85
Gambar 13. Contoh directed graph untuk DFS.	88
Gambar 14. Contoh tree untuk UCS.	93
Gambar 15. Contoh directed graph untuk UCS.	96
Gambar 16. Grafik persebaran kota yang akan dikunjungi oleh salesman pada soal Hill Climbing.	101
Gambar 17. Grafik persebaran kota yang akan dikunjungi oleh salesman pada contoh soal Simulated Annealing.	106
Gambar 18. Grafik persebaran kota yang akan dikunjungi oleh salesman pada soal latihan Simulated Annealing.	109
Gambar 19. Contoh directed graph untuk Greedy BFS.	110
Gambar 20. Contoh directed graph untuk A*.	116
Gambar 21. Tampilan terminal saat instalasi Scikit-Fuzzy berhasil.	129

Gambar 22. Grafik fungsi keanggotaan untuk soal latihan. (a) Grafik fungsi keanggotaan quality.
(b) Grafik fungsi keanggotaan service. 137



DAFTAR TABEL

Tabel 1. Urutan operasi aritmatika yang dilakukan pada Python.	5
Tabel 2. Tipe data numerik beserta keterangannya pada Python.	6
Tabel 3. Perbedaan array pada Python dan NumPy Array.	25
Tabel 4. Struktur dimensi array NumPy.....	26
Tabel 5. Data nilai mahasiswa untuk soal latihan library Matplotlib.	74
Tabel 6. Jarak antar kota yang harus dikunjungi salesman.	102
Tabel 7. Nilai heuristik untuk directed graph Greedy BFS.....	111
Tabel 8. Nilai heuristik untuk directed graph A^*	116



Modul 1 PENGENALAN PYTHON

Tujuan Praktikum:

1. Mahasiswa mengetahui instalasi dan *working environment* pada Python (menggunakan Terminal dan *Jupyter Notebook*).
2. Mahasiswa memahami cara pembuatan program dengan Python.
3. Mahasiswa mengetahui fitur-fitur utama yang tersedia pada Python.

1.1. Python dengan Anaconda

REFERENSI: <https://conda.io/projects/conda/en/latest/user-guide/index.html>

Anaconda adalah distribusi Python yang populer dalam ilmu data dan pengembangan kecerdasan buatan (AI). Anaconda menyediakan *environment* yang terisolasi untuk menjalankan berbagai proyek Python, lengkap dengan manajemen *package* yang mudah. Anaconda sudah dilengkapi dengan *package* bawaan yang berguna untuk implementasi algoritma AI seperti NumPy, Scikit-Learn, NetworkX, SymPy, dan lain-lain.

Anaconda dapat diunduh pada *link*: <https://www.anaconda.com/download>

Langkah-langkah instalasi tertera pada *link*:
<https://docs.anaconda.com/anaconda/install>



1.2. Membuat Program dengan Python

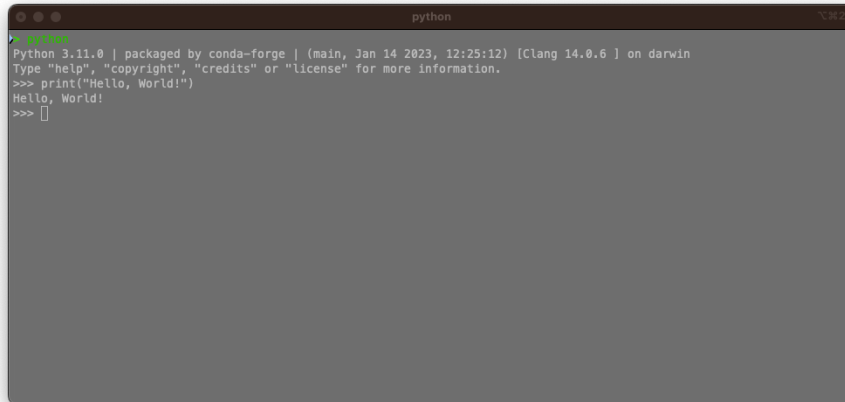
REFERENSI: <https://docs.python.org/3/tutorial/index.html>

Python adalah *high-level programming language* yang populer karena penulisan sintaks yang sederhana dan mudah dibaca. Python sering digunakan dalam pengembangan kecerdasan buatan dan ilmu data karena Python memiliki *library* yang banyak dan mudah digunakan untuk mendukung pengolahan data dan pengembangan model pembelajaran mesin.

1.2.1. Membuka Python dengan Terminal / CMD

REFERENSI: <https://docs.python.org/3/tutorial/introduction.html>

Program Python dapat dijalankan melalui Terminal dengan menjalankan perintah ``python`` pada Terminal / *Command Prompt*. Gambar 1 menunjukkan tampilan Terminal setelah membuka Python.



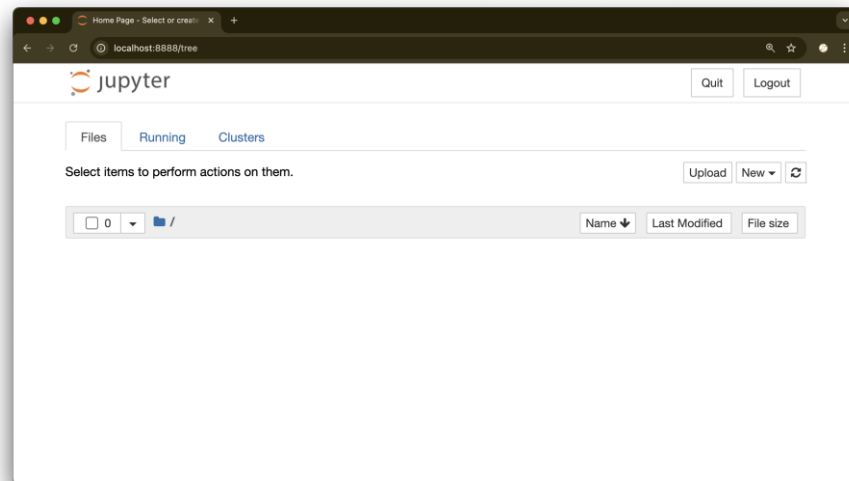
```
> python
Python 3.11.0 | packaged by conda-forge | (main, Jan 14 2023, 12:25:12) [Clang 14.0.6 ] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
>>>
```

Gambar 1. Tampilan terminal setelah menjalankan perintah ``python``.

1.2.2. Membuka Python dengan Jupyter Notebook

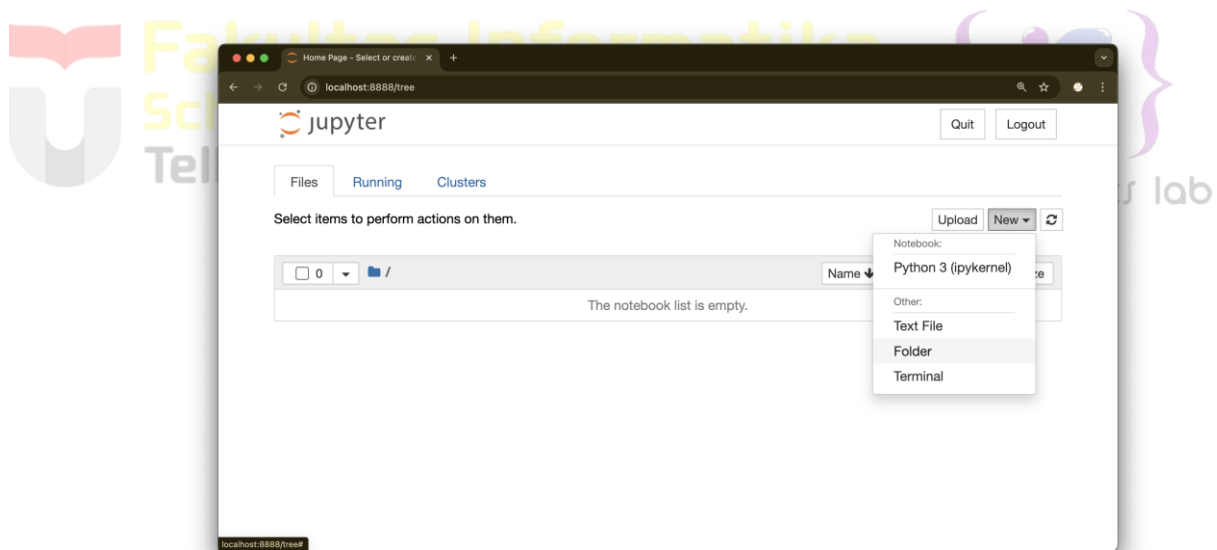
Program Python juga dapat dijalankan dari *Jupyter Notebook*. *Jupyter Notebook* memungkinkan Python dijalankan dengan format *Python Notebook*. Untuk membuka program Python dengan *Jupyter Notebook*, lakukan langkah berikut:

1. Buka **Anaconda Prompt** pada Windows atau **Terminal** pada MacOS.
2. Pada **MacOs**, Jupyter Notebook dapat dibuka dengan menjalankan perintah ``jupyter notebook /Users/[User]/Documents`` untuk membuka *Jupyter Notebook* pada folder *Document*. Ubah ``[user]`` dengan nama *user* yang digunakan. Sedangkan pada **Windows**, Jupyter Notebook dapat dibuka dengan ``jupyter notebook C:\Users\[User]\Documents``.
3. *Jupyter Notebook* akan terbuka otomatis pada *browser* dengan alamat ``localhost:8888/tree``.
4. Gambar 2 menunjukkan tampilan dari *Jupyter Notebook*.



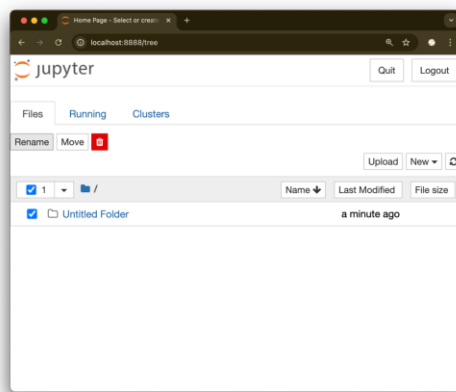
Gambar 2. Tampilan awal Jupyter Notebook.

5. Untuk membuat folder baru, klik tombol `new`, kemudian pilih `Folder`. Tombol untuk membuat folder baru terletak seperti pada Gambar 3.

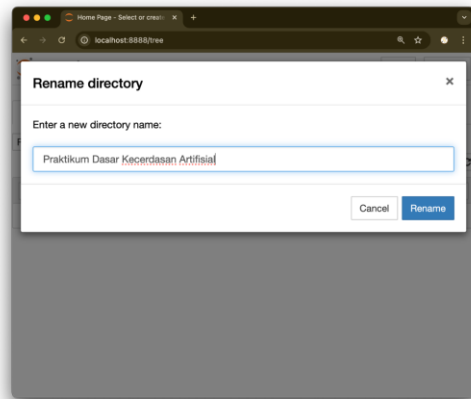


Gambar 3. Tampilan menu untuk membuat folder baru pada Jupyter Notebook.

6. Untuk mengganti nama folder, ceklis folder kemudian klik tombol `Rename`, yang terletak pada kiri atas seperti yang ditunjukkan oleh Gambar 4(a), kemudian tuliskan nama folder yang akan diganti pada *pop-up* yang muncul pada Gambar 4(b).



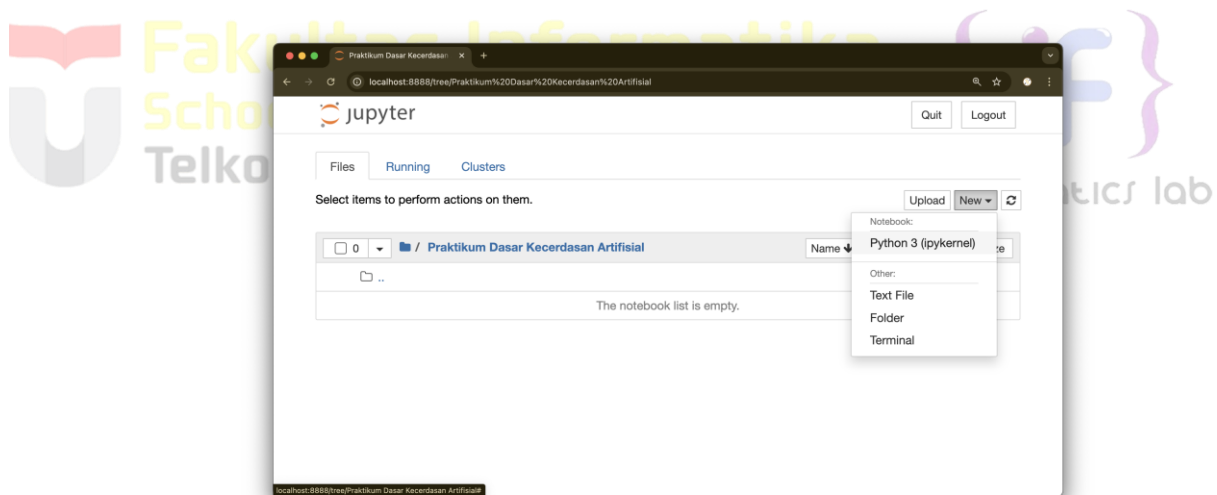
(a)



(b)

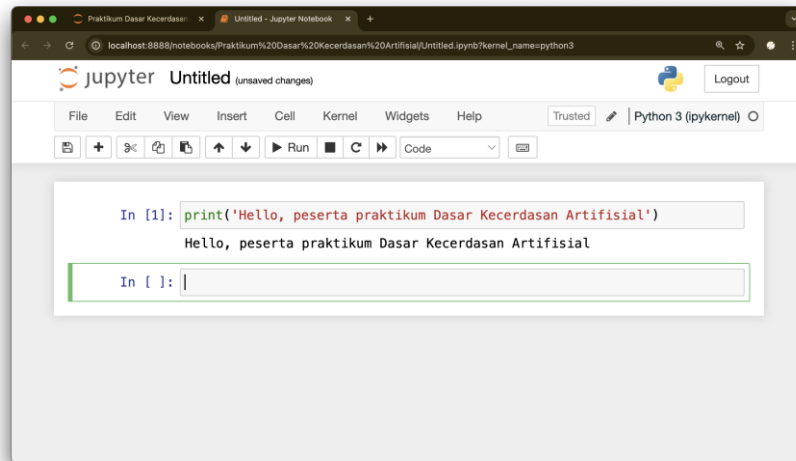
Gambar 4. Tampilan menu untuk mengubah nama folder. (a) Menu untuk memilih folder yang akan diubah namanya. (b) Menu untuk mengisi nama baru dari folder yang dipilih.

7. Untuk membuat Python Notebook, klik tombol `new` di kanan atas dan pilih `Python 3 (ipykernel)` seperti yang ditunjukkan pada Gambar 5.



Gambar 5. Tampilan menu untuk membuat file Python Notebook baru.

8. Jika sudah, maka akan terbuka Python Notebook dan dapat menuliskan kode pada Notebook tersebut seperti pada Gambar 6.



Gambar 6. Tampilan menu Python Notebook.

Program HelloWorld menggunakan Terminal Python

>>>	<code>print("Hello, peserta praktikum Dasar Kecerdasan Artifisial!")</code>
	Output: Hello, peserta praktikum Dasar Kecerdasan Artifisial!

Program HelloWorld menggunakan Jupyter Notebook

[1]:	<code>print("Hello, peserta praktikum Dasar Kecerdasan Artifisial!")</code>
	Output: Hello, peserta praktikum Dasar Kecerdasan Artifisial!

1.2.3. Python sebagai Kalkulator

Python mengikuti aturan prioritas operasi standar matematika ketika mengeksekusi ekspresi aritmatika. Aturan ini dikenal juga sebagai **PEMDAS** (*Parentheses, Exponents, Multiplication and Division, Addition and Substraction*). Prioritas operasi Python tertera pada Tabel 1.

Tabel 1. Urutan operasi aritmatika yang dilakukan pada Python.

Prioritas	Operator	Jenis Operasi
1	()	Tanda Kurung
2	**	Pangkat
3	*, /, //, %	Perkalian, Pembagian, Pembagian Integer, Modulus
4	+, -	Penjumlahan, Pengurangan

Berikut adalah contoh penggunaan Python sebagai kalkulator:

>>>	<code>2 + 2</code>
-----	--------------------

	Output: 4
>>>	5 * 6
	Output: 30
>>>	10 - 5
	Output: -5
>>>	(3 + 2) * 5
	Output: 25
>>>	5 - 3 * 2
	Output: -1

Berikut adalah contoh operasi pembagian dan pembagian *integer* pada Python :

>>>	10 / 2 # pembagian
	Output: 5.0
>>>	10 // 2 # pembagian integer
	Output: 5
>>>	15 / 4 # pembagian
	Output: 3.75
>>>	15 // 4 # pembagian integer
	Output: 3

1.2.4. Tipe Data Pada Python

Tipe data dalam Python adalah kategori atau jenis data yang menentukan bagaimana data tersebut dapat digunakan dan dioperasikan dalam program. Python memiliki beberapa tipe data utama, di antaranya:

A. Tipe Data Numerik

Tipe data numerik dalam Python adalah jenis data yang digunakan untuk menyimpan nilai angka. Jenis tipe data numerik beserta contohnya terdapat pada Tabel 2.

Tabel 2. Tipe data numerik beserta keterangannya pada Python.

Tipe Data	Contoh	Keterangan
Integer (`int`)	10, -5, 0	Digunakan untuk menyimpan bilangan bulat.

Float ('float')	3.14, -0.001, 2.0	Digunakan untuk menyimpan bilangan desimal / real.
Complex ('complex')	1 + 2i, 3j	Digunakan untuk menyimpan bilangan kompleks, yang memiliki bilangan real dan bagian imajiner.

B. Tipe Data *String*

String dalam Python adalah tipe data yang digunakan untuk merepresentasikan teks. *String* merupakan urutan karakter yang didefinisikan dalam tanda kutip ('...' / "..."). Berikut adalah cara mendefinisikan *string*:

[1]:	<pre>str = "Hello, World!" print(str)</pre>
	Output: Hello, World!

Terdapat beberapa operasi yang dapat dilakukan pada *string* dalam Python, diantaranya:

Penggabungan (<i>Concatenation</i>)	
[1]:	<pre>str1 = "Hello" str2 = "World" result = str1 + " " + str2 print(result)</pre>
	Output: Hello World
Pengulangan (<i>Repetition</i>)	
[2]:	<pre>str = "Hello" result = str * 3 print(result)</pre>
	Output: HelloHelloHello
Mengakses karakter	
[3]:	<pre>str = DKA first_char = str[0] # indeks dimulai dari 0 last_char = str[-1] # indeks -1 adalah indeks terakhir print(first_char, last_char)</pre>
	Output: D A
<i>String slicing</i>	
[4]:	<pre>str = "Hello, World!" sliced = str[0:5] # Mengambil karakter ke 0 - 5 print(sliced)</pre>
	Output: Hello
Mengukur panjang string	
[5]:	<pre>str = "Hello, World!" print(len(str))</pre>
	Output:

String dalam Python juga memiliki *method* bawaan untuk memanipulasi string itu sendiri, seperti:

Kapitalisasi string	
[1]:	<pre>str = "Hello, World!" print(str.upper())</pre>
	Output: HELLO, WORLD!
Konversi ke huruf kecil	
[2]:	<pre>str = "Hello, World!" print(str.lower())</pre>
	Output: hello, world!
Menghapus spasi berlebih	
[3]:	<pre>str = " Hello, World! " print(str.strip())</pre>
	Output: Hello, World!
Mengganti bagian string	
[4]:	<pre>str = "Hello, World!" replaced = str.replace("World", "Dunia") # .replace([old], [new]) print(replaced)</pre>
	Output: Hello, Dunia!

C. Tipe Data *List*

List adalah struktur data yang digunakan untuk menyimpan kumpulan elemen dalam satu variabel. *List* dapat berisi berbagai tipe data, seperti angka, *string*, atau bahkan *list* lain. *List* didefinisikan menggunakan tanda kurung siku `[ ]`, dengan elemen-elemen di dalamnya dipisahkan oleh koma. Berikut adalah contoh pendefinisian *list*:

[1]:	<pre>list_of_numbers = [1,3,5,10] print(list_of_numbers)</pre>
	Output: [1, 3, 5, 10]
[2]:	<pre>list_of_strings = ["dasar", "kecerdasan", "artifisial"] print(list_of_strings)</pre>
	Output: ['dasar', 'kecerdasan', 'artifisial']
[3]:	<pre>list_of_items = [1, "hello", ["ini", "list"]] print(list_of_items)</pre>
	Output: [1, 'hello', ['ini', 'list']]

Indeks digunakan untuk mengakses elemen pada *list*. Indeks pertama adalah `0`, dan indeks terakhir adalah `-1`.

[1]:	<pre>my_list = [1, 3, 5, "DKA", True] # Pengambilan elemen berdasarkan indeks first_elm = my_list[0] second_elm = my_list[1] last_elm = my_list[-1] second_last_elm = my_list[-2] # Cetak hasil print(first_elm, second_elm, last_elm, second_last_elm)</pre>
	Output: 1 3 True 'DKA'
[2]:	<pre>my_list = [1, 3, 5, "DKA", True] elms = my_list[1:4] # Mengambil anggota ke 1 - 4 dalam list print(elms)</pre>
	Output: [3, 5, 'DKA']
[3]:	<pre>numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] first_five = numbers[:5] # Mengambil 5 anggota pertama last_three = numbers[-3:] # Mengambil 3 anggota terakhir print(first_five) print(last_three)</pre>
	Output: [1, 2, 3, 4, 5] [8, 9, 10]
[4]:	<pre>numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] numbers[0] = 24 # Mengubah nilai pada indeks ke 0 print(numbers)</pre>
	Output: [24, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Python memiliki metode *list comprehension* yang merupakan cara singkat dan elegan untuk membuat *list* baru dari *list* yang sudah ada atau *iterable* lain seperti range atau string. *List comprehension* memungkinkan untuk menulis *loop* dalam satu baris di dalam suatu *list*. *list comprehension* dapat dituliskan dengan sintaks berikut:

<code>[expression for item in iterable if condition]</code>
<pre>`expression`: Nilai atau operasi yang akan dimasukkan ke dalam list baru. `item`: Variabel yang mengambil setiap elemen dari `iterable`. `iterable`: Objek yang dapat diiterasi seperti list, range, string, dll. `condition`: (opsional) Kondisi untuk menyertakan elemen dalam list baru.</pre>

Berikut adalah contoh sederhana dari penggunaan *list comprehension*:

Membuat <i>list</i> dari <i>range</i> menggunakan <i>list comprehension</i>	
[1]:	<pre>squares = [x**2 for x in range(10)] print(squares)</pre>
	Output:

	[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
List comprehension dengan kondisi	
[2]:	<code>even_nums = [x for x in range(10) if x % 2 == 0] print(even_nums)</code>
	Output: [0, 2, 4, 6, 8]
List comprehension dengan operasi pada string	
[3]:	<code>fruits = ["apple", "banana", "cherry"] uppercased = [fruit.upper() for fruit in fruits] print(uppercased)</code>
	Output: ['APPLE', 'BANANA', 'CHERRY']
Nested list comprehension	
[4]:	<code>pairs = [(x,y) for x in range(2) for y in range(2)] print(pairs)</code>
	Output: [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]

D. Tipe Data *Dictionary*

Dictionary dalam Python adalah tipe data yang digunakan untuk menyimpan pasangan *key-value*. Setiap *key* dalam *dictionary* bersifat unik dan digunakan untuk mengakses *value* yang terkait. *Dictionary* didefinisikan menggunakan tanda kurung kurawal `{ }` seperti berikut:

[1]:	<code>my_dict = { "nama": "Fizz", "umur": 21, "kota": "Bandung", "universitas": "Telkom University" } print(my_dict["nama"]) print(my_dict["kota"])</code>
	Output: Fizz Bandung

Terdapat beberapa operasi yang dapat dilakukan pada *dictionary*, diantaranya:

Menambahkan atau memperbarui nilai	
[1]:	<code>my_dict = { "nama": "Fizz", "umur": 21, "kota": "Bandung", "universitas": "Telkom University" } # Menambahkan item baru my_dict["negara"] = "Indonesia" print("Setelah ditambah data negara") print(my_dict)</code>

	<pre># Memperbarui nilai print("Setelah nilai umur diperbarui") my_dict["umur"] = 23 print(my_dict)</pre>
	Output: Setelah ditambah data negara <pre>{ 'nama': 'Alice', 'umur': 21, 'kota': 'Bandung', 'universitas': 'Telkom University', 'negara': 'Indonesia' }</pre> Setelah nilai umur diperbarui <pre>{ 'nama': 'Alice', 'umur': 26, 'kota': 'Bandung', 'universitas': 'Telkom University', 'negara': 'Indonesia' }</pre>
Menghapus elemen	
	<pre>[2]: my_dict = { "nama": "Fizz", "umur": 21, "kota": "Bandung", "universitas": "Telkom University" }</pre> # Cara 1 <pre>del my_dict["kota"] # Penghapusan data kota</pre> # Cara 2 # Penghapusan data kota dan value masuk ke variable <pre>kota = my_dict.pop("kota") print(my_dict)</pre>
	Output: <pre>{ 'nama': 'Alice', 'umur': 26, 'universitas': 'Telkom University' }</pre>
Mengakses seluruh <i>keys</i> atau <i>values</i>	
	<pre>[3]: my_dict = { "nama": "Fizz", "umur": 21, "kota": "Bandung", "universitas": "Telkom University" }</pre> <pre>kunci = my_dict.keys() nilai = my_dict.values()</pre>

	<pre>print("kunci:", kunci) print("nilai:", nilai)</pre>
	Output: kunci: ['nama', 'umur', 'kota', 'universitas'] nilai: ['Fizz', 21, 'Bandung', 'Telkom University']
Iterasi dalam dictionary	
[4]:	<pre>my_dict = { "nama": "Fizz", "umur": 21, "kota": "Bandung", "universitas": "Telkom University" } for key, value in my_dict.items(): print("Nilai dari", key, "adalah", value)</pre>
	Output: Nilai dari nama adalah Fizz Nilai dari umur adalah 21 Nilai dari kota adalah Bandung Nilai dari universitas adalah Telkom University
Memeriksa keberadaan key	
[5]:	<pre>my_dict = { "nama": "Fizz", "umur": 21, "kota": "Bandung", "universitas": "Telkom University" } # Mengecek apakah key 'nama' ada di my_dict if "nama" in my_dict: print("Kunci nama ditemukan") else: print("Kunci nama tidak ditemukan") # Mengecek apakah key 'negara' ada di my_dict if "negara" in my_dict: print("Kunci negara ditemukan") else: print("Kunci negara tidak ditemukan")</pre>
	Output: Kunci nama ditemukan Kunci negara tidak ditemukan

E. Tipe Data Set

Set pada Python adalah tipe data yang digunakan untuk menyimpan kumpulan elemen yang bersifat unik dan tidak berurutan. Artinya, setiap elemen dalam set harus berbeda dan tidak duplikat. Set didefinisikan menggunakan tanda kurung kurawal `{}` atau dengan menggunakan `set()` seperti contoh di bawah:

[1]:	<pre># Membuat set kosong my_set = set()</pre>
------	--

	<pre># Membuat set dengan beberapa elemen my_set = {1, 2, 3, "apple", 3.14} print(my_set)</pre>
	Output: {1, 2, 3, 3.14, 'apple'}

Terdapat beberapa operasi yang dapat dilakukan pada *set*, diantaranya:

Menambahkan elemen	
[1]:	<pre>my_set = {1, 2, 3, "apple", 3.14} my_set.add("orange") print(my_set)</pre>
	Output: {1, 2, 3.14, 3, 'apple', 'orange'}
Menghapus elemen	
[2]:	<pre>my_set = {1, 2, 3, "apple", 3.14} # Akan error jika apple tidak ada di set my_set.remove("apple") # Tidak akan error jika banana tidak ada di set my_set.remove("banana") print(my_set)</pre>
	Output: {1, 2, 3.14, 3}
Menggabungkan <i>set</i> (<i>union</i>)	
[3]:	<pre>set_1 = {1, 2, 3} set_2 = {3, 4, 5} union_set = set_1 set_2 print(union_set)</pre>
	Output: {1, 2, 3, 4, 5}
Irisan <i>set</i> (<i>intersection</i>)	
[4]:	<pre>set_1 = {1, 2, 3} set_2 = {3, 4, 5} intersection_set = set_1 & set_2 print(intersection_set)</pre>
	Output: {3}
Selisih <i>set</i> (<i>difference</i>)	
[5]:	<pre>set_1 = {1, 2, 3} set_2 = {3, 4, 5} difference_set = set_1 - set_2 print(difference_set)</pre>
	Output: {1, 2}

Selisih simetris (symmetric difference)	
[6]:	<pre>set_1 = {1, 2, 3} set_2 = {3, 4, 5} symmetric_difference_set = set_1 ^ set_2 print(symmetric_difference_set)</pre>
	Output: {1, 2, 4, 5}
Memeriksa keanggotaan	
[7]:	<pre>set_1 = {1, 2, 3} if 3 in set_1 : print("3 ada dalam set_1") else : print("3 tidak ada dalam set_1")</pre>
	Output: 3 ada dalam set_1
Menghitung jumlah elemen	
[8]:	<pre>set_1 = {1, 2, 3} print(len(set_1))</pre>
	Output: 3

F. Tipe Data Boolean

Boolean adalah tipe data dalam Python yang hanya memiliki dua nilai: `True` dan `False`. tipe data *Boolean* digunakan dalam operasi logika dan kondisi untuk mengatur alur program, seperti dalam pernyataan `if` dan `while`. `True` dan `False` juga direpresentasikan dengan nilai `1` dan `0`. Berikut adalah contoh penggunaan *Boolean*:

Nilai Boolean sederhana	
[1]:	<pre>is_student = True informatics_student = False print(is_student, informatics_student)</pre>
	Output: True False
Operasi perbandingan	
[2]:	<pre>x = 10 y = 20 print(x < y) # True print(x > y) # False</pre>
	Output: True False
Operasi logika	
[3]:	<pre>a = True</pre>

	<pre> b = False print(a and b) # False print(a or b) # True print(not a) # False </pre>
	<pre> Output: False True False </pre>
Menggunakan <i>Boolean</i> dalam kondisi	
[4]:	<pre> is_raining = True if is_raining: print("Bawa payung!") else: print("Tidak perlu payung.") </pre>
	<pre> Output: Bawa payung! </pre>

1.3. *Input dan Output* Pada Python

A. *Input* Pada Python

`input()` adalah fungsi bawaan dalam Python yang digunakan untuk menerima masukan dari pengguna. Saat fungsi `input()` dipanggil, eksekusi program akan terhenti dan menunggu pengguna memasukkan data. Setelah pengguna mengetikkan data dan menekan `Enter`, data tersebut akan dikembalikan sebagai **string** oleh fungsi `input()`. Berikut adalah cara menggunakan `input()`:

[1]:	<pre> # Meminta pengguna memasukkan nama name = input("Masukkan nama Anda: ") print("Halo,", name) </pre>
	<pre> Output: Masukkan nama Anda: Fizz Halo, Fizz </pre>
[2]:	<pre> # Meminta dan mengonversi input dari pengguna age = int(input("Masukkan usia Anda: ")) # Casting ke integer print("Tahun depan, usia Anda akan menjadi:", age+1) </pre>
	<pre> Output: Masukkan usia Anda: 21 Tahun depan, usia Anda akan menjadi 22 </pre>

B. *Output* Pada Python

Output pada Python merujuk pada proses menampilkan data atau informasi kepada pengguna, biasanya melalui terminal. Fungsi utama yang digunakan *output* dalam Python adalah `print()`. Fungsi ini memungkinkan Python untuk menampilkan teks, angka, atau nilai dari variable. Berikut adalah contoh penggunaan `print()`:

Penggunaan <code>print()</code> sederhana	
[1]:	<pre>print("Hello, World!") name = "Fizz" print("Nama saya adalah", name)</pre>
	Output: Hello, World! Nama saya adalah Fizz
Penggunaan Format <code>%</code>	
[2]:	<pre>name = "Fizz" age = 21 print("Nama saya adalah %s dan saya berumur %d tahun." % (name, age)) # %s : digunakan untuk menyisipkan string # %d : digunakan untuk menyisipkan integer # %f : digunakan untuk menyisipkan float</pre>
	Output: Nama saya adalah Fizz dan saya berumur 21 tahun.
Menggunakan metode <code>str.format()</code>	
[3]:	<pre>name = "Fizz" age = 21 print("Nama saya adalah {} dan saya berumur {} tahun.".format(name, age))</pre>
	Output: Nama saya adalah Fizz dan saya berumur 21 tahun.
Menggunakan F-Strings	
[4]:	<pre>name = "Fizz" age = 21 print(f"Nama saya adalah {name} dan saya berumur {age} tahun.")</pre>
	Output: Nama saya adalah Fizz dan saya berumur 21 tahun.

1.4. Range Statement

`range` adalah fungsi bawaan Python yang digunakan untuk menghasilkan deret angka secara bertahap. `range` umumnya digunakan dalam *loop* seperti `for` untuk mengiterasi melalui urutan angka. Fungsi ini digunakan ketika memerlukan sesuatu berulang kali untuk sejumlah iterasi tertentu. Berikut adalah contoh penggunaan `range`:

<code>range(start, stop, step)</code>
<code>start</code> : (opsional) Angka awal. <i>Default</i> adalah 0.
<code>stop</code> : Angka akhir (tidak termasuk dalam hasil).
<code>step</code> : (opsional) Besarnya langkah antar angka. <i>Default</i> adalah 1.

Berikut adalah contoh penggunaan `range`:

Menggunakan hasil `range` sebagai list	
[1]:	<pre>numbers = list(range(10)) print(numbers)</pre>
	Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Penggunaan `range` dalam `for`	
[2]:	<pre>fruits = ["apple", "orange", "tomato", "papaya", "pear"] for i in range(0, len(fruits), 2) : print(fruits[i])</pre>
	Output: apple tomato pear
`range` dengan langkah negatif	
[3]:	<pre>for i in range(3, 0, -1) : print(i)</pre>
	Output: 3 2 1

1.5. Percabangan Pada Python

Percabangan dalam Python adalah struktur kontrol yang memungkinkan program untuk membuat keputusan berdasarkan kondisi tertentu. Dengan menggunakan percabangan, program dapat mengeksekusi blok kode tertentu jika kondisi yang diberikan bernilai `True`, atau mengambil jalur alternatif jika kondisi tersebut `False`.

1.5.1. Percabangan `if`

`if` digunakan untuk mengeksekusi blok kode hanya jika kondisi yang diberikan bernilai `True`. Berikut adalah sintaks dan contoh penggunaan *statement* `if`:

Sintaks `if`	
<pre>if kondisi : # Kode yang dieksekusi jika kondisi True</pre>	
Contoh 1	
[1]:	<pre>x = 10 if x > 5 : print("x lebih besar dari 5")</pre>
	Output: x lebih besar dari 5
Contoh 2	
[2]:	<pre>x = 10 if x > 3 : print("x lebih besar dari 3")</pre>

	<pre>if x > 5 : print("x lebih besar dari 5")</pre>
	Output: x lebih besar dari 3 x lebih besar dari 5

1.5.2. Percabangan `if-else`

`else` digunakan untuk mengeksekusi blok kode jika semua kondisi `if` dan `elif` di atasnya bernilai `False`. `else` harus berada di akhir blok percabangan. Berikut adalah implementasi dan contoh percabangan `if-else`:

Sintaks `if-else`	
<pre>if kondisi : # Kode yang dieksekusi jika kondisi True else : # Kode yang dieksekusi jika kondisi False</pre>	
Contoh 1	
[1]:	<pre>x = 10 if x < 5 : print("x kurang dari 5") else : print("x lebih besar atau sama dengan 5")</pre>
	Output: x lebih besar atau sama dengan 5
Contoh 2	
[2]:	<pre>x = 10 if x < 5 : print("x kurang dari 5") elif x < 7 : print("x kurang dari 7") else : print("x lebih besar atau sama dengan 7")</pre>
	Output: x lebih besar atau sama dengan 7

1.5.3. Percabangan `if-elif-else`

`elif` *statement* (singkatan dari "*else if*") digunakan untuk memeriksa kondisi lain jika kondisi sebelumnya bernilai `False`. Pernyataan `elif` dapat dibuat lebih dari satu. Berikut adalah sintaks dan contoh penggunaan *statement* `if-elif-else`:

Sintaks `if-elif-else`	
<pre>if kondisi_1 : # Kode yang dieksekusi jika kondisi_1 True elif kondisi_2 : # Kode yang dieksekusi jika kondisi_2 True else :</pre>	

# Kode yang dieksekusi jika kondisi_1 dan kondisi_2 False	
Contoh 1	
[1]:	<pre>x = 10 if x < 5 : print("x kurang dari 5") else : print("x lebih besar atau sama dengan 5")</pre>
	Output: x lebih besar atau sama dengan 5
Contoh 2	
[2]:	<pre>x = 8 if x < 5 : print("x kurang dari 5") elif x < 7 : print("x kurang dari 7") elif x < 9 : print("x kurang dari 9") else : print("x lebih besar atau sama dengan 7")</pre>
	Output: x kurang dari 9

1.6. Perulangan *While*

Perulangan `while` pada Python adalah struktur kontrol yang memungkinkan kode untuk dieksekusi berulang kali selama suatu kondisi tertentu tetap terpenuhi. Perulangan ini terus berjalan selama kondisi yang ditentukan bernilai `True`, dan akan berhenti begitu kondisi tersebut bernilai `False`. Berikut adalah sintaks dan contoh perulangan `while`:

Sintaks <code>while</code>	
<pre>while kondisi : # Blok kode yang akan dieksekusi selama kondisi True</pre>	
Contoh 1	
[1]:	<pre>count = 1 while count <= 3 : print(count, end=' ') count += 1</pre>
	Output: 1 2 3
Contoh 2	
[2]:	<pre>number = 10 while number > 0: number -= 1 if number == 5 :</pre>

	<pre>print("Berhenti pada 5") break # Menghentikan loop secara paksa</pre>
	Output: Berhenti pada 5

1.7. Perulangan *For*

Perulangan `for` pada Python adalah struktur kontrol yang digunakan untuk mengiterasi atau mengulang eksekusi blok kode berdasarkan urutan elemen dari suatu iterable, seperti *list*, *tuple*, *string*, atau *range*. Berikut adalah sintaks dan contoh perulangan `for`:

Sintaks <code>for</code>	
<pre>for elemen in iterable : # Blok kode yang akan dieksekusi selama kondisi True</pre>	
Iterasi pada list	
[1]:	<pre>fruits = ["apple", "banana", "cherry"] for fruit in fruits : print(fruit)</pre>
	Output: 'apple' 'banana' 'cherry'
Iterasi pada string	
[2]:	<pre>for char in "DKA" : print(char)</pre>
	Output: D K A
Penggunaan <code>range()</code> dalam <code>for</code>	
[3]:	<pre>for i in range(5) : print(i)</pre>
	Output: 0 1 2 3 4
[4]:	<pre>for I in range(1, 50, 10): print(i)</pre>
	Output: 1 11 21 31 41

1.8. Fungsi

Fungsi dalam Python adalah blok kode yang dirancang untuk melakukan tugas tertentu dan dapat dipanggil kapan saja dalam program. Penggunaan fungsi memungkinkan untuk mengelompokkan kode yang sering digunakan atau kode yang kompleks menjadi unit yang lebih kecil dan lebih mudah dikelola.

Untuk mendefinisikan fungsi dalam Python, digunakan kata kunci `def`, diikuti dengan nama fungsi, tanda kurung `()` yang dapat berisi parameter, dan titik dua `:`. Kode blok yang termasuk pada fungsi harus dimasukkan dengan indentasi. Berikut adalah sintaks dan contoh dari fungsi dalam Python:

Sintaks fungsi	
	<pre>def nama_fungsi(parameter1, parameter2, ...) : # Blok kode fungsi return hasil # (opsional) Mengembalikan nilai hasil</pre>
Fungsi tanpa parameter	
[1]:	<pre>def hello_world(): print("Hello, World!") # memanggil fungsi hello_world()</pre>
	Output: Hello, World!
Fungsi dengan parameter	
[2]:	<pre>def greet(name): print(f"Halo, {name}!") greet("Fizz")</pre>
	Output: Halo, Fizz!
Fungsi dengan return value	
[3]:	<pre>def add(a, b): return a + b result = add(5,3) print(result)</pre>
	Output: 8

1.8.1. Parameter dan Argumen Pada Fungsi

Parameter adalah variable yang didefinisikan dalam tanda kurung saat mendeklarasikan fungsi. Argumen adalah nilai aktual yang diberikan ke parameter saat fungsi dipanggil. Berikut adalah contoh penggunaan parameter dan argumen pada fungsi:

Penggunaan parameter dan argumen
<pre>def subtract(x, y): # x dan y adalah parameter return x - y</pre>

<pre>result = subtract(10, 3) # 10 dan 3 adalah argument print(result)</pre>	
Fungsi dengan parameter default Parameter default adalah parameter yang memiliki nilai default, yang digunakan jika tidak ada argumen yang diberikan saat fungsi dipanggil.	
[1]:	<pre># name merupakan positional argument, message merupakan keyword argument def greet(name, message="Selamat datang!"): print(f"{message}, {name}") greet("Fizz") # Menggunakan nilai untuk default message greet("Buzz", "Halo") # Menimpa nilai default untuk message</pre>
	Output: Selamat dataing, Fizz Halo, Buzz
Fungsi dengan parameter args Parameter args digunakan untuk menerima sejumlah argumen posisi yang tidak terbatas.	
[2]:	<pre>def sum(*args): return sum(args) print(sum(1, 2, 3, 4, 5))</pre>
	Output: 15
Fungsi dengan parameter kwargs Parameter kwargs digunakan untuk menerima sejumlah <i>keyword argument</i> yang tidak terbatas.	
[3]:	<pre>def print_info(**kwargs): for key, value in kwargs.items(): print(f"{key}: {value}") print_info(nama="Fizz", umur=21, universitas="Telkom University")</pre>
	Output: nama: Fizz umur: 21 universitas: Telkom University

1.8.2. Fungsi Anonim

Python juga mendukung fungsi anonim dengan kata kunci ``lambda``, yang merupakan fungsi kecil yang didefinisikan tanpa nama. *Lambda function* biasanya digunakan untuk operasi sederhana. Berikut adalah implementasi dan contoh fungsi anonim:

[1]:	<pre># Fungsi lambda untuk menambahkan dua angka add = lambda x, y: x + y print(add(3,4))</pre>
	Output: 7

1.9. Pemrograman Modular

Pemrograman modular adalah pendekatan pemrograman di mana sebuah program dibagi menjadi bagian-bagian kecil yang disebut **modul**. Setiap modul adalah bagian

terpisah dari kode yang menangani fungsi tertentu dalam program. Dengan memisahkan kode menjadi modul-modul yang lebih kecil dan terorganisir, program menjadi lebih mudah dipahami, dikelola, dan dikembangkan. Berikut adalah cara menerapkan pemrograman modular pada Python:

File `matematika.py`	
[1]:	<pre>def tambah(a, b): return a + b def kali(a, b): return a * b</pre>
File `main.py`	
[2]:	<pre>import matematika # Import seluruh fungsi dalam matematika.py # Menggunakan fungsi dari matematika.py hasil_tambah = matematika.tambah(5, 3) hasil_kali = matematika.kali(4, 2) print("Hasil tambah:", hasil_tambah) print("Hasil kali:", hasil_kali)</pre>
	Output: <pre>8 8</pre>

1.10. Soal Latihan Pemrograman Python

1. Buatlah fungsi (*function*) *isKabisat* dalam Python, yaitu fungsi untuk menentukan apakah suatu bilangan tahun merupakan tahun kabisat atau bukan. *Input* dari fungsi ini adalah *integer* (bilangan tahun), output dari fungsi ini adalah *boolean* (`True` bila kabisat, `False` bila bukan kabisat). Selanjutnya panggil fungsi tersebut dan periksa hasilnya.
2. Buatlah sebuah fungsi dan sebuah prosedur untuk mencetak bilangan *Fibonacci* menggunakan Python! Untuk *input* `n`, maka fungsi ini mereturn angka *Fibonacci* ke-n, sedangkan prosedurnya akan mencetak barisan bilangan *Fibonacci* mulai dari suku ke 1 hingga suku ke n. Selanjutnya panggil fungsi dan prosedur tersebut dan periksa hasilnya.

Modul 2 PENGENALAN *LIBRARY* NUMPY

Tujuan Praktikum:

1. Mahasiswa memahami cara mengelola dan memproses komputasi *array* dengan *library* NumPy.
2. Mahasiswa mengetahui fitur-fitur utama yang tersedia di *library* Numpy.

REFERENSI:

<https://numpy.org/doc/2.0/>

NumPy adalah *library* fundamental dalam bahasa pemrograman Python yang digunakan untuk melakukan komputasi numerik dengan efisiensi tinggi. *Library* ini menyediakan dukungan untuk *array* multidimensi, yang memungkinkan pengolahan data secara lebih cepat dan efisien dibandingkan dengan *list* Python biasa. NumPy juga dilengkapi dengan berbagai fungsi matematika dan operasi aljabar linier yang sangat berguna dalam analisis data, pemrosesan gambar, serta pemodelan kecerdasan buatan dan *machine learning*. Kemampuan NumPy untuk melakukan operasi vektor dan matriks secara optimal menjadikannya alat yang esensial bagi pengembang dalam mengelola dan menganalisis data skala besar.

2.1. Cara Import NumPy

Numpy seharusnya sudah terdapat pada instalasi Anaconda. Untuk menggunakan modul NumPy, harus dilakukan *import* terlebih dahulu. Berikut adalah cara standar untuk mengimport *library* Numpy:

```
import numpy as np
```

2.2. Definisi Array dan Array NumPy

Array adalah struktur data yang menyimpan sejumlah elemen yang dimiliki tipe data yang sama secara berurutan dalam memori. Pada Python, *array* biasa dapat dideklarasikan menggunakan *list* dengan sintaks seperti `x = [1, 2, 3]`, di mana elemen – elemen *array* ditempatkan dalam tanda kurung siku `[ ]` dan dipisahkan oleh koma. *Array* dapat juga dideklarasikan dengan menggunakan *library* NumPy, perbedaan deklarasi *array* biasa dengan *array* NumPy dapat dilihat pada Tabel 3.

Tabel 3. Perbedaan array pada Python dan NumPy Array.

Aspek	Array Biasa	Array NumPy
Deklarasi	<code>x = [1, 2, 3]</code>	<code>import numpy as np</code> <code>x = np.array([1, 2, 3])</code>
Tipe Data	Heterogen (dapat menyimpan berbagai tipe data)	Homogen (semua elemen harus memiliki tipe data yang sama)
Dimensi	1 dimensi (dapat menampung <i>nested lists</i> untuk menyimpan lebih banyak dimensi)	Mendukung banyak dimensi (1D, 2D, 3D, dll) langsung
Operasi Matematika	Tidak mendukung operasi langsung (<code>[1, 2, 3] + [4, 5, 6]</code> akan menggabungkan <i>list</i>)	Mendukung operasi langsung (<code>np.array([1, 2, 3]) + np.array([4, 5, 6])</code> menghasilkan <code>[5, 7, 9]</code>)
Penggunaan Memori	Kurang efisien (karena fleksibilitas tipe data)	Lebih efisien (dengan alokasi memori tetap dan tipe data tunggal)
Fungsionalitas	Dasar, terutama untuk operasi non-numerik	Mendukung fungsi matematika, aljabar linear, statistik, dll.
<i>Slicing</i>	Mendukung <i>slicing</i> dasar (<code>x[0:2]</code> akan menghasilkan <code>[1, 2]</code>)	Mendukung <i>slicing</i> dengan fitur tambahan seperti <code>x[:, 1:3]</code> untuk array multidimensi
<i>Broadcasting</i>	Tidak mendukung	Mendukung operasi antara <i>array</i> dengan ukuran berbeda selama kompatibel
Penambahan Elemen	Mudah, menggunakan <code>append()</code> atau <code>+</code>	Perlu menggunakan metode seperti <code>np.append()</code> atau perlu membuat <i>array</i> baru

2.3. Membuat Program dengan *Library* Numpy

Berikut merupakan implementasi penggunaan *array* dengan menggunakan NumPy. Sebagai catatan **pastikan mengeksekusi baris `import numpy as np` setiap kali anda memulai sesi baru (run time) atau script Python yang memerlukan penggunaan library NumPy.**

Operasi aritmatika penjumlahan pada elemen <i>array</i> NumPy menggunakan tanda <code>+</code>	
[1]:	<pre>import numpy as np x = np.array([1, 2, 3, 4]) y = np.array([5, 6, 7, 8]) z = x + y print(z) print(z.tolist()) # Gunakan sintaks <code>tolist()</code> untuk menampilkan tiap elemen yang disahkan dengan tanda koma (,)</pre>
	Output: <pre>[6 8 10 12]</pre>

	[6, 8, 10, 12]
Penggabungan 2 array yang berbeda menggunakan sintaks <code>`concatenate`</code>	
[2]:	<pre> x = np.array([1, 2, 3, 4]) y = np.array([5, 6, 7, 8]) z = np.concatenate((x, y)) print(c) </pre>
	Output: [1 2 3 4 5 6 7 8]

2.4. Atribut Array NumPy

Atribut dalam *array* NumPy adalah properti bawaan yang menyediakan informasi penting mengenai struktur, ukuran, tipe data, dan bentuk *array*. Atribut – atribut ini memungkinkan anda untuk dengan mudah mengakses dan memanipulasi karakteristik fundamental dari *array* tanpa perlu melakukan operasi manual yang kompleks.

2.4.1. Atribut ndim

Atribut ``ndim`` menunjukkan jumlah dimensi atau level dalam *array* tersebut. Ini adalah salah satu atribut dasar yang membantu memahami struktur data yang sedang dilakukan pengoperasian program. Sebagai contoh, *array* 1D memiliki ``ndim`` sebesar 1, *array* 2D memiliki ``ndim`` sebesar 2, dan seterusnya. Struktur dimensi pada *array* NumPy dijelaskan pada Tabel 4.

Tabel 4. Struktur dimensi array NumPy.

Jumlah Dimensi	Contoh Struktur Array	Deskripsi
1D	<code>`[1, 2, 3]`</code>	Array satu dimensi atau vektor, berisi elemen-elemen dalam satu baris.
2D	<code>`[[1, 2, 3], [4, 5, 6]]`</code>	Array dua dimensi atau matriks, berisi elemen-elemen dalam baris dan kolom.
3D	<code>`[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]`</code>	Array tiga dimensi, berisi data dalam bentuk kubus dengan kedalaman, baris, dan kolom.
4D	<code>`[[[ [1], [2] ], [3], [4] ], [ [5], [6] ], [ [7], [8] ] ]]`</code>	Array empat dimensi, biasanya digunakan untuk data dengan banyak saluran (misalnya <i>batch</i> gambar dalam model <i>deep learning</i>).

Berikut merupakan contoh penggunaan sintaks ``ndim``:

Contoh 1	
[1]:	<pre>import numpy as np x = np.array([85, 90, 78, 92, 88]) print("Dimensi array x:", x.ndim)</pre>
	Output: Dimensi array x: 1
Contoh 2	
[2]:	<pre>x = np.array([[30, 32, 31], [29, 33, 30], [28, 34, 29]]) print("Dimensi array x:", x.ndim)</pre>
	Output: Dimensi array x: 2

2.4.2. Atribut shape

Atribut ``shape`` menunjukkan dimensi atau ukuran *array* dalam bentuk *tuple*. Ini memberikan informasi tentang jumlah elemen di setiap sumbu *array*. Kegunaannya adalah untuk memastikan dimensi *array* sesuai sebelum operasi matematika, manipulasi data, atau pemrosesan lebih lanjut, seperti memasukkan data ke dalam model kecerdasan buatan. Berikut contoh penggunaan atribut ``shape``. Berikut adalah implementasi dan contoh atribut `shape` pada *array* NumPy:

Contoh 1	
[1]:	<pre>import numpy as np x = np.array([[1, 2, 3], [4, 5, 6]]) print("Shape x:", x.shape)</pre>
	Output: Shape x: (2, 3)
Penjelasan:	
Atribut <code>`shape`</code> mengembalikan (2, 3), yang berarti <i>array</i> memiliki 2 baris dan 3 kolom.	
Contoh 2	
[2]:	<pre>x = np.array([[1, 2, 3], [4, 5, 6]]) y = np.array([[7, 8, 9], [10, 11, 12]], [[13, 14, 15], [16, 17, 18]]) # Cek apakah x dan y memiliki shape yang sama if x.shape == y.shape: print("Array x dan y memiliki shape yang sama.") else: print("Array x dan y memiliki shape yang berbeda.") print("Shape x:", x.shape) # Output: (2, 3) print("Shape y:", y.shape) # Output: (2, 2, 3)</pre>
	Output: Array x dan y memiliki shape yang berbeda. Shape x: (2, 3) Shape y: (2, 2, 3)

Penjelasan:

``x`` adalah *array* 2D dengan bentuk (2, 3), yang berarti 2 baris dan 3 kolom.

``y`` adalah *array* 3D dengan bentuk (2, 2, 3), yang berarti memiliki 2 matriks, masing-masing berisi 2 baris dan 3 kolom.

Matriks pertama yaitu ``[[7, 8, 9], [10, 11, 12]]``

Matriks kedua yaitu ``[[13, 14, 15], [16, 17, 18]]``

2.4.3. Atribut size

Atribut ``size`` menunjukkan jumlah total elemen yang ada dalam *array*, tanpa memperhatikan dimensi atau bentuknya. Kegunaan utama dari atribut ini adalah untuk mengetahui ukuran keseluruhan *array*. Atribut ``size`` membantu memastikan bahwa jumlah elemen dalam *array* sesuai dengan yang diharapkan, terutama saat melakukan operasi seperti ``reshape`` atau mengalokasikan memori untuk data yang sangat besar. Berikut adalah implementasi dan contoh atribut `size` dalam NumPy:

Contoh 1

```
[1]: import numpy as np

x = np.array([[1, 2, 3], [4, 5, 6]])
print("Size x:", x.size)
```

Output:
Size x: 6

Penjelasan:

Atribut ``size`` mengembalikan 6, yang menunjukkan bahwa terdapat 6 elemen total dalam *array* ``x`` ini.

Contoh 2

```
[2]: x = np.array([1, 2, 3, 4, 5, 6])
y = np.array([[5, 6], [7, 8], [9, 10]])

# Membandingkan ukuran kedua array
if x.size == y.size:
    print("Kedua array memiliki jumlah elemen yang sama.")
else:
    print("Kedua array memiliki jumlah elemen yang berbeda.")
```

Output:
Kedua array memiliki jumlah elemen yang sama.

Penjelasan:

Atribut ``size`` membantu dalam membandingkan apakah dua *array* memiliki jumlah elemen yang sama.

Dalam contoh ini, ``x`` memiliki 6 elemen dan `y` memiliki 6 elemen, sehingga hasilnya adalah "Kedua array memiliki jumlah elemen yang sama."

2.4.4. Atribut dtype

Atribut ``dtype`` menunjukkan tipe data dari elemen-elemen dalam *array*, seperti *integer*, *float*, atau *string*. Kegunaannya adalah untuk memastikan elemen *array* memiliki tipe data yang sesuai dengan kebutuhan pemrosesan, mempengaruhi operasi matematis, alokasi memori, dan mencegah kesalahan tipe data saat bekerja dengan *array* besar. Berikut adalah contoh dan implementasi untuk memeriksa tipe data dari elemen dalam *array*:

Contoh 1 (Memeriksa tipe data suatu array)	
[1]:	<pre>import numpy as np # Array dengan tipe data integer x = np.array([1, 2, 3]) print("Tipe data x:", x.dtype) # Array dengan tipe data float y = np.array([1.1, 2.2, 3.3]) print("Tipe data y:", y.dtype)</pre>
	Output: Tipe data x: int64 Tipe data y: float64
Contoh 2 (Memastikan tipe data saat membuat array)	
[2]:	<pre># Membuat array dengan tipe data string x = np.array(["apple", "banana", "cherry"], dtype=np.str_) print("Tipe data x:", x.dtype) # Membuat array dengan tipe data float 64-bit y = np.array([1.1, 2.2, 3.3], dtype=np.float64) print("Tipe data y:", y.dtype)</pre>
	Output: Tipe data x: <U6 Tipe data y: float64
Penjelasan: Menentukan tipe data <code>dtype</code> saat membuat array untuk memastikan elemen <i>array</i> <code>x</code> adalah <i>string</i> . Di sini, <code><U6</code> menunjukkan <i>string Unicode</i> dengan panjang maksimum 6 karakter.	

2.5. Deklarasi Array dengan NumPy

Deklarasi *array* dengan NumPy memungkinkan pembuatan *array* multidimensi dengan berbagai inisialisasi yang sesuai dengan kebutuhan analisis data. Dengan fungsi seperti ``np.zeros()``, ``np.ones()``, ``np.empty()``, ``np.arange()``, ``np.linspace()``, ``np.full()``, ``np.random.random()``, ``np.random.randint()``, dan ``np.fromfunction()``, dapat dengan mudah membuat *array* dengan nilai default, urutan tertentu, atau nilai yang terdistribusi secara

merata. Kegunaan utama dari metode ini adalah untuk mempersiapkan data secara efisien sebelum pemrosesan lebih lanjut. Berikut merupakan beberapa cara untuk deklarasi *array* dengan NumPy:

Sintaks <code>np.zeros()</code>	
[1]:	<pre>x = np.zeros((2, 3)) print(x)</pre>
	Output: [[0. 0. 0.] [0. 0. 0.]]
Penjelasan: <code>np.zeros()</code> membuat <i>array</i> dengan bentuk (2, 3) dan mengisi semua elemen dengan nilai nol.	
Sintaks <code>np.ones()</code>	
[2]:	<pre>x = np.ones((3, 4)) print(x)</pre>
	Output: [[1. 1. 1. 1.] [1. 1. 1. 1.] [1. 1. 1. 1.]]
Penjelasan: <code>np.ones()</code> membuat <i>array</i> dengan bentuk (3, 4) dan mengisi semua elemen dengan nilai satu.	
Sintaks <code>np.empty()</code>	
[3]:	<pre>x = np.empty((2, 2)) print(x)</pre>
	Output: [[7.34 2.4] [6.3 2.162]]
Penjelasan: <code>np.empty()</code> membuat <i>array</i> dengan bentuk (2, 2) tetapi tidak menginisialisasi nilai-nilainya. Nilai-nilai tersebut akan berupa data acak yang sebelumnya ada di memori. Ini lebih cepat daripada <code>np.zeros()</code> atau <code>np.ones()</code> .	
Sintaks <code>np.arange()</code>	
[4]:	<pre>x = np.arange(0, 10, 2) print(x)</pre>
	Output: [0 2 4 6 8]
Penjelasan: <code>np.arange()</code> membuat <i>array</i> dengan urutan nilai mulai dari 0 hingga kurang dari 10 dengan interval 2. Ini berguna untuk menghasilkan rentang nilai dengan interval tertentu.	
Sintaks <code>np.linspace()</code>	
[5]:	<pre>x = np.linspace(0, 1, 5) print(x)</pre>
	Output: [0. 0.25 0.5 0.75 1.]

Penjelasan: <code>`np.linspace()`</code> membuat <i>array</i> dengan 5 nilai yang terdistribusi secara merata dari 0 hingga 1. Ini berguna untuk membuat data dengan interval yang sama dalam rentang yang ditentukan.	
Sintaks <code>`np.full()`</code>	
[6]:	<pre>x = np.full((2, 2), 7) print(x)</pre>
	Output: <pre>[[7 7] [7 7]]</pre>
Sintaks <code>`np.random.random()`</code>	
[7]:	<pre>x = np.random.random((2, 3)) print(x)</pre>
	Output <pre>[[0.123 0.456 0.789] [0.012 0.345 0.678]]</pre>
Penjelasan: <code>`np.random.random()`</code> membuat <i>array</i> dengan nilai acak yang terdistribusi secara uniform antara 0 dan 1. Ini berguna untuk membuat data acak untuk simulasi atau pengujian.	
Sintaks <code>`np.random.randint()`</code>	
[8]:	<pre>x = np.random.randint(0, 10, size=(3, 4)) print(x)</pre>
	Output: <pre>[[2 5 7 1] [8 9 0 3] [4 2 6 1]]</pre>
Penjelasan: <code>`np.random.randint()`</code> membuat <i>array</i> dengan nilai acak dalam rentang yang ditentukan (0 hingga 10) dengan bentuk (3, 4). Ini sering digunakan untuk menghasilkan data acak dalam rentang <i>integer</i> tertentu.	
Sintaks <code>`np.fromfunction()`</code>	
[9]:	<pre>def penjumlahan(bilangan1, bilangan2): return bilangan1 + bilangan2 x = np.fromfunction(penjumlahan, (3, 3), dtype=int) print(x)</pre>
	Output: <pre>[[0 1 2] [1 2 3] [2 3 4]]</pre>
Penjelasan: <code>`np.fromfunction()`</code> membuat <i>array</i> dengan bentuk (3, 3) berdasarkan fungsi yang diterapkan pada indeks <i>array</i> . Di sini, fungsi penjumlahan digunakan untuk mengisi nilai-nilai dalam <i>array</i> . Jadi pada indeks [1][2] akan terisi nilai 3. Ini berguna untuk menghasilkan <i>array</i> berdasarkan perhitungan atau formula tertentu.	

2.6. Penambahan Elemen Array

Penambahan elemen dalam *array* NumPy adalah proses penting yang sering digunakan dalam berbagai aplikasi, terutama dalam pemrosesan data dan analisis dalam kecerdasan buatan. Meskipun *array* NumPy memiliki ukuran tetap, ada beberapa metode yang memungkinkan Anda untuk menambahkan elemen baru dengan menciptakan array baru yang mencakup elemen tambahan. Berikut adalah implementasi dan contoh untuk menambahkan elemen pada *array* di NumPy:

Sintaks <code>`np.append()`</code>	
[1]:	<pre># append pada array 1D a = np.array([1, 2, 3]) a = np.append(a, [4, 5]) print("1D array setelah append:\n", a) # append pada array 2D x = np.array([[1, 2], [3, 4]]) x = np.append(x, [[5, 6]], axis=0) print("2D array setelah append:\n", x)</pre>
	Output: 1D array setelah append: [1 2 3 4 5] 2D array setelah append: [[1 2] [3 4] [5 6]]
Sintaks <code>`np.concatenate()`</code>	
[2]:	<pre># concatenate pada array 1D a = np.array([1, 2]) b = np.array([3, 4]) c = np.concatenate((a, b)) print("1D array setelah concatenate:\n", c) # concatenate pada array 2D x = np.array([[1, 2], [3, 4]]) y = np.array([[5, 6]]) z = np.concatenate((x, y), axis=0) print("1D array setelah concatenate:\n", z)</pre>
	Output: 1D array setelah concatenate: [1 2 3 4] 2D array setelah concatenate: [[1 2] [3 4]]

	[5 6]]
Sintaks	<code>`np.insert()`</code>
[3]:	<pre> # insert pada array 1D a = np.array([1, 2, 3]) a = np.insert(a, 1, 4) # Insert '4' pada index 1 print("1D array setelah insert:\n", a) # insert pada array 2D x = np.array([[1, 2], [3, 4]]) x = np.insert(x, 0, [5, 6], axis=0) # Insert baris [5, 6] index 0 print("2D array setelah insert:\n", x) </pre>
	Output: <pre> 1D array setelah insert: [1 4 2 3] 2D array setelah insert: [[5 6] [1 2] [3 4]] </pre>
Sintaks	<code>`np.stack()`</code>
[4]:	<pre> # stack pada array 1D a = np.array([1, 2, 3]) b = np.array([4, 5, 6]) c = np.stack((a, b)) print("1D array setelah stack:\n", c) print("Shape dari array c:", c.shape) # stack pada array 2D x = np.array([[1, 2], [3, 4]]) y = np.array([[5, 6], [7, 8]]) z = np.stack((x, y)) print("2D array setelah stack:\n", z) print("Shape dari array z:", z.shape) </pre>
	Output: <pre> 1D array setelah stack: [[1 2 3] [4 5 6]] Shape dari array c: (2, 3) 2D array setelah stack: [[[1 2] [3 4]] [[5 6] [7 8]]] Shape dari array z: (2, 2, 2) </pre>

2.7. Penghapusan Elemen Array

Penghapusan elemen dari *array* NumPy adalah langkah penting dalam pembersihan dan manipulasi data. Penghapusan elemen memungkinkan anda untuk menghapus elemen berdasarkan indeks, kondisi, atau kriteria tertentu. Penggunaan metode ini membantu dalam mengelola data yang tidak relevan atau tidak diinginkan, serta mempersiapkan data untuk analisis lebih lanjut atau pemodelan. Berikut adalah contoh dan implementasi penghapusan elemen pada *array* NumPy:

Sintaks <code>np.delete()</code> untuk menghapus elemen pada <i>array</i> 1D	
[1]:	<pre>x = np.array([1, 2, 3, 4, 5]) # Menghapus elemen pada indeks 2 x = np.delete(x, 2) print(x)</pre>
	Output: [1 2 4 5]
Sintaks <code>np.delete()</code> untuk menghapus baris pada <i>array</i> 2D	
[2]:	<pre>x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Menghapus baris pada indeks 1 (baris kedua) x = np.delete(x, 1, axis=0) print(x)</pre>
	Output: [[1 2 3] [7 8 9]]
Sintaks <code>np.delete()</code> untuk menghapus kolom pada <i>array</i> 2D	
[3]:	<pre>x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Menghapus kolom pada indeks 1 (kolom kedua) x = np.delete(x, 1, axis=1) print(x)</pre>
	Output: [[1 3] [4 6] [7 9]]
Sintaks <code>np.where()</code>	
[4]:	<pre>x = np.array([1, 2, 3, 4, 5]) # Mengganti elemen yang lebih besar dari 3 dengan NaN x = np.where(x > 3, np.nan, x) # Menghapus elemen NaN dari array x = x[~np.isnan(x)] print(x)</pre>
	Output: [1. 2. 3.]

2.8. Pengurutan Elemen Array

Pengurutan *array* NumPy adalah operasi penting dalam analisis data dan pemodelan kecerdasan buatan. Dengan mengurutkan elemen dalam urutan tertentu, Anda dapat mempermudah analisis dan pemrosesan data. NumPy menyediakan fungsi seperti ``np.sort()``, ``array.sort()``, dan ``np.argsort()`` untuk mengurutkan data secara langsung, di tempat, atau mendapatkan indeks urutan elemen. Berikut adalah implementasi dan contoh pengurutan elemen pada *array* NumPy:

Sintaks <code>`np.sort()`</code>	
[1]:	<pre>x = np.array([3, 1, 2]) y = np.sort(x) print("Array yang telah diurutkan:", y)</pre>
	Output: Array yang telah diurutkan: [1 2 3]
Sintaks <code>`array.sort()`</code>	
[2]:	<pre>x = np.array([3, 1, 2]) x.sort() # elemen array diurutkan secara langsung di dalam array asli print("Array yang telah diurutkan:", x)</pre>
	Output: Array yang telah diurutkan: [1 2 3]
Sintaks <code>`np.sort()`</code> untuk Array Multidimensi	
[3]:	<pre>x = np.array([[3, 1, 2], [9, 7, 8]]) y = np.sort(x, axis=0) # Mengurutkan sepanjang kolom print("Array yang telah diurutkan sepanjang kolom:\n", y) z = np.sort(x) # Mengurutkan sepanjang baris print("Array yang telah diurutkan sepanjang baris:\n", z)</pre>
	Output: Array yang telah diurutkan sepanjang kolom: [[3 1 2] [9 7 8]] Array yang telah diurutkan sepanjang baris: [[1 2 3] [7 8 9]]

2.9. Indexing dan Slicing Array

Indexing dan *slicing* adalah teknik penting dalam pengolahan data menggunakan NumPy. *Indexing* memungkinkan akses elemen spesifik dalam *array* berdasarkan indeks, sedangkan *slicing* memungkinkan pengambilan subset dari *array* menggunakan rentang indeks. Keduanya mempermudah manipulasi data, memfilter, dan mengakses informasi

relevan, sangat berguna dalam analisis data dan pemodelan kecerdasan buatan. Berikut adalah implementasi dan contoh *indexing* dan *slicing* pada *array* NumPy:

Indexing Sederhana	
[1]:	<pre>x = np.array([10, 20, 30, 40, 50]) print(x[2])</pre>
	Output: 30
Penjelasan: Indexing sederhana digunakan untuk mengakses elemen tunggal dari <i>array</i> berdasarkan indeksinya. Dalam contoh ini, <code>x[2]</code> mengakses elemen ke-3 dari <i>array</i> <code>x</code> , yaitu nilai 30.	
Slicing	
[2]:	<pre>x = np.array([10, 20, 30, 40, 50]) # Slicing untuk mendapatkan elemen dari indeks 1 sampai 3 subset = x[1:4] print(subset)</pre>
	Output: [20 30 40]
Penjelasan: Slicing digunakan untuk mengambil subset dari <i>array</i> dengan rentang indeks. Dalam contoh ini, <code>x[1:4]</code> mengakses elemen mulai dari indeks 1 hingga 3 (indeks akhir tidak termasuk), menghasilkan <i>array</i> <code>[20, 30, 40]</code> .	
Boolean Indexing	
[3]:	<pre>x = np.array([10, 20, 30, 40, 50]) # Boolean mask untuk memilih elemen yang lebih besar dari 25 mask = x > 25 print("Hasil boolean dari masking:", mask) subset = x[mask] print("Hasil elemen:", subset)</pre>
	Output: Hasil boolean dari masking: [False False True True True] Hasil elemen: [30 40 50]
Penjelasan: Penjelasan: Boolean indexing menggunakan <i>array</i> boolean (mask) untuk memilih elemen yang memenuhi kondisi tertentu. Di sini, <code>x > 25</code> menghasilkan mask <code>[False, False, True, True, True]</code> , yang kemudian digunakan untuk memilih elemen-elemen yang lebih besar dari 25.	
Fancy Indexing	
[4]:	<pre>x = np.array([10, 20, 30, 40, 50]) # Array indeks indeks = [1, 3] subset = x[indeks] print(subset)</pre>
	Output: [20 40]
Penjelasan:	

Fancy indexing mengakses elemen array menggunakan <i>array</i> indeks. Dalam contoh ini, <code>x[indeks]</code> memilih elemen pada indeks 1 dan 3 dari array <code>x</code> , yaitu 20 dan 40.	
Multi-Dimensional Indexing	
[5]:	<pre>x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Mengakses elemen pada baris ke-1 dan kolom ke-2 element = x[1, 2] print(element)</pre>
	Output: 6
Penjelasan: Multi-dimensional indexing digunakan untuk mengakses elemen dalam <i>array</i> multidimensi. Di sini, <code>x[1, 2]</code> mengakses elemen yang berada di baris ke-1 dan kolom ke-2 dari <i>array</i> 2D, yaitu 6.	
Ellipsis `...`	
[6]:	<pre>x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Mengakses semua elemen pada baris ke-1 subset = x[1, ...] print(subset)</pre>
	Output: [4, 5, 6]
Penjelasan: <code>x[1, ...]</code> mengakses seluruh elemen pada baris kedua dari <i>array</i> 2D <code>x</code> , menghasilkan <i>array</i> <code>[4, 5, 6]</code> . Penggunaan <i>Ellipsis</i> (<code>...</code>) di sini setara dengan <code>x[1, :]</code> dan tidak memberikan perbedaan karena <i>array</i> hanya memiliki dua dimensi.	
Sintaks <code>np.ix_()</code>	
[7]:	<pre>x = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]) # Indeks baris dan kolom rows = [0, 2] cols = [1, 2] subset = arr[np.ix_(rows, cols)] print(subset)</pre>
	Output [[2 3] [8 9]]
Penjelasan: <code>np.random.random()</code> membuat <i>array</i> dengan nilai acak yang terdistribusi secara <i>uniform</i> antara 0 dan 1. Ini berguna untuk membuat data acak untuk simulasi atau pengujian.	

2.10. Penyalinan (Copy) Array

Menyalin array NumPy memungkinkan manipulasi data tanpa memengaruhi array asli. Ada dua jenis penyalinan: *shallow copy* dan *deep copy*. *Shallow copy* (seperti menggunakan `view()` atau *slicing*) membuat salinan yang berbagi data dengan *array* asli, sehingga perubahan pada salah satu akan memengaruhi yang lain. Sebaliknya, *deep*

copy (menggunakan `copy()`) membuat salinan independen, sehingga perubahan pada salinan tidak memengaruhi *array* asli. Memilih jenis *copy* yang tepat penting untuk mengelola data secara efektif dalam berbagai operasi. Berikut adalah implementasi dan contoh penyalinan *array* pada Numpy:

Sintaks <code>view()</code>	
[1]:	<pre> x = np.array([1, 2, 3, 4, 5]) # Membuat shallow copy menggunakan view() y = x.view() # Mengubah elemen pertama dari shallow copy y[0] = 99 print("Array asli:", x) print("Shallow copy:", y) </pre>
	Output: Array asli: [99 2 3 4 5] Shallow copy: [99 2 3 4 5]
Penjelasan: Dalam contoh ini, <i>array</i> 'y' adalah shallow copy dari <i>array</i> 'x', dibuat menggunakan <code>view()</code>. Saat elemen pertama di <i>array</i> 'y' diubah, perubahan tersebut juga tercermin di <i>array</i> 'x', karena keduanya berbagi data yang sama	
Sintaks <code>copy()</code>	
[2]:	<pre> x = np.array([1, 2, 3, 4, 5]) # Membuat deep copy menggunakan copy() y = x.copy() # Mengubah elemen pertama dari deep copy y[0] = 99 print("Array asli:", x) print("Deep copy:", y) </pre>
	Output: Array asli: [1 2 3 4 5] Deep copy: [99 2 3 4 5]
Penjelasan: Di sini, <i>array</i> 'y' adalah deep copy dari <i>array</i> 'x', dibuat menggunakan <code>copy()</code>. Ketika elemen pertama di <i>array</i> 'y' diubah, <i>array</i> asli 'x' tidak terpengaruh, karena <i>deep copy</i> membuat salinan independen yang tidak berbagi data dengan <i>array</i> asli.	

2.11. Operasi Dasar Array

Operasi dasar pada *array* NumPy mencakup perhitungan maksimum, minimum, jumlah total, dan rata-rata dari elemen-elemen *array*. Fungsi-fungsi ini, seperti `max()`, `min()`, `sum()`, dan `mean()`, sering digunakan dalam analisis data dan pemodelan kecerdasan buatan untuk mengevaluasi dan memahami karakteristik utama

dari *dataset*. Dengan operasi dasar ini, anda dapat dengan cepat mendapatkan wawasan tentang distribusi dan skala data yang sedang diolah, sehingga memudahkan dalam langkah-langkah pemrosesan data berikutnya. Berikut adalah implementasi dan contoh operasi dasar pada *array* NumPy:

Sintaks <code>`np.max()`</code>	
[1]:	<pre> x = np.array([1, 5, 3, 8, 2]) max_value = np.max(x) print("Nilai maksimum:", max_value) </pre>
	Output: Nilai maksimum: 8
Sintaks <code>`np.min()`</code>	
[2]:	<pre> x = np.array([1, 5, 3, 8, 2]) min_value = np.min(x) print("Nilai minimum:", min_value) </pre>
	Output: Nilai minimum: 1
Sintaks <code>`np.sum()`</code>	
[3]:	<pre> x = np.array([1, 5, 3, 8, 2]) sum_value = np.sum(x) print("Jumlah total:", sum_value) </pre>
	Output: Jumlah total: 19
Sintaks <code>`np.mean()`</code>	
[4]:	<pre> x = np.array([1, 5, 3, 8, 2]) mean_value = np.mean(x) print("Rata-rata:", mean_value) </pre>
	Output: Rata-rata: 3.8

2.12. Transpose dan Reshape Array

Reshape dan *transpose* adalah dua operasi penting dalam manipulasi *array*. *Reshape* mengubah bentuk atau dimensi *array* tanpa mengubah datanya, memungkinkan penyesuaian data untuk operasi tertentu atau model pembelajaran mesin. *Transpose* menukar dimensi *array*, seperti mengubah baris menjadi kolom, dan sering digunakan dalam operasi linear atau saat mengalihkan orientasi data. Keduanya memberikan fleksibilitas dalam mengelola struktur data. Berikut adalah implementasi dan contoh *transpose* dan *reshape* pada *array* NumPy:

Sintaks <code>`transpose()`</code>	
[1]:	<pre> x = np.array([[1, 2, 3], [4, 5, 6]]) y = x.transpose() print(y) </pre>
	Output: [[1 4] [2 5] [3 6]]
Sintaks <code>`np.transpose()`</code>	
[2]:	<pre> x = np.array([[1, 2, 3], [4, 5, 6]]) y = np.transpose(x) print(y) </pre>
	Output: [[1 4] [2 5] [3 6]]
Sintaks <code>`.T`</code> untuk <i>transpose</i>	
[3]:	<pre> x = np.array([[1, 2, 3], [4, 5, 6]]) y = x.T print(y) </pre>
	Output: [[1 4] [2 5] [3 6]]
Sintaks <code>`reshape()`</code>	
[4]:	<pre> x = np.array([1, 2, 3, 4, 5, 6]) # Mengubah menjadi array 2D dengan 2 baris dan 3 kolom y = x.reshape((2, 3)) print(y) </pre>
	Output: [[1 2 3] [4 5 6]]
Sintaks <code>`np.reshape()`</code>	
[5]:	<pre> x = np.array([1, 2, 3, 4, 5, 6]) # Mengubah menjadi array 2D dengan 2 baris dan 3 kolom y = np.reshape(x, (2, 3)) print(y) </pre>
	Output: [[1 2 3]]

2.13. Soal Latihan Pemrograman Numpy

1. Buatlah program Python menggunakan *library* Numpy yang dapat menyimpan data *array* of mahasiswa (Nama, NIM, Nilai, TahunMasuk). Nama dan NIM bertipe *string*, Nilai bertipe bilangan *real*, sedangkan TahunMasuk bertipe *integer*. Program Anda harus dapat menerima input/masukan data mahasiswa secara interaktif (dari *keyboard*), menampilkan seluruh data yang sudah diinput, serta menampilkan nilai tertinggi, nilai terendah, dan nilai rata-rata. Program Anda juga harus dapat melakukan pencarian mahasiswa berdasarkan NIM/Nama, kemudian menampilkan data lengkap mahasiswa hasil pencarian tersebut.
2. Buatlah program Python menggunakan *library* NumPy yang dapat menyimpan data inventaris barang di sebuah gudang. Data yang perlu disimpan mencakup NamaBarang (*string*), KodeBarang (*string*), Jumlah (*integer*), dan HargaPerUnit (bilangan *real*). Program Anda harus dapat menerima input data barang secara interaktif (dari *keyboard*), menampilkan seluruh data yang sudah diinput, serta menampilkan total nilai inventaris (Jumlah x HargaPerUnit untuk setiap barang). Selain itu, program Anda juga harus dapat melakukan pencarian barang berdasarkan KodeBarang atau NamaBarang, dan menampilkan data lengkap dari barang hasil pencarian tersebut.

Modul 3 PENGENALAN *LIBRARY* NETWORKX

Tujuan Praktikum:

1. Mahasiswa memahami cara mengelola struktur data graf dengan *library* NetworkX.
2. Mahasiswa mengetahui fitur-fitur utama yang tersedia di *library* NetworkX.

REFERENSI UTAMA: <https://networkx.org/documentation/stable/tutorial.html>

REFERENSI TAMBAHAN:

<https://networkx.org/documentation/stable/reference/index.html>

NetworkX adalah *library* Python yang digunakan untuk membuat, memanipulasi, dan mempelajari struktur dan fungsi dari graf. NetworkX mendukung operasi pada graf tidak berarah, graf berarah, graf berbobot, dan lain-lain.

3.1. Cara *Import* NetworkX

NetworkX seharusnya sudah terdapat pada instalasi Anaconda. Untuk menggunakan modul NetworkX, harus dilakukan *import* terlebih dahulu. Berikut adalah cara standar untuk mengimport *library* NetworkX:

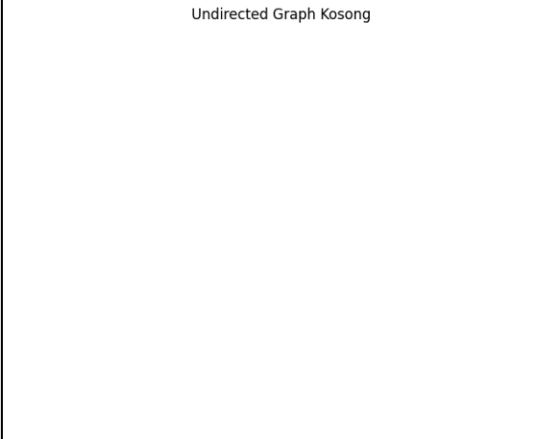
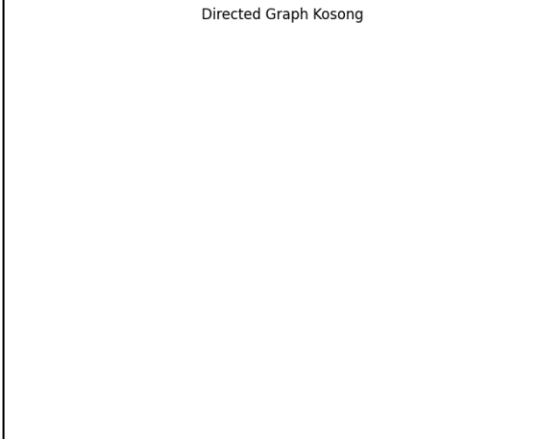
```
import networkx as nx
```

3.2. Membuat Graf Kosong pada NetworkX

Pada NetworkX, terdapat dua tipe graf dasar yang akan dipelajari. Yang pertama adalah **`Graph`**, yang menghasilkan graf tidak berarah (*undirected graph*), di mana setiap *edge* tidak memiliki arah dan bersifat simetris. Jika terdapat *edge* antara dua *node*, maka kedua *node* tersebut saling terhubung tanpa memerhatikan arah. Kedua ialah **`DiGraph`** yang menghasilkan graf berarah, sehingga hubungan antara dua *node* tidak harus simetris. Ini berarti bahwa jika ada *edge* dari *node* A ke *node* B, secara tidak langsung harus ada *edge* dari *node* B ke *node* A. Berikut adalah cara pendefinisian graf kosong pada NetworkX:

```
# Import library yang digunakan
import matplotlib.pyplot as plt
import networkx as nx

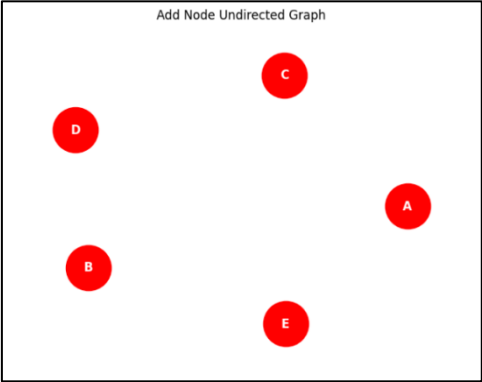
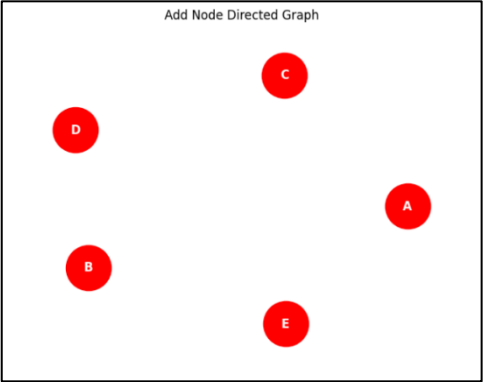
# Posisi node yang digunakan pada modul ini
pos = {'A': [ 0.93888004, -0.0449161 ],
       'B': [-0.73226196, -0.53345128],
       'C': [0.29334637, 1.          ],
       'D': [-0.79991428,  0.56134392],
       'E': [ 0.29994983, -0.98297654]}
```

<pre> # Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='') : if pos == None : pos = nx.spring_layout(G) nx.draw(G, pos, with_labels=True, node_color='red', node_size=2000, font_color="white", font_weight="bold", width=5) edge_labels = nx.get_edge_attributes(G, 'weight') nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='blue', font_weight="bold", font_size=12,) plt.margins(0.2) plt.title(title) plt.show() </pre>	
Pendefinisian `Graph` kosong <pre> G_undirected = nx.Graph() # Cetak graf show_graph(G_undirected, title="Undirected Graph Kosong") </pre>	Pendefinisian `DiGraph` kosong <pre> G_directed = nx.DiGraph() # Cetak graf show_graph(G_directed, title="Directed Graph Kosong") </pre>
Output: 	Output: 

Dari hasil diatas, dapat dilihat bahwa pendefinisian graf kosong berhasil karena tercetak gambar kosong yang memiliki judul.

3.3. Menambahkan *Node* Pada Graf

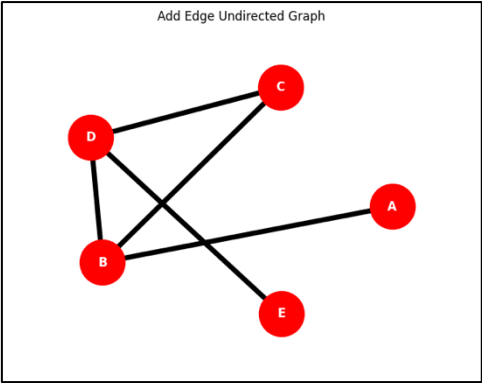
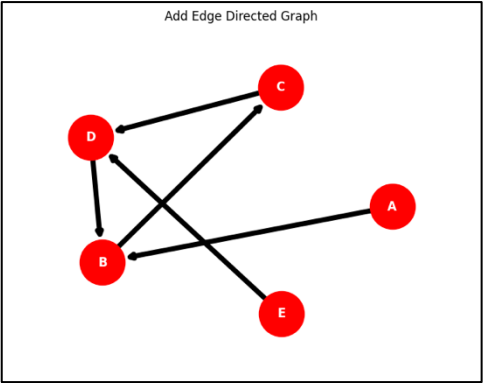
Terdapat dua cara untuk menambahkan *node* di NetworkX, yaitu dengan ``add_node()`` untuk menambahkan satu *node* dan dengan ``add_nodes_from()`` untuk menambahkan beberapa *node* sekaligus. Posisi *node* saat dicetak mungkin berbeda-beda setiap kode dijalankan. Berikut adalah implementasi untuk menambahkan *node* pada graf:

Menambahkan <i>node</i> pada <i>undirected graph</i>	Menambahkan <i>node</i> pada <i>directed graph</i>
<pre># Cara add_node() G_undirected.add_node('A') G_undirected.add_node('B') G_undirected.add_node('C') G_undirected.add_node('D') G_undirected.add_node('E') # Cara add_nodes_from() node_data = ['A', 'B', 'C', 'D', 'E'] G_undirected.add_nodes_from(node_data) # Mencatat posisi node agar konsisten pos = nx.spring_layout(G_undirected) # Cetak graf show_graf(G_undirected, pos=pos, title="Add Node Undirected Graph") # Melihat jumlah node pada graf print("Jumlah node:", G_undirected.number_of_nodes())</pre>	<pre># Cara add_node() G_directed.add_node('A') G_directed.add_node('B') G_directed.add_node('C') G_directed.add_node('D') G_directed.add_node('E') # Cara add_nodes_from() node_data = ['A', 'B', 'C', 'D', 'E'] G_directed.add_nodes_from(node_data) # Mencatat posisi node agar konsisten pos = nx.spring_layout(G_directed) # Cetak graf show_graf(G_directed, pos=pos, title="Add Node Directed Graph") # Melihat jumlah node pada graf print("Jumlah node:", G_directed.number_of_nodes())</pre>
Output:  Jumlah node: 5	Output:  Jumlah node: 5

Dari kedua hasil diatas, dapat dilihat bahwa kedua graf masih terlihat sama. Ini karena perbedaan *undirected graph* dan *directed graph* terletak pada *edge*.

3.4. Menambahkan *Edge* Pada Graf

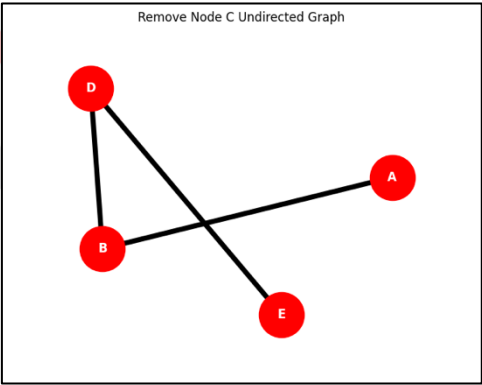
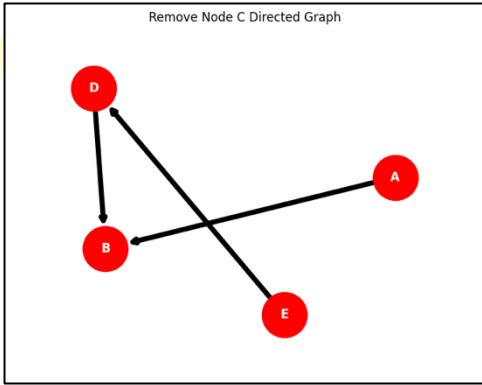
Untuk menambahkan *edge* di NetworkX, metode ``add_edge()`` digunakan untuk menambahkan satu *edge*, dan ``add_edges_from()`` digunakan untuk menambahkan beberapa *edge* sekaligus. *Edge* antara dua *node* dapat ditambahkan dengan menentukan *node* awal dan akhir. Jika *node* yang disebutkan belum ada dalam graf, maka *node* tersebut akan otomatis ditambahkan. Pada *directed graph*, *edge* memiliki arah dari *node* pertama ke *node* kedua. Berikut adalah implementasi untuk menambahkan *edge* pada graf di NetworkX:

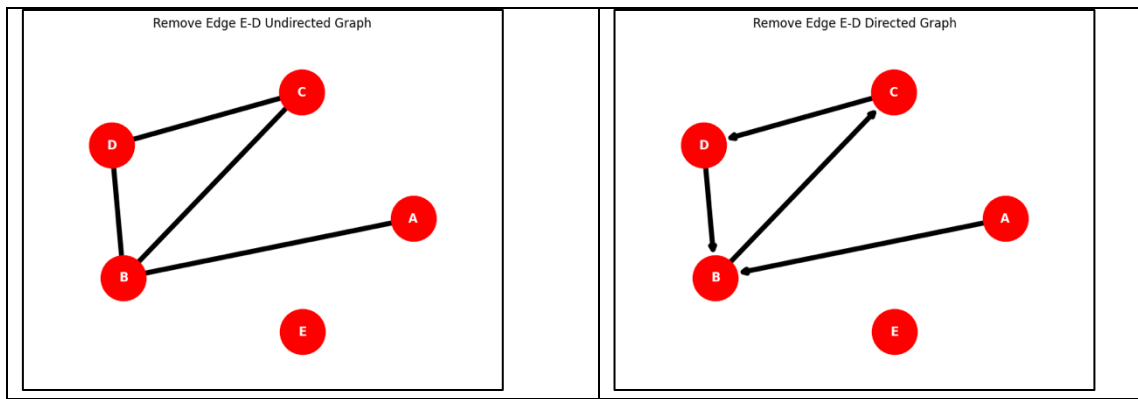
Menambahkan <i>edge</i> pada <i>undirected graph</i>	Menambahkan <i>edge</i> pada <i>directed graph</i>
<pre># Cara add_edge() G_undirected.add_edge('A', 'B') G_undirected.add_edge('B', 'C') G_undirected.add_edge('C', 'D') G_undirected.add_edge('D', 'B') G_undirected.add_edge('E', 'D') # Cara add_edges_from() edge_data = [('A', 'B'), ('B', 'C'), ('C', 'D'), ('D', 'B'), ('E', 'D')] G_undirected.add_edges_from(edge_data) # Cetak graf show_graf(G_undirected, pos=pos, title="Add Edge Undirected Graph") # Melihat jumlah edge pada graf print("Jumlah edge:", G_undirected.number_of_edges())</pre>	<pre># Cara add_edge() G_directed.add_edge('A', 'B') G_directed.add_edge('B', 'C') G_directed.add_edge('C', 'D') G_directed.add_edge('D', 'B') G_directed.add_edge('E', 'D') # Cara add_edges_from() edge_data = [('A', 'B'), ('B', 'C'), ('C', 'D'), ('D', 'B'), ('E', 'D')] G_directed.add_edges_from(edge_data) # Cetak graf show_graf(G_directed, pos=pos, title="Add Edge Directed Graph") # Melihat jumlah edge pada graf print("Jumlah edge:", G_directed.number_of_edges())</pre>
Output:  Jumlah edge: 5	Output:  Jumlah edge: 5

Perbedaan antara *undirected graph* dan *directed graph* terlihat setelah penambahan *edge* pada graf. Urutan penambahan *edge* pada *directed graph* tidak sama, dimana ``add_edge('A', 'B')`` berbeda dengan ``add_edge('B', 'A')``.

3.5. Menghapus *Node* dan *Edge*

Pada NetworkX, *node* dan *edge* dapat dihapus dari graf menggunakan metode ``remove_node()`` dan ``remove_edge()``. Pada ``remove_node()``, *node* yang dipilih akan dihapus dari graf, Bersama dengan semua *edge* yang terhubung pada *node* tersebut. Sedangkan pada ``remove_edge()``, *edge* yang dipilih akan dihapus dari graf, tetapi *node* yang dihubungkan oleh *edge* tersebut akan tetap ada. Berikut adalah implementasi penghapusan *node* dan *edge* pada graf di NetworkX:

Penghapusan <i>node</i> pada graf	
Penghapusan <i>node</i> pada <i>undirected graph</i>	Penghapusan <i>node</i> pada <i>directed graph</i>
<pre># Penghapusan node 'C' G_undirected_remove.remove_node('C') # Cetak graf show_graph(G_undirected_remove, pos=pos, title='Remove Node C Undirected Graph')</pre>	<pre># Penghapusan node 'C' G_directed_remove.remove_node('C') # Cetak graf show_graph(G_directed_remove, pos=pos, title='Remove Node C Directed Graph')</pre>
Output: 	Output: 
Penghapusan <i>edge</i> pada graf	
Penghapusan <i>edge</i> pada <i>undirected graph</i>	Penghapusan <i>edge</i> pada <i>directed graph</i>
<pre># Menghapus edge E - D G_undirected_remove.remove_edge('E', 'D') # Cetak graf show_graph(G_undirected_remove, pos=pos, title='Remove Edge E-D Undirected Graph')</pre>	<pre># Menghapus edge E - D G_directed_remove.remove_edge('E', 'D') # Cetak graf show_graph(G_directed_remove, pos=pos, title='Remove Edge E-D Directed Graph')</pre>
Output:	Output:



3.6. Elemen Pada Graf

Elemen-elemen pada graf dalam NetworkX adalah komponen-komponen utama yang membentuk struktur graf dan memungkinkan melakukan analisis terhadap graf yang dibuat. Berikut penjelasan mengenai berbagai elemen graf beserta metode yang digunakan untuk mengaksesnya:

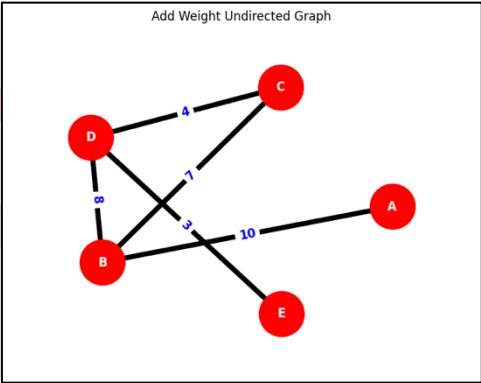
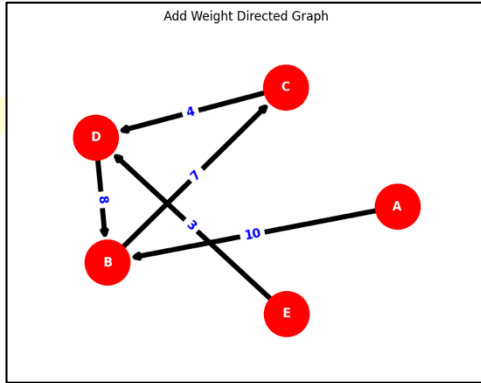
<code>`.nodes()`</code> <i>Method ini digunakan untuk mengembalikan daftar semua node dalam graf</i>	
<pre>print("Node pada G_undirected:", G_undirected.nodes()) print("Node pada G_directed:", G_directed.nodes())</pre>	
Output: Node pada G_undirected: ['A', 'B', 'C', 'D', 'E'] Node pada G_directed: ['A', 'B', 'C', 'D', 'E']	
<code>`.edges()`</code> <i>Method ini digunakan untuk mengembalikan daftar semua edges dalam graf</i>	
<pre>print("Edge pada G_undirected:", G_undirected.edges()) print("Edge pada G_directed:", G_directed.edges())</pre>	
Output: Edge pada G_undirected: [('A', 'B'), ('B', 'C'), ('B', 'D'), ('C', 'D'), ('D', 'E')] Edge pada G_directed: [('A', 'B'), ('B', 'C'), ('C', 'D'), ('D', 'B'), ('E', 'D')]	
<code>`.adj()`</code> <i>Method yang digunakan untuk mengakses daftar tetangga atau adjacency dari setiap node. Method ini memberikan akses langsung ke node-node yang terhubung dengan node tertentu.</i>	
<code>`.adj()` pada undirected graph</code>	<code>`.adj()` pada directed graph</code>
<pre>print("Tetangga G_undirected") for node in G_undirected.nodes(): # Tetangga dari node tertentu adj = list(G_undirected.adj[node]) print(f"Tetangga {node}: {adj}")</pre>	<pre>print("Tetangga G_directed") for node in G_directed.nodes(): # Tetangga dari node tertentu adj = list(G_directed.adj[node]) print(f"Tetangga {node}: {adj}")</pre>
Output: Tetangga G_undirected Tetangga A: ['B'] Tetangga B: ['A', 'C', 'D'] Tetangga C: ['B', 'D']	Output: Tetangga G_directed Tetangga A: ['B'] Tetangga B: ['C'] Tetangga C: ['D']

Tetangga D: ['C', 'B', 'E'] Tetangga E: ['D']	Tetangga D: ['B'] Tetangga E: ['D']
<code>` .degree () `</code> <i>Method</i> ini digunakan untuk mengembalikan derajat dari setiap node, yaitu jumlah total <i>edge</i> yang terhubung ke node tersebut. Pada <i>undirected graph</i> , ini adalah jumlah total koneksi, sedangkan pada <i>directed graph</i> ini adalah jumlah <i>incoming</i> dan <i>outgoing edges</i> .	
<code>` .adj () `</code> pada <i>undirected graph</i>	<code>` .adj () `</code> pada <i>directed graph</i>
<pre>print("Derajat node G_undirected") for node, degree in G_undirected.degree(): print(f"Derajat {node}: {degree}")</pre>	<pre>print("Derajat node G_directed") for node, degree in G_directed.degree():</pre>
Output: Tetangga G_undirected Tetangga A: ['B'] Tetangga B: ['A', 'C', 'D'] Tetangga C: ['B', 'D'] Tetangga D: ['C', 'B', 'E'] Tetangga E: ['D']	Output: Tetangga G_directed Tetangga A: ['B'] Tetangga B: ['C'] Tetangga C: ['D'] Tetangga D: ['B'] Tetangga E: ['D']
<code>` .in_degree () `</code> <i>Method</i> ini hanya dapat digunakan pada <i>directed graph</i> untuk menganalisis <i>in-degree</i> dari setiap node, yaitu jumlah <i>edge</i> yang masuk ke node tersebut.	
<pre>print("In-degree node G_directed") for node, degree in G_directed.in_degree(): print(f"In-degree {node}: {degree}")</pre>	
Output: In-degree node G_directed In-degree A: 0 In-degree B: 3 In-degree C: 2 In-degree D: 3 In-degree E: 1	
<code>` .out_degree () `</code> <i>Method</i> ini hanya dapat digunakan pada <i>directed graph</i> untuk menganalisis <i>out-degree</i> dari setiap node, yaitu jumlah <i>edge</i> yang keluar dari node tersebut.	
<pre>print("Out-degree node G_directed") for node, degree in G_directed.out_degree(): print(f"Out-degree {node}: {degree}")</pre>	
Output: Out-degree node G_directed Out-degree A: 1 Out-degree B: 1 Out-degree C: 1 Out-degree D: 1 Out-degree E: 1	

3.7. Menambahkan *Weight* Pada *Graph*

Weight atau bobot dapat ditambahkan pada *edge* dalam graf untuk merepresentasikan nilai dari hubungan antara dua *node*. Terdapat dua metode utama untuk menambahkan *weight* pada *edge*. Pertama dengan menggunakan `` .add_edge () ``, yang digunakan untuk menambahkan satu *edge* pada graf dan dapat ditambahkan sebagai atribut menggunakan parameter `` weight ``. Kedua, untuk menambahkan `` weight `` dapat

menggunakan metode `.add_weighted_edges_from()` yang digunakan untuk menambahkan beberapa *edge* sekaligus dengan *weight* yang telah ditentukan. Berikut adalah implementasi dan contoh penerapan untuk menambahkan *weight* pada *graph*:

Menambahkan <i>weight</i> pada <i>undirected graph</i>	Menambahkan <i>weight</i> pada <i>directed graph</i>
<pre># Cara add_edge() G_undirected.add_edge('A', 'B', weight=10) G_undirected.add_edge('B', 'C', weight=7) G_undirected.add_edge('C', 'D', weight=4) G_undirected.add_edge('D', 'B', weight=8) G_undirected.add_edge('E', 'D', weight=3) # Cara add_weighted_edges_from() edge_weight_data = [('A', 'B', 10), ('B', 'C', 7), ('C', 'D', 4), ('D', 'B', 8), ('E', 'D', 3)] G_undirected.add_weighted_edges_from(edge_weight_data) # Cetak graf show_graf(G_undirected, pos=pos, title="Add Weight Undirected Graph")</pre>	<pre># Cara add_edge() G_directed.add_edge('A', 'B', weight=10) G_directed.add_edge('B', 'C', weight=7) G_directed.add_edge('C', 'D', weight=4) G_directed.add_edge('D', 'B', weight=8) G_directed.add_edge('E', 'D', weight=3) # Cara add_weighted_edges_from() edge_weight_data = [('A', 'B', 10), ('B', 'C', 7), ('C', 'D', 4), ('D', 'B', 8), ('E', 'D', 3)] G_directed.add_weighted_edges_from(edge_weight_data) # Cetak graf show_graf(G_undirected, pos=pos, title="Add Weight Directed Graph")</pre>
Output: 	Output: 

3.8. Analisis Graf

Dalam NetworkX, berbagai fungsi untuk menganalisis graf disediakan untuk membantu memahami struktur dan karakteristik dari graf yang dibuat. Berikut adalah penjelasan dan contoh implementasi dari masing-masing fungsi:

<pre><code>.connected_components()</code></pre> Fungsi ini digunakan untuk menemukan semua <i>connected components</i> dalam <i>undirected graph</i> yang berarti setiap <i>node</i> dalam suatu kelompok dapat dijangkau oleh <i>node</i> lainnya dalam kelompok yang sama.
<pre>import networkx as nx G = nx.Graph() # Membuat graf baru # Menambahkan edge (sekaligus node) pada graf G.add_edges_from([('B', 'D'), ('B', 'E'), ('C', 'A')]) # Membuat kelompok connected components</pre>

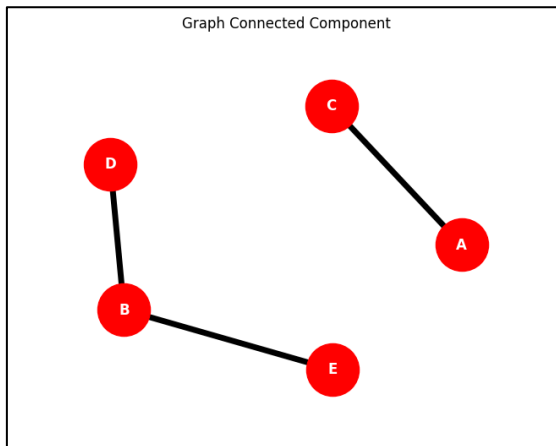
```

components = list(nx.connected_components(G))
print(components)

# Cetak graf
show_graph(G, title="Graph Connected Component")

```

Output:
 [{ 'D', 'E', 'B' }, { 'A', 'C' }]



``.clustering()``

Fungsi ini mengukur *clustering coefficient* dari setiap *node* dalam graf. *Clustering coefficient* adalah ukuran lokal yang menggambarkan seberapa cenderung *node-node* tetangga dari suatu *node* saling terhubung satu sama lain.

```

import networkx as nx

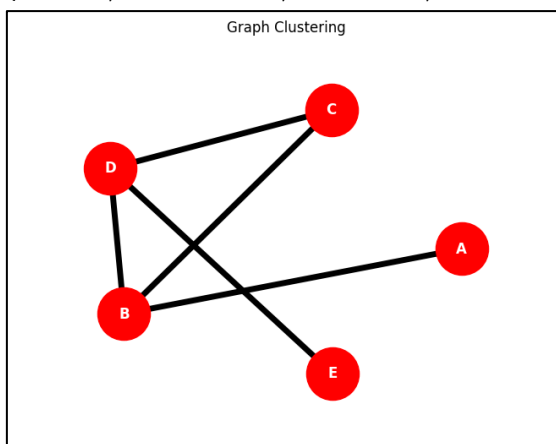
G = nx.Graph() # Membuat graf baru
# Menambahkan edge (sekaligus node) pada graf
edge_data = [( 'A', 'B' ), ( 'B', 'C' ), ( 'C', 'D' ), ( 'D', 'B' ), ( 'E', 'D' )]
G.add_edges_from(edge_data)

# Menghitung clustering coefficient setiap node
clustering_coefficients = nx.clustering(G)
print(clustering_coefficients)

# Cetak graf
show_graph(G, title="Graph Clustering")

```

Output:
 { 'A': 0, 'B': 0.333, 'C': 1.0, 'D': 0.333, 'E': 0 }



`.all_shortest_path()`

Fungsi ini digunakan untuk menemukan semua jalur terpendek antara dua *node* dalam graf. Fungsi ini mencari semua rute yang memiliki nilai *edge* terkecil antara dua *node* tertentu.

```
import networkx as nx
```

```
G = nx.Graph() # Membuat graf baru
```

```
# Menambahkan edge dan weight (sekaligus node) pada graf
```

```
edge_weight_data = [('A', 'B', 10), ('B', 'C', 7), ('C', 'D', 4), ('D', 'B', 8), ('E', 'D', 3)]
```

```
G.add_weighted_edges_from(edge_weight_data)
```

```
# Membuat daftar jalur terpendek antara dua node untuk seluruh node
```

```
shortest_paths = dict(nx.all_pairs_shortest_path(G))
```

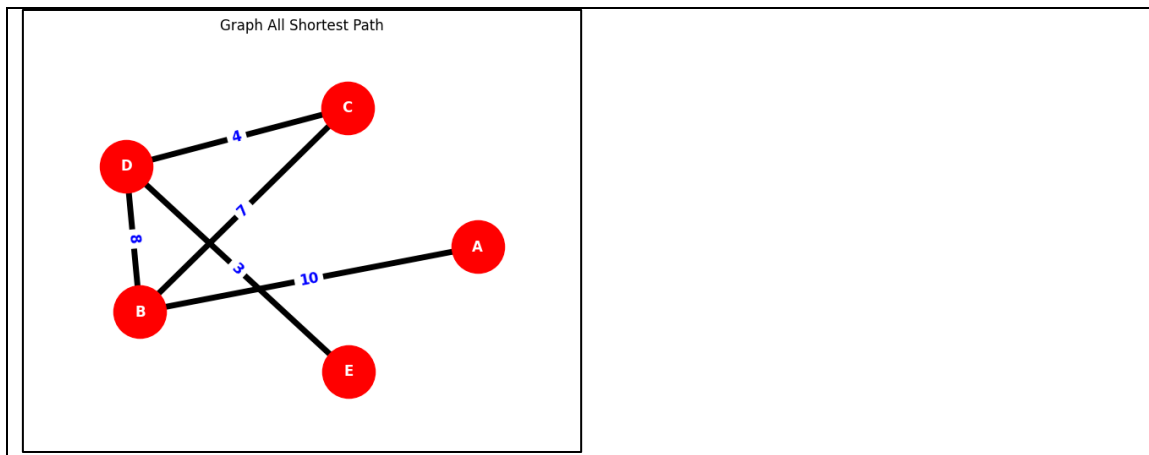
```
print(shortest_paths)
```

```
# Cetak graf
```

```
show_graph(G, title="Graph All shortest path")
```

Output:

```
{
  'A': {
    'A': ['A'],
    'B': ['A', 'B'],
    'C': ['A', 'B', 'C'],
    'D': ['A', 'B', 'D'],
    'E': ['A', 'B', 'D', 'E']
  },
  'B': {
    'B': ['B'],
    'A': ['B', 'A'],
    'C': ['B', 'C'],
    'D': ['B', 'D'],
    'E': ['B', 'D', 'E']
  },
  'C': {
    'C': ['C'],
    'B': ['C', 'B'],
    'D': ['C', 'D'],
    'A': ['C', 'B', 'A'],
    'E': ['C', 'D', 'E']
  },
  'D': {
    'D': ['D'],
    'C': ['D', 'C'],
    'B': ['D', 'B'],
    'E': ['D', 'E'],
    'A': ['D', 'B', 'A']
  },
  'E': {
    'E': ['E'],
    'D': ['E', 'D'],
    'C': ['E', 'D', 'C'],
    'B': ['E', 'D', 'B'],
    'A': ['E', 'D', 'B', 'A']
  }
}
```



3.9. Menampilkan Graf dengan Matplotlib

Matplotlib berperan sebagai *library* visualisasi utama yang digunakan oleh NetworkX untuk membuat plot graf. Dengan Matplotlib, tampilan graf dapat dikustomisasi secara mendetail, termasuk warna, ukuran, tata letak, label, dan lainnya, untuk membuat visualisasi yang lebih informatif dan menarik.

3.9.1. Tata Letak *Node*

Tata letak *node* dalam graf dalam plot akan ditampilkan secara acak dan akan berubah setiap kali blok kode untuk menampilkan graf dijalankan. Agar posisi tersebut konsisten, maka posisi *node* dapat disimpan ke dalam suatu *variable* yang biasa diberi nama ``pos``.

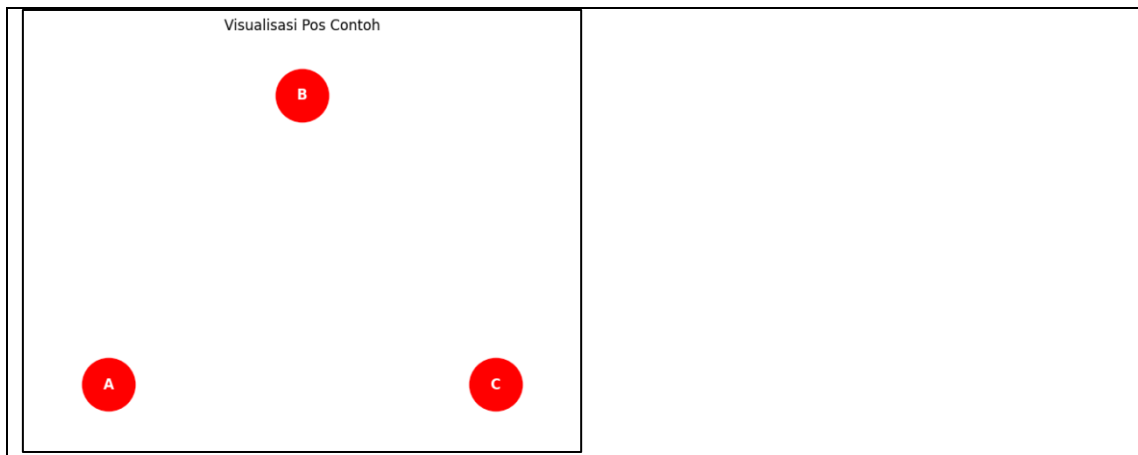
Terdapat beberapa aturan agar NetworkX dan Matplotlib dapat membaca koordinat yang diberikan. Berikut adalah aturan ``pos``:

- Tipe data ``pos`` harus sebuah *dictionary*.
- ``key`` dalam *dictionary* adalah nama-nama *node* dalam graf, misalnya ``A``, ``B``, ``C``.
- ``value`` dari *key* yang terkait adalah tuple atau list yang berisi koordinat ``(x, y)``, yang menentukan posisi *node* pada graf.

Berikut adalah contoh pendefinisian variabel ``pos`` untuk *node* ``A``, ``B``, ``C``:

```
pos = {
    'A': (0,0), # Posisi untuk node A
    'B': (1,1), # Posisi untuk node B
    'C': (2,0)  # Posisi untuk node C
}
```

Visualisasi Koordinat ``pos``:



3.9.2. Fungsi *Layout* Pada Graf

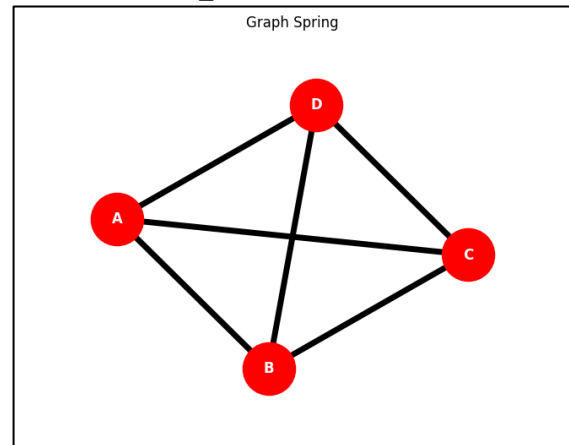
NetworkX menyediakan berbagai fungsi *layout* otomatis untuk mengatur tata letak *node* di ruang 2 dimensi dengan mempertimbangkan struktur graf, hubungan antar *node*, dan keterbacaan visual. Dengan fungsi *layout*, graf yang cukup kompleks dapat dengan mudah diatur, terutama ketika graf menjadi kompleks dengan banyak *node*. Berikut adalah beberapa fungsi *layout* yang sering digunakan di NetworkX:

``spring_layout()``

Fungsi ini menggunakan model *spring*, dimana seolah-olah *node* terhubung oleh pegas. *Node* akan ditempatkan cenderung lebih dekat satu sama lain, sementara *node* yang tidak terhubung akan diletakkan lebih jauh.

`pos = nx.spring_layout(G)`

Contoh `spring_layout()`:

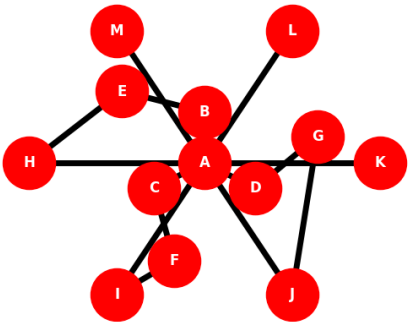
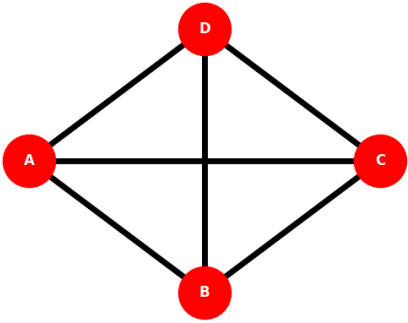
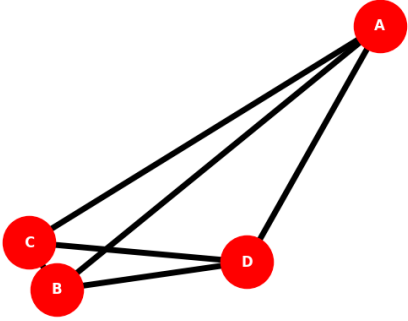


``shell_layout()``

Fungsi ini menempatkan *node* dalam beberapa lapisan, Setiap lapis adalah lingkaran yang mengelilingi pusat, dan *node* tertentu dikelompokkan dalam *shell* yang berbeda. Parameter ``nlist`` mendefinisikan kelompok *node* yang akan ditempatkan di lapisan tertentu.

```
pos = nx.shell_layout(
    G_shell,
    nlist=[
        ['A'], # Lapis 1
        ['B', 'C', 'D'], # Lapis 2
```

Contoh `shell_layout()`:

<pre> ['E', 'F', 'G'], # Lapis 3 ['H', 'I', 'J', 'K', 'L', 'M']]) </pre>	Graph Shell 
<pre> `circular_layout()` </pre> Fungsi ini menempatkan semua <i>node</i> dalam graf pada lingkaran dengan jarak yang sama dari pusat. Posisi setiap <i>node</i> dalam lingkaran ditentukan secara merata sehingga membentuk susunan yang simetris. <pre> pos = nx.circular_layout(G) </pre>	Contoh <code>circular_layout()</code>: Graph Shell 
<pre> `random_layout()` </pre> Fungsi ini menempatkan <i>node</i> secara acak. Setiap node diberi posisi koordinat yang menghasilkan secara acak, tanpa memperhatikan hubungan atau struktur graf. <pre> pos = nx.random_layout(G) </pre>	Contoh <code>random_layout()</code>: Graph Random 

3.9.3. Fungsi `show_graph` Untuk Visualisasi Graf

Fungsi `show_graph` adalah sebuah fungsi pendukung yang dirancang untuk memvisualisasikan graf yang dibangun dengan NetworkX. Fungsi ini menyediakan cara yang mudah dan fleksibel untuk menampilkan graf dengan berbagai opsi. Seluruh graf yang divisualisasikan pada modul praktikum ini ditampilkan menggunakan fungsi `show_graph` yang dimodifikasi parameternya sesuai kebutuhan. Terdapat beberapa parameter yang didefinisikan pada fungsi ini, berikut penjelasannya:

- **Parameter `G`**
`G` adalah graf yang akan divisualisasikan. Graf ini dapat berupa *undirected graph* (`.Graph()`) atau *directed graph* (`.DiGraph()`).
- **Parameter `pos`**
`pos` merupakan *dictionary* yang menentukan posisi dari setiap *node* dalam graf. Jika parameter `pos` tidak diberikan saat pemanggilan, fungsi ini akan otomatis menghasilkan tata letak *node* menggunakan `.spring_layout()`.
- **Parameter `title`**
`title` adalah *string* opsional yang digunakan untuk menambahkan judul pada plot graf. Jika tidak ada judul yang diberikan, maka graf akan ditampilkan tanpa judul.

Berikut adalah fungsi `show_graf` beserta penjelasan parameternya:

```
import matplotlib.pyplot as plt
import networkx as nx

# Fungsi pendukung untuk mencetak graf
def show_graph(G, pos=None, title='') :
    # Membuat pos jika pos tidak diberikan
    if pos is None:
        pos = nx.spring_layout(G)

    # Fungsi untuk menggambar node
    nx.draw(
        G,                    # Graf NetworkX
        pos,                  # Posisi node
        with_labels=True,     # Menampilkan nama node
        node_color='red',     # Warna node
        node_size=2000,       # Ukuran node
        font_color="white",    # Warna font label node
        font_weight="bold",   # Ketebalan font label node
        width=5               # Ketebalan garis edge
    )

    # Mengambil label edge jika ada weight
    edge_labels = nx.get_edge_attributes(G, 'weight')
    # Fungsi untuk menggambar node
```



```

nx.draw_networkx_edge_labels(
    G,
    pos,
    edge_labels=edge_labels, # Data weight
    font_color='blue',      # Warna font label edge
    font_weight="bold",      # Berat font label edge
    font_size=12,           # Ukuran font label edge
)

plt.margins(0.2) # Memberikan margin pada plot
plt.title(title) # Menampilkan judul graf jika diberikan
plt.show()      # Menampilkan graf menggunakan matplotlib

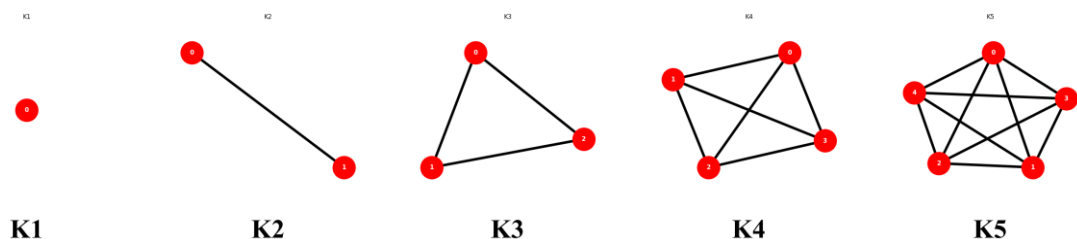
```

3.10. Membuat Graf Secara Otomatis

Dalam NetworkX, graf dapat dibuat secara otomatis menggunakan fungsi bawaan yang menghasilkan berbagai jenis graf berdasarkan karakteristik dan struktur tertentu. NetworkX menyediakan berbagai cara untuk membangun graf yang sudah dikenal dalam teori graf. Terdapat beberapa cara dan metode untuk membuat graf secara otomatis.

3.10.1. Complete Graph

Complete graph (graf lengkap) adalah graf di mana setiap pasangan *node* saling terhubung dengan tepat satu *edge* seperti yang ditunjukkan pada Gambar 7. Gambar contoh Complete Graph. Ini berarti bahwa graf lengkap dengan n *node* akan memiliki $\frac{n(n-1)}{2}$ *edge*.



Gambar 7. Gambar contoh *Complete Graph*.

Berikut adalah implementasi dan contoh pembuatan *complete graph* pada NetworkX:

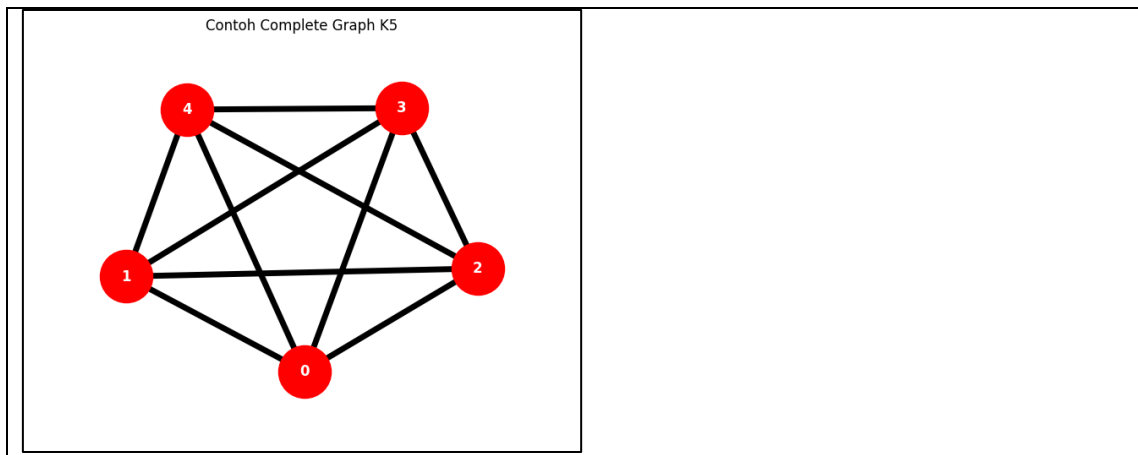
```

# Membuat graf lengkap dengan 5 node
G = nx.complete_graph(5)

# Cetak graf
show_graph(G, title="Contoh Complete Graph K5")

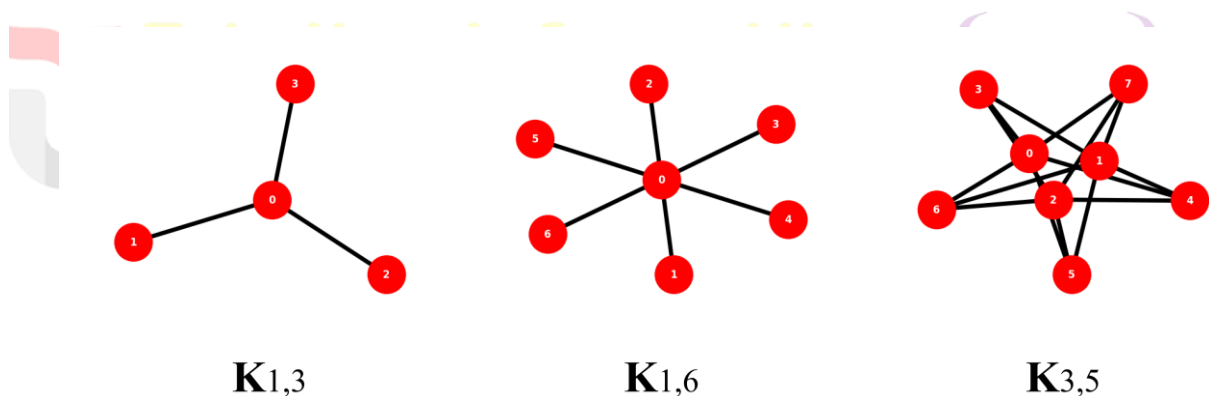
```

Output:



3.10.2. Complete Bipartite Graph

Complete bipartite graph (graf bipartite lengkap) yang ditunjukkan pada Gambar 8 adalah graf dimana *node* dibagi menjadi dua set terpisah, dan setiap *node* dalam satu set terhubung dengan setiap *node* dalam set lainnya, tetapi tidak ada *edge* di dalam set yang sama.



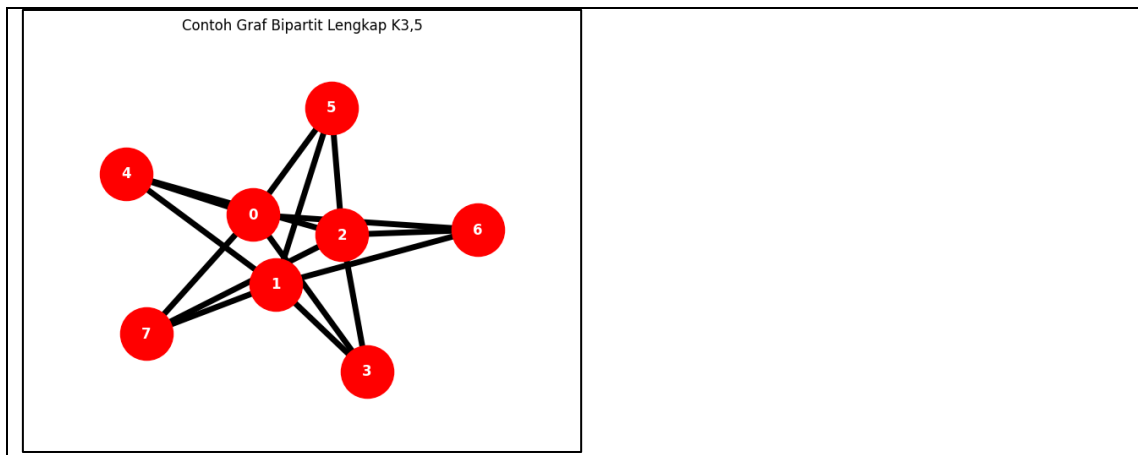
Gambar 8. Gambar contoh Bipartite Graph.

Berikut adalah implementasi dan contoh pembuatan *complete bipartite graph* pada NetworkX:

```
# Membuat graf bipartite lengkap K3,5
G = nx.complete_bipartite_graph(3,5)

# Cetak graf
show_graph(G, title="Contoh Complete Graph K5")
```

Output:



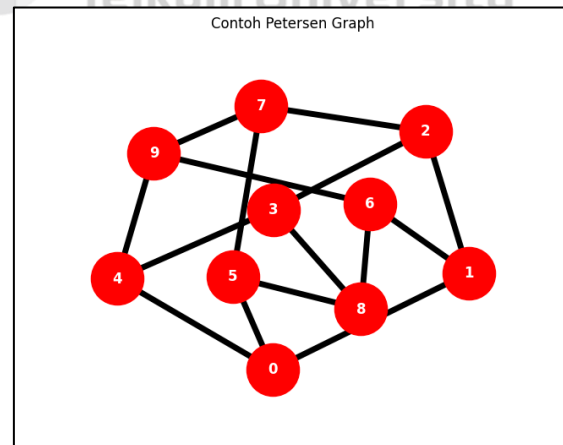
3.10.3. Petersen Graph

Graf *Petersen* adalah graf yang terdiri dari 10 *node* dan 15 *edge*. Graf ini sering digunakan sebagai studi dalam berbagai masalah teori graf, termasuk terkait tentang pewarnaan graf dan graf *Hamilton*. Berikut adalah implementasi dan contoh pembuatan *Petersen Graph* pada NetworkX:

```
# Membuat graf bipartite lengkap  $K_{3,5}$ 
G = nx.petersen_graph()

# Cetak graf
show_graph(G, title="Contoh Petersen Graph")
```

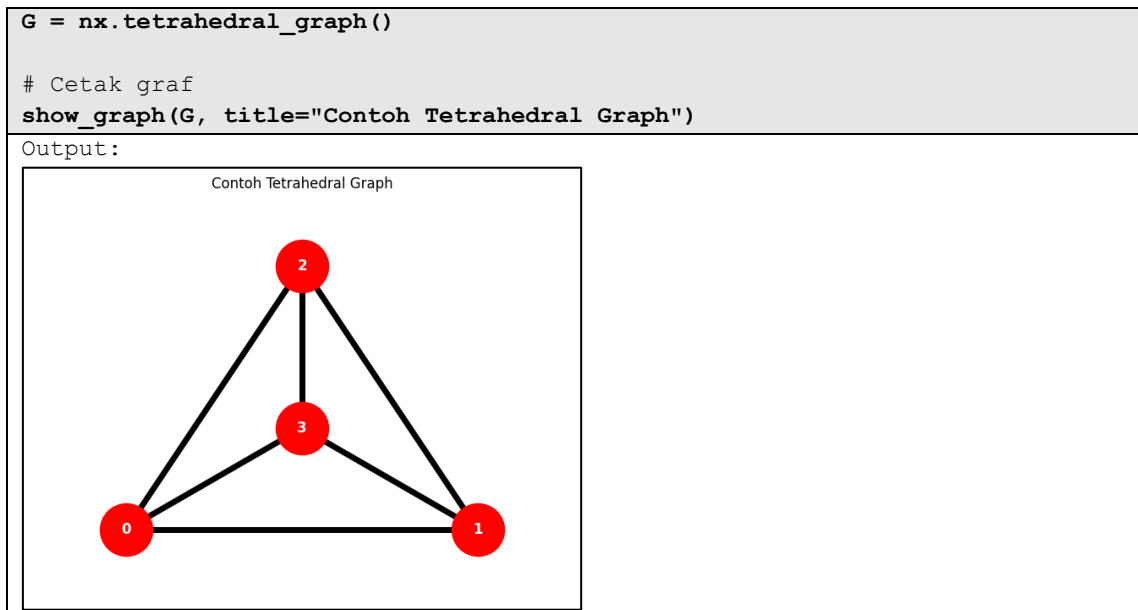
Output:



3.10.4. Tetrahedral Graph

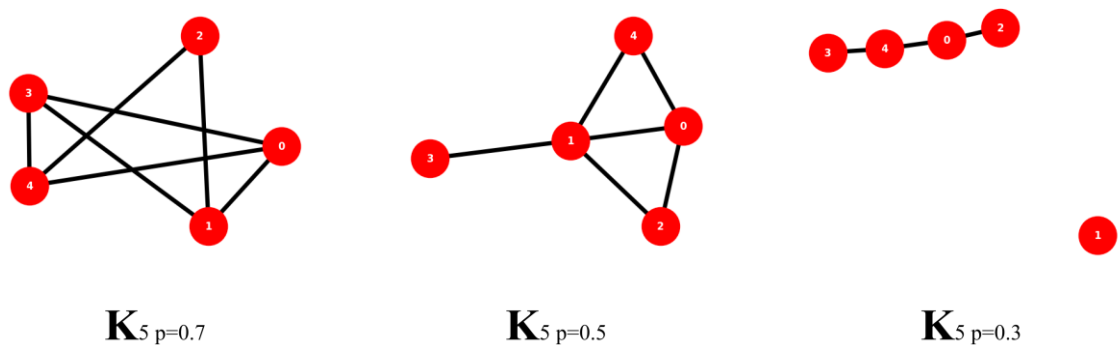
Graf *tetrahedral* adalah graf yang merepresentasikan struktur sebuah *tetrahedron*, yang terdiri dari 4 *node* dan 6 *edge*. *Tetrahedral Graph* merupakan graf lengkap dengan 4 *node* (K_4). Berikut adalah implementasi dan contoh dari pembuatan *tetrahedral graph* pada NetworkX:

```
# Membuat graf bipartite lengkap  $K_{3,5}$ 
```



3.10.5. Edros-Renyi Graph

Edros-Renyi Graph adalah jenis graf acak di mana setiap *edge* antara dua *node* memiliki probabilitas tetap p untuk ada atau tidak ada seperti Gambar 9. Graf ini biasanya digunakan untuk mempelajari sifat-sifat jaringan acak, seperti distribusi derajat, komponen terhubung, dan ambang batas keterhubungan.

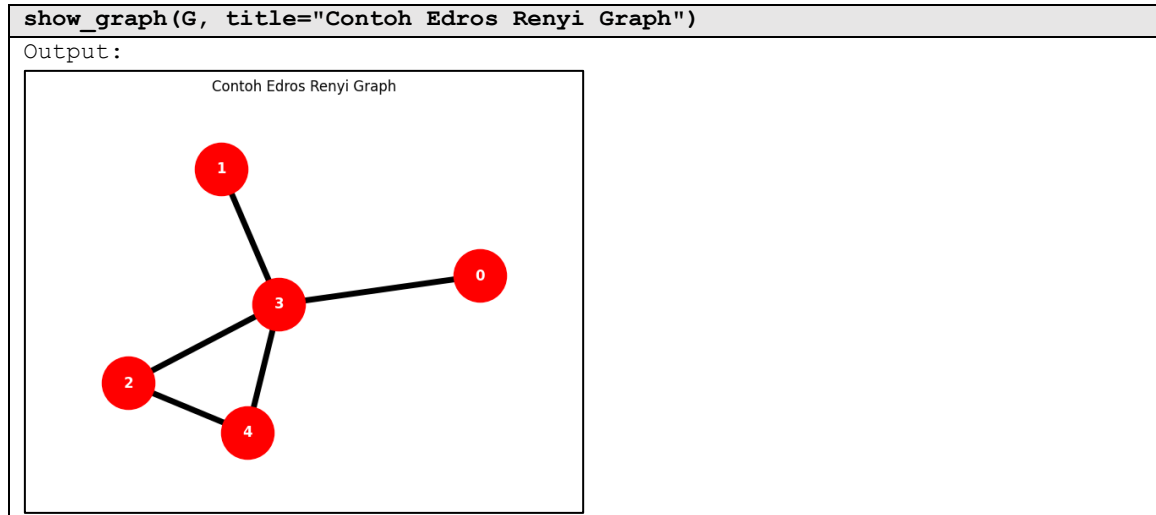


Gambar 9. Gambar contoh *Edros-Renyi Graph*.

Berikut adalah implementasi dan contoh dari pembuatan *Edros-Renyi Graph* pada NetworkX:

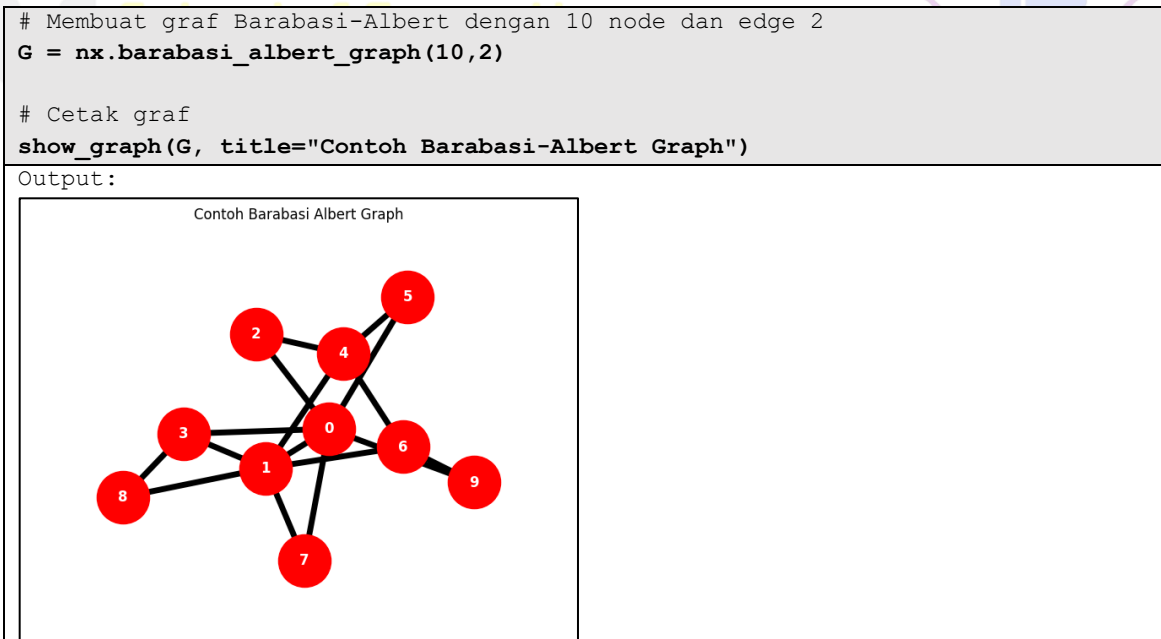
```
# Membuat graf edros-renyi K5 dengan p=0.5
G = nx.edros_renyi_graph(5, 0.5)

# Cetak graf
```



3.10.6. Barabasi-Albert Graph

Graf *Barabasi-Albert* adalah graf acak yang dibangun menggunakan mekanisme *preferential attachment*, di mana *node* baru cenderung terhubung ke *node* yang sudah memiliki banyak koneksi. Graf ini digunakan untuk memodelkan jaringan sosial, internet, dan jaringan biologis. Berikut ini adalah implementasi dan contoh dari pembuatan graf *Barabasi-Albert*:

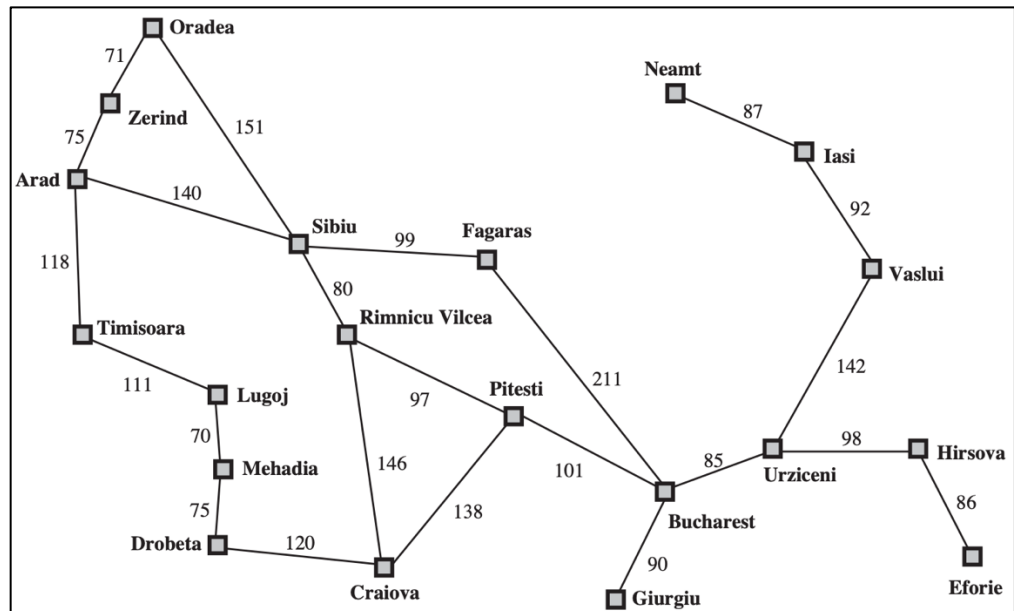


3.11. Soal Latihan Pemrograman NetworkX

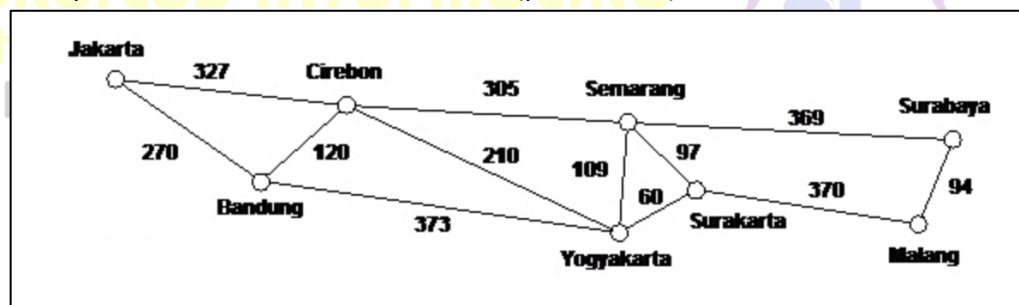
1. Buatlah graf berbobot (*weighted graf*) dari 2 gambar graf berikut ini menggunakan library NetworkX. Nama simpul harus sesuai dengan nama kota yang tertera pada

gambar. Jenis graf yang digunakan adalah *undirected graph*. Kemudian, dengan menggunakan fungsi shortest path yang ada di NetworkX, carilah semua pasangan shortest path dari graf tersebut!

a. Graf peta kota-kota besar di Eropa:



b. Graf peta kota-kota besar di Indonesia (pulau Jawa):



Modul 4 PENGENALAN *LIBRARY* MATPLOTLIB

Tujuan Praktikum:

1. Mahasiswa memahami cara membuat dan menampilkan grafik dengan *library* Matplotlib.
2. Mahasiswa mengetahui fitur-fitur utama yang tersedia di *library* Matplotlib.

REFERENSI UTAMA:

https://matplotlib.org/stable/plot_types/index.html

https://matplotlib.org/stable/users/explain/quick_start.html

REFERENSI TAMBAHAN:

<https://matplotlib.org/stable/index.html>

<https://matplotlib.org/stable/tutorials/index.html>

Matplotlib merupakan *library* Python yang digunakan untuk membuat visualisasi data dalam bentuk grafik. *Library* ini menyediakan berbagai alat untuk membuat grafik seperti *linechart*, *barchart*, *histogram*, *scatter plot*, dan banyak lagi.

4.1. Cara *Import* Matplotlib

Matplotlib seharusnya sudah terdapat pada instalasi Anaconda. Untuk menggunakan modul Matplotlib, harus dilakukan *import* terlebih dahulu. Berikut adalah cara standar untuk *mengimport* library Matplotlib:

```
import matplotlib.pyplot as plt
```

4.2. Elemen *Figure* Pada Matplotlib

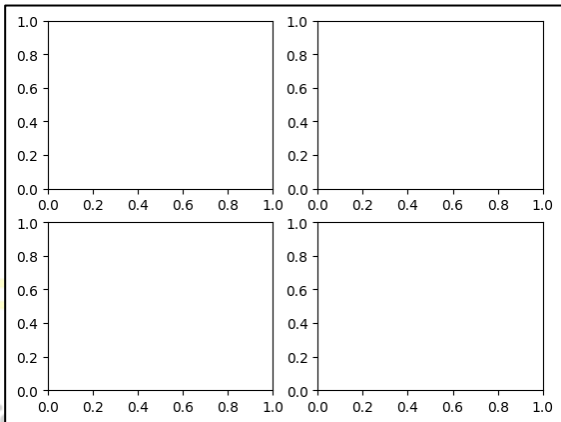
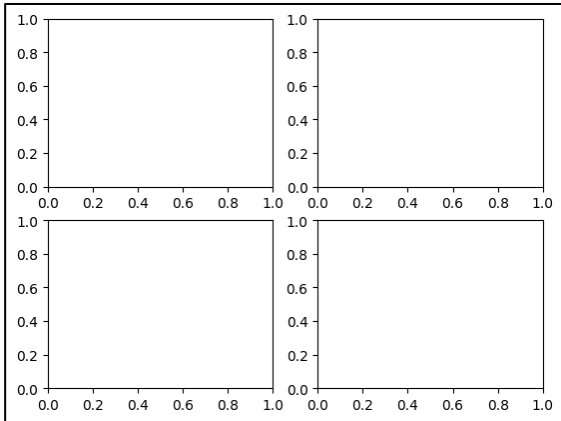
Figure adalah objek dasar dalam Matplotlib yang merepresentasikan seluruh area atau kanvas tempat plot atau grafik akan digambar. Setiap kali Matplotlib memvisualisasikan grafik, grafik tersebut ditempatkan di dalam sebuah *figure*. Berikut adalah implementasi fitur utama dari *figure*:

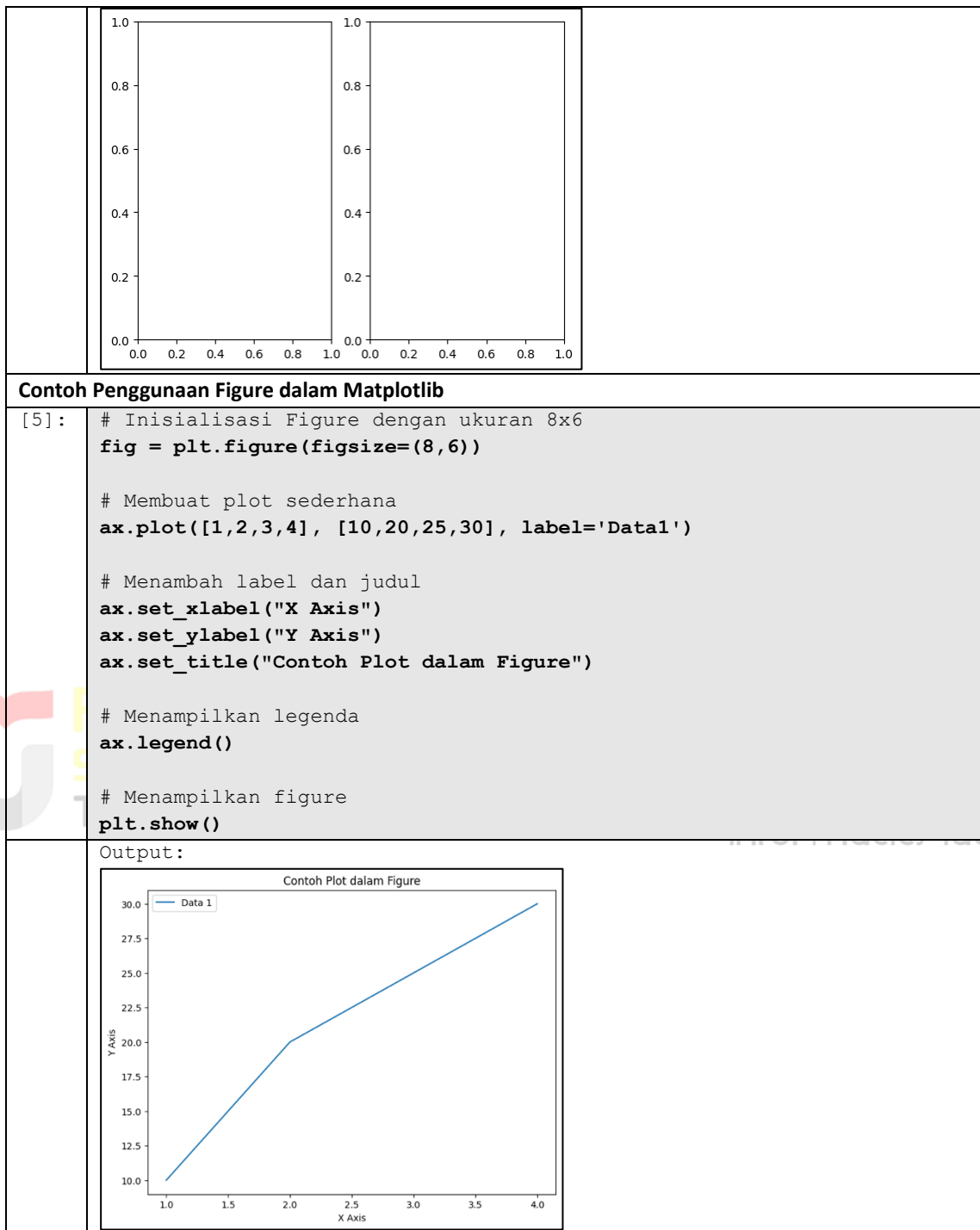
Area Plotting

Figure ditempatkan di suatu area plotting yang dibuat dengan ``plt.figure()`` di mana semua elemen visual (plot, teks, legenda, dan anotasi) ditempatkan. Ini adalah kanvas besar yang dapat berisi satu atau lebih subplot.

```
[1]: # Membuat figure kosong
      fig = plt.figure()
      fig
```

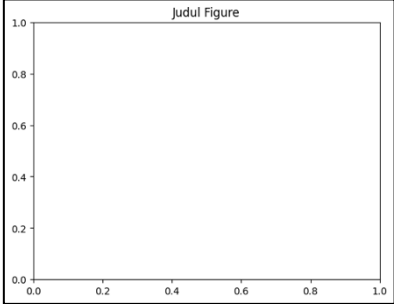
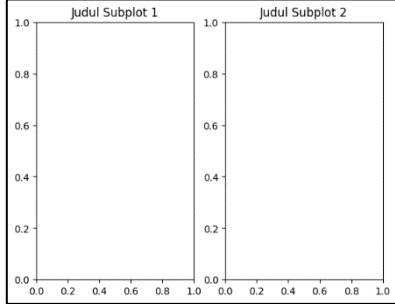
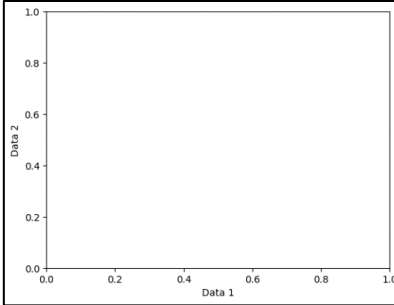
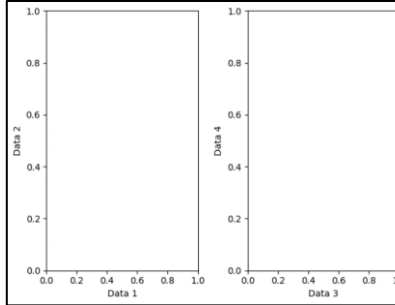
```
Output:
<Figure size 640x480 with 0 Axes>
```

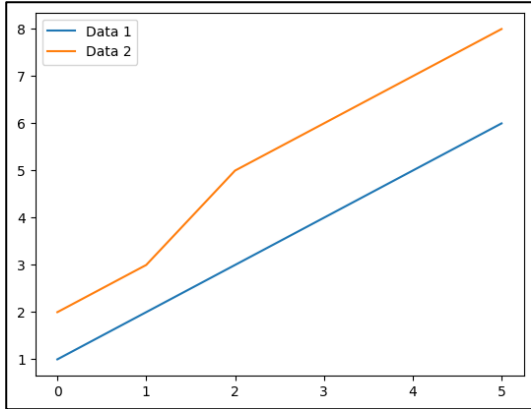
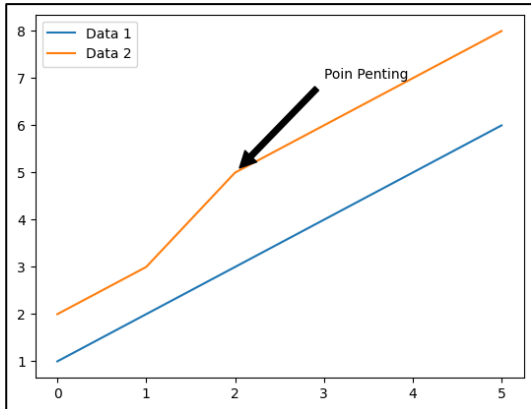
Mengatur Ukuran dan Resolusi Ukuran dan resolusi dari <i>figure</i> dapat disesuaikan menggunakan parameter seperti <code>`figsize`</code> untuk mengatur dimensi dan <code>`dpi`</code> untuk mengatur resolusi gambar.	
[2]:	<pre># Membuat figure dengan ukuran 8x6 inci dan resolusi 100 DPI fig = plt.figure(figsize=(8,6), dpi=100) fig</pre>
	Output: <Figure size 800x600 with 0 Axes>
Subplot <i>Figure</i> dapat dibagi menjadi beberapa <i>subplot</i> , yang merupakan aneka plot individual di dalam <i>figure</i> yang sama. Setiap <i>subplot</i> dapat digunakan untuk menampilkan grafik yang berbeda.	
[3]:	<pre># Membuat plot dengan subplot 2x2 fig, ax = plt.subplots(2,2) fig, ax</pre>
	Output: 
Menampilkan Figure Setelah semua elemen ditambahkan ke <i>figure</i> , <i>figure</i> dapat ditampilkan di layer menggunakan <code>`plt.show()`</code>	
[4]:	<pre># Figure pertama fig, ax = plt.subplots(2,2) plt.show() # Figure kedua fig, ax = plt.subplots(2,2) plt.show()</pre>
	Output: 

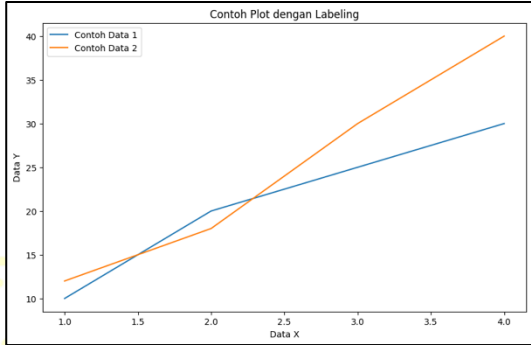


4.3. Labeling Plot

Labeling plot adalah proses memberikan judul, label sumbu, dan legenda pada grafik untuk menjelaskan elemen yang ditampilkan. Labeling membantu pembaca untuk memahami konten grafik dan data yang direpresentasikan. Berikut adalah beberapa fungsi untuk melakukan pelabelan plot secara mudah:

Judul Grafik Judul grafik digunakan untuk memberikan gambaran umum tentang apa yang direpresentasikan oleh grafik.		
[1]:	<pre># Membuat figure fig = plt.figure() # Menambahkan judul pada figure plt.title("Judul Figure") # Menampilkan figure plt.show()</pre>	<pre># Figure dengan 2 subplot fig, ax = plt.subplots(1,2) # Menambahkan judul subplot 1 ax[0].set_title("Judul 1") # Menambahkan judul subplot 2 ax[1].set_title("Judul 2") # Menampilkan figure plt.show()</pre>
	Output: 	Output: 
Memberikan Label Pada Plot Label sumbu x dan sumbu y biasa digunakan untuk menunjukkan jenis data yang direpresentasikan di masing-masing sumbu.		
[2]:	<pre># Membuat figure fig = plt.figure() # Menambahkan label pada plot plt.xlabel("Data 1") plt.ylabel("Data 2") # Menampilkan figure plt.show()</pre>	<pre># Figure dengan 2 subplot fig, ax = plt.subplots(2,2) # Menambahkan label subplot ax[0].set_xlabel("Data 1") ax[0].set_ylabel("Data 2") ax[1].set_xlabel("Data 3") ax[1].set_ylabel("Data 4") # Menampilkan figure plt.show()</pre>
	Output: 	Output: 
Memberikan Legenda Pada Plot Legenda digunakan untuk mengidentifikasi setiap elemen atau data yang digambar pada plot. Legenda membantu membedakan antara berbagai set data.		
[3]:	<pre># Membuat figure fig = plt.figure() # Membuat linechart di figure</pre>	

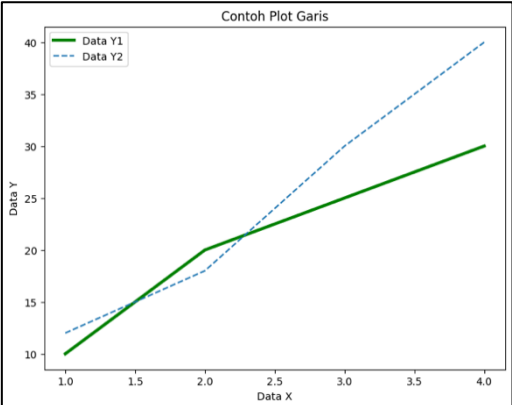
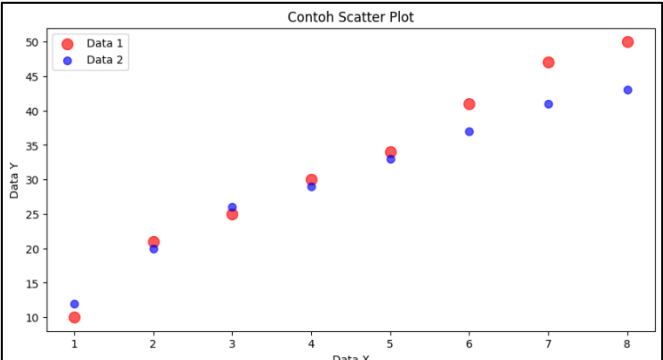
	<pre>plt.plot([1, 2, 3, 4, 5, 6], label="Data 1") plt.plot([2, 3, 5, 6, 7, 8], label="Data 2") # Menampilkan legenda plt.legend() # Menampilkan figure plt.show()</pre>
	Output: 
Memberikan Anotasi Pada Label Anotasi adalah cara untuk menandai poin tertentu pada plot, biasanya dengan teks yang mengacu pada titik atau fitur penting dalam grafik.	
[4]:	<pre># Membuat figure fig = plt.figure() # Membuat linechart pada plt.plot([1, 2, 3, 4, 5, 6], label="Data 1") plt.plot([2, 3, 5, 6, 7, 8], label="Data 2") # Membuat anotasi pada data plt.annotate("Poin Penting", xy=(2, 5), xytext=(3, 7), arrowprops=dict(facecolor='black', shrink=0.05)) # Menampilkan legenda berdasarkan label pada data plt.legend() # Menampilkan figure plt.show()</pre>
	Output: 
Contoh Penggunaan Labeling dalam Plot	
[5]:	<pre># Data x = [1, 2, 3, 4] y1 = [10, 20, 25, 30] y2 = [12, 18, 30, 40]</pre>

	<pre> # Membuat plot fig = plt.figure(figsize=(10,6)) plt.plot(x, y1, label="Contoh Data 1") plt.plot(x, y2, label="Contoh Data 2") # Menambahkan judul dan label plt.title("Contoh Plot dengan Labeling") plt.xlabel("Data X") plt.ylabel("Data Y") # Menambahkan legenda plt.legend() # Menampilkan figure plt.show() </pre>
	Output: 

4.4. Plot *Pairwise* Pada Matplotlib

Plot *pairwise* merujuk pada cara untuk menggambar grafik yang melibatkan pasangan data (x, y) yang direpresentasikan dalam berbagai jenis grafik seperti plot garis, *scatter* plot, *bar* plot, *fill between*, dan *stack* plot. Setiap jenis plot memiliki kegunaan dan karakteristiknya sendiri dalam visualisasi data. Berikut adalah contoh dan implementasi dari plot *pairwise* pada Matplotlib:

Plot Garis (<code>plt.plot()</code>) Plot garis adalah jenis grafik yang paling dasar, digunakan untuk menggambarkan hubungan antara dua set data (x,y) dengan menghubungkan titik-titik data menggunakan garis. Ini sering digunakan untuk menunjukkan tren atau perubahan dari waktu ke waktu.	
[1]:	<pre> # Data x = [1, 2, 3, 4] y1 = [10, 20, 25, 30] y2 = [12, 18, 30, 40] # Membuat figure fig = plt.figure(figsize=(8,6)) # Membuat linechart dengan plt.plot() plt.plot(x, y1, color="green", linewidth=3, label="Data Y1") plt.plot(x, y2, linestyle='--', label="Data Y2") # Labeling plot </pre>

	<pre>plt.title("Contoh Plot Garis") plt.xlabel("Data X") plt.ylabel("Data Y") # Menampilkan figure plt.legend() plt.show()</pre>
	Output: 
	Scatter Plot (`plt.scatter()`) Scatter plot digunakan untuk menggambarkan distribusi dan hubungan antara dua variabel. Setiap titik pada scatter plot mewakili pasangan (x, y), dan tidak ada garis yang menghubungkan titik ini. Scatter plot berguna untuk melihat pola, hubungan korelasi, atau penciran dalam data.
[2]:	<pre># Data x = [1, 2, 3, 4, 5, 6, 7, 8] y1 = [10, 21, 25, 30, 34, 41, 47, 50] y2 = [12, 20, 26, 29, 33, 37, 41, 43] # Membuat figure fig = plt.figure(figsize=(10,5)) # Membuat scatter plot dengan plt.scatter() plt.scatter(x, y1, s=100, c='red', alpha=0.65, label='Data 1') plt.scatter(x, y2, s=50, c='blue', alpha=0.65, label='Data 2') # Labeling plot plt.title("Contoh Scatter Plot") plt.xlabel("Data X") plt.ylabel("Data Y") # Menampilkan figure plt.legend() plt.show()</pre>
	Output: 

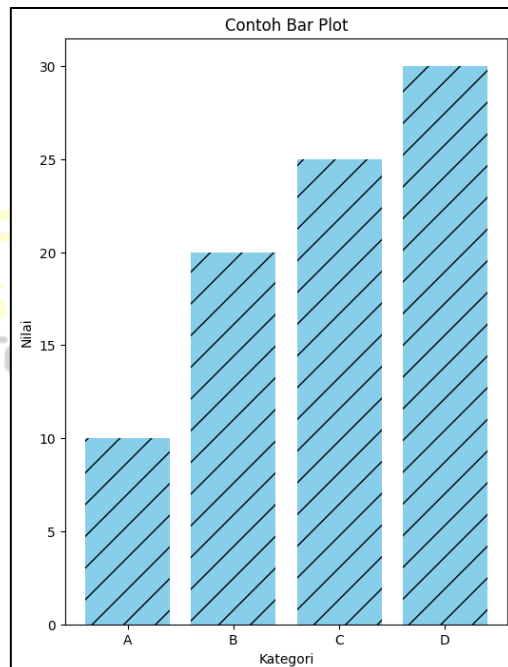
Bar Plot (`plt.bar()`)

Bar plot digunakan untuk merepresentasikan data kategorikal atau untuk membandingkan beberapa nilai data. Panjang batang menunjukkan besarnya nilai data.

```
[3]: # Data
kategori = ['A', 'B', 'C', 'D'] # Label setiap kategori
nilai = [10, 20, 25, 30] # Nilai setiap kategori

# Membuat Figure
fig = plt.figure(figsize=(6,8))
# Membuat bar plot dengan plt.bar()
plt.bar(kategori, nilai, hatch='/', color='skyblue')
# Labeling plot
plt.title("Contoh Bar Plot")
plt.xlabel("Kategori")
plt.ylabel("Nilai")
# Menampilkan figure
plt.show()
```

Output:

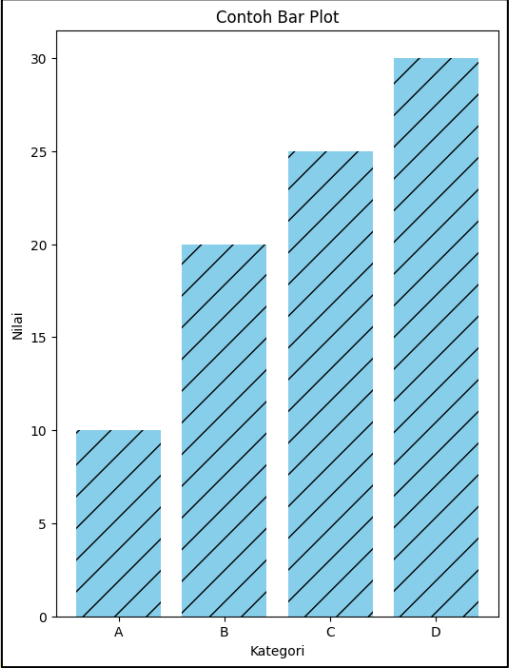


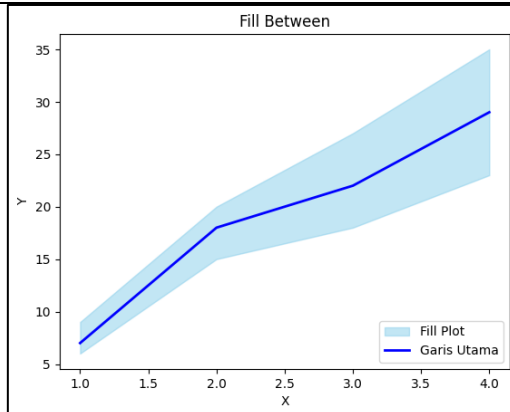
Bar Plot (`plt.bar()`)

Bar plot digunakan untuk merepresentasikan data kategorikal atau untuk membandingkan beberapa nilai data. Panjang batang menunjukkan besarnya nilai data.

```
[4]: # Data
kategori = ['A', 'B', 'C', 'D'] # Label setiap kategori
nilai = [10, 20, 25, 30] # Nilai setiap kategori

# Membuat Figure
fig = plt.figure(figsize=(6,8))
# Membuat bar plot dengan plt.bar()
plt.bar(kategori, nilai, hatch='/', color='skyblue')
# Labeling plot
plt.title("Contoh Bar Plot")
plt.xlabel("Kategori")
```

	<pre>plt.ylabel("Nilai") # Menampilkan figure plt.show()</pre>
	Output: 
	Fill Between (<code>plt.fill_between()</code>) Plot <i>fill between</i> digunakan untuk mengisi area antara dua kurva dan sumbu X. Plot ini sering digunakan untuk menunjukkan area di bawah kurva atau untuk menyoroti kisaran tertentu dari data.
[5]:	<pre># Data x = [1, 2, 3, 4] y1 = [9, 20, 27, 35] y_main = [7, 18, 22, 29] # Data garis utama y2 = [6, 15, 18, 23] # Membuat figure fig = plt.plot(figsize=(10,6)) # Membuat fill_between plot plt.fill_between(x, y1, y2, color='skyblue', alpha=0.5, label='Fill Plot') # Membuat plot garis utama plt.plot(x, y_main, color='blue', linewidth=2, label='Garis Utama') # Labeling plot plt.title("Fill Between") plt.xlabel("X") plt.ylabel("Y") # Menampilkan figure plt.legend(loc='lower right') plt.show()</pre>
	Output:



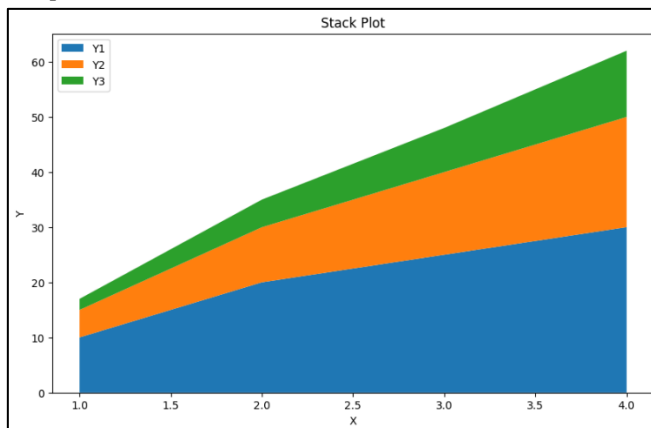
Stack Plot (`plt.stackplot()`)

Stack plot digunakan untuk menampilkan beberapa set data yang ditumpuk di atas satu sama lain. Ini membantu melihat bagaimana komponen-komponen individual berkontribusi terhadap total secara kumulatif.

```
[6]: # Data
x = [1, 2, 3, 4]
y1 = [10, 20, 25, 30]
y2 = [5, 10, 15, 20]
y3 = [2, 5, 8, 12]

# Membuat figure
fig = plt.figure(figsize=(10,6))
# Membuat stack plot dengan .stackplot()
plt.stackplot(x, y1, y2, y3, labels=['Y1', 'Y2', 'Y3'])
# Labeling plot
plt.title("Stack Plot")
plt.xlabel("X")
plt.ylabel("Y")
# Menampilkan figure
plt.legend(loc='upper left')
plt.show()
```

Output:



4.5. Plot Distribusi Statistik Pada Matplotlib

Plot distribusi statistik digunakan untuk menggambarkan bagaimana data tersebar, memberikan gambaran tentang distribusi, median, *outlier*, dan variasi dalam dataset. Berikut adalah penjelasan tentang beberapa jenis plot distribusi statistik yang umum digunakan pada Matplotlib:

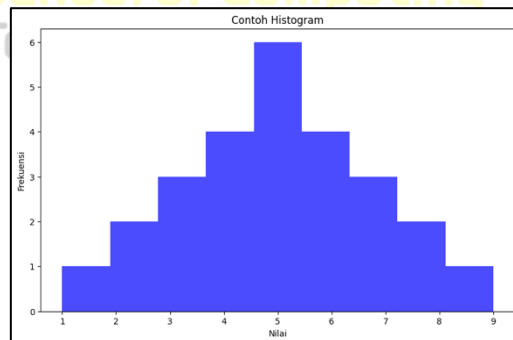
Histogram (`plt.hist()`)

Histogram adalah jenis plot yang digunakan untuk mewakili distribusi frekuensi dari dataset. Data dibagi menjadi *bin* atau interval, dan tinggi dari setiap batang mewakili jumlah data yang jatuh dalam interval tersebut. *Histogram* berguna untuk memahami distribusi data secara keseluruhan, termasuk mengidentifikasi *skewness*, *outlier*, dan *mode*.

```
[1]: # Data
data = [1,9,2,2,8,8,7,7,7,3,3,3,4,4,4,4,6,6,6,6,5,5,5,5,5]

# Membuat plot
fig = plt.figure(figsize=(10,6))
# Membuat histogram dengan .hist()
plt.hist(data, bins=9, color="blue", alpha=0.7)
# Labeling plot
plt.title("Contoh Histogram")
plt.xlabel("Nilai")
plt.ylabel("Frekuensi")
# Menampilkan figure
plt.show()
```

Output:

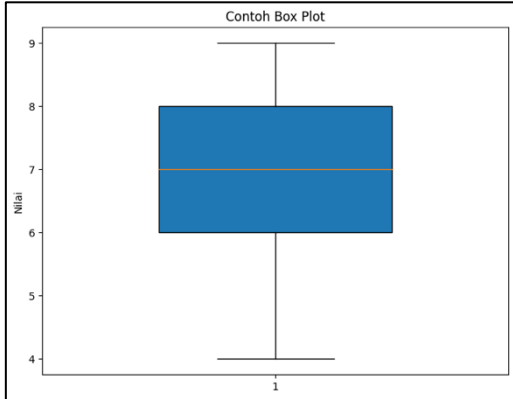
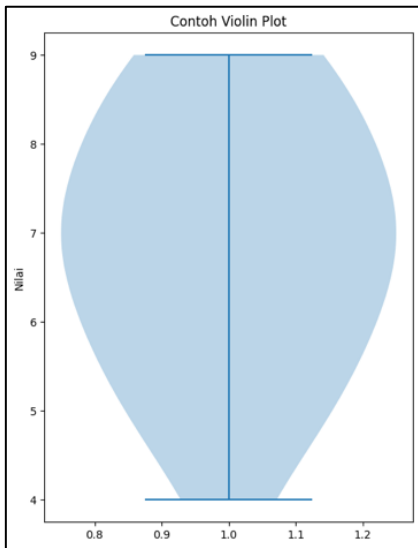


Box Plot (`plt.boxplot()`)

Box plot adalah plot yang menampilkan distribusi data berdasarkan lima angka utama: minimum, Q1, median, Q3, dan maksimum. *Box plot* berguna untuk menganalisis distribusi data, menemukan *outlier*, dan membandingkan distribusi antar beberapa grup data.

```
[2]: # Data
data = [7, 5, 8, 9, 6, 7, 5, 8, 6, 7, 6, 8, 9, 4, 7, 5, 9, 8,
6, 7]

# Membuat plot
fig = plt.figure(figsize=(8,6))
# Membuat box plot dengan .boxplot()
plt.boxplot(data, patch_artist=True, widths=0.50)
# Labeling Plot
plt.title("Contoh Box Plot")
```

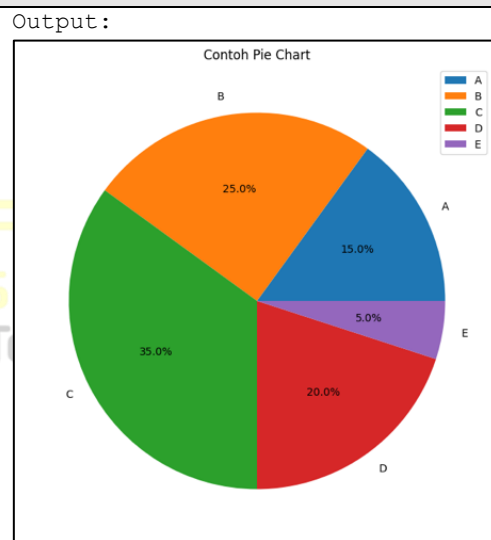
	<pre>plt.ylabel("Nilai") # Menampilkan figure plt.show()</pre>
	Output: 
	Violin Plot (`plt.violinplot()`) Violin plot adalah kombinasi dari <i>box plot</i> dan <i>kernel density</i> plot. Ini tidak hanya menyampaikan informasi yang diberikan oleh <i>box plot</i>, tetapi juga memberikan gambaran tentang distribusi probabilitas data di berbagai level nilai. Bentuk <i>violin</i> plot menunjukkan kepadatan data. Plot ini berguna untuk memahami distribusi data, terutama kasus data yang memiliki lebih dari satu puncak distribusi.
	<pre>[3]: # Data data = [7, 5, 8, 9, 6, 7, 5, 8, 6, 7, 6, 8, 9, 4, 7, 5, 9, 8, 6, 7] # Membuat Plot fig = plt.figure(figsize=(6,8)) # Membuat violin plot dengan .violinplot() plt.violinplot(data) # Labeling Plot plt.title("Contoh Violin Plot") plt.ylabel("Nilai") # Menampilkan figure plt.show()</pre>
	Output: 

Pie Chart (`plt.pie()`)

Pie chart adalah plot berbentuk lingkaran yang digunakan untuk menunjukkan proporsi atau presentase dari seluruh data. Setiap irisan dalam *pie chart* mewakili proporsi data dalam kategori tertentu. *Pie chart* berguna untuk memvisualisasikan proporsi data dalam suatu kategori terhadap keseluruhan data dalam konteks data kategorikal.

```
[4]: # Data
label = ['A', 'B', 'C', 'D', 'E']
nilai = [15, 25, 35, 20, 5]

# Membuat Plot
fig = plt.figure(figsize=(8,8))
# Membuat pie chart dengan .pie()
plt.pie(nilai, labels=label, autopct='%1.1f%%')
# Labeling plot
plt.title("Contoh Pie Chart")
# Menampilkan figure
plt.legend()
plt.show()
```



4.6. Soal Latihan *Library* Matplotlib

1. Tampilkan grafik nilai maksimum, minimum, dan rata-rata untuk setiap grup angkatan masuk mahasiswa dari data yang terdapat pada Tabel 5!

Tabel 5. Data nilai mahasiswa untuk soal latihan *library* Matplotlib.

Nama	Algoritma Pemrograman	Matematika Diskrit	Dasar Kecerdasan Artifisial	Tahun Angkatan
Budi	81.24	73.66	95.89	2023
Siti	98.52	84.86	88.7	2024
Asep	91.96	71.03	79.93	2022

Dewi	87.96	97.28	71.91	2024
Andi	74.68	77.76	79.33	2023
Lestari	74.68	89.88	79.76	2022
Joko	71.74	79.35	91.89	2022
Rina	95.99	85.6	89.13	2022
Fajar	88.03	86.4	96.62	2024
Indah	91.24	75.55	84.17	2023
Agus	70.62	99.09	73.59	2022
Fitri	99.1	93.25	91.4	2022
Rudi	94.97	98.18	92.82	2022
Maya	76.37	96.84	86.84	2024
Hadi	75.45	87.94	93.13	2024
Sri	75.5	97.66	84.81	2023
Wawan	79.13	72.65	85.68	2024
Yuni	85.74	75.88	82.83	2022
Bambang	82.96	71.36	70.76	2023
Ayu	78.74	79.76	73.24	2022

2. Tampilkan gambar graf dari soal latihan di Subbab 3.11 ke layar dengan menggunakan library Matplotlib! Silakan diatur sendiri posisi absis dan ordinat (X dan Y) untuk setiap simpul/kota-nya, sedemikian sehingga tampilan yang dihasilkan oleh Matplotlib menyerupai gambar graf aslinya.

Modul 5 IMPLEMENTASI *BREADTH-FIRST SEARCH*

Tujuan Praktikum:

1. Mahasiswa memahami dan mampu mengimplementasikan algoritma *Breadth-First Search* pada Python dengan menggunakan *library* NetworkX.

Referensi:

https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.traversal.breadth_first_search.bfs_edges.html#networkx.algorithms.traversal.breadth_first_search.bfs_edges

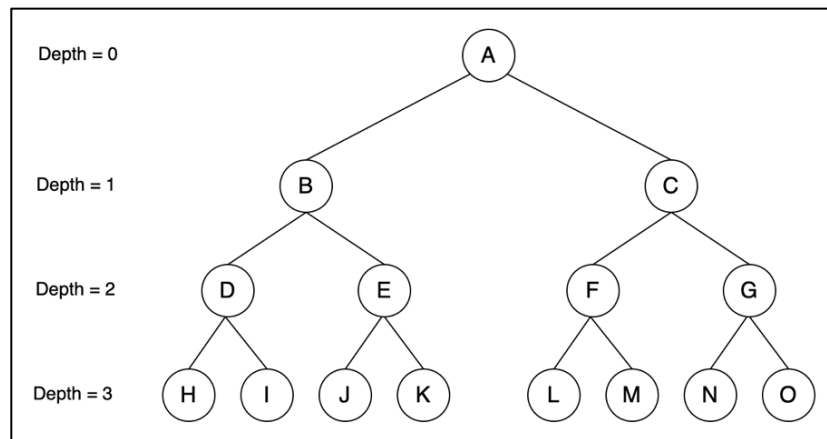
Breadth-First Search (BFS) adalah algoritma pencarian atau traversal dalam struktur graf atau *tree*. BFS bekerja dengan mengeksplorasi semua tetangga dari suatu *node* sebelum bergerak ke level berikutnya. BFS sangat berguna untuk menemukan jalur terpendek.

Dalam NetworkX, BFS dapat dengan mudah diimplementasikan menggunakan fungsi bawaan. NetworkX menggunakan ``nx.bfs_edges()`` untuk mengembalikan urutan *node* yang dieksplorasi oleh BFS. Algoritma BFS pada memungkinkan untuk menjelajahi graf secara sistematis dan mendapatkan urutan eksplorasi yang dihasilkan.

5.1. Implementasi BFS Pada NetworkX

5.1.1. Implementasi BFS Pada *Tree*

Implementasi BFS *tree* dimulai dari akar (*root*) dan menjelajahi semua *node* pada level yang sama sebelum berpindah ke level berikutnya. BFS akan menjelajahi node pada depth 0 (*root*), kemudian pada node kedalaman 1, dan seterusnya.

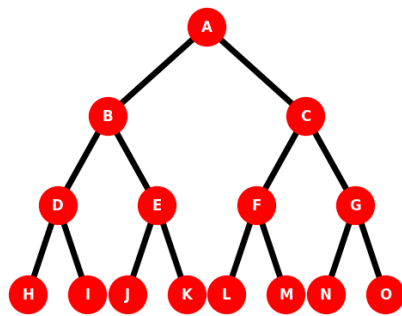


Gambar 10. Contoh Implementasi Tree untuk BFS.

Berikut adalah kode implementasi untuk BFS pada *tree* untuk Gambar 10:

Import <i>library</i> yang digunakan	
[1]:	<pre>import networkx as nx # untuk membuat grafik import matplotlib.pyplot as plt # untuk plot grafik</pre>
Function untuk cetak plot	
[2]:	<pre># Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='') : # Membuat pos jika pos tidak diberikan if pos is None: pos = nx.spring_layout(G) # Fungsi untuk menggambar node nx.draw(G, # Graf NetworkX pos, # Posisi node with_labels=True, # Menampilkan nama node node_color='red', # Warna node node_size=2000, # Ukuran node font_color="white", # Warna font label node font_weight="bold", # Ketebalan font label node width=5 # Ketebalan garis edge) # Mengambil label edge jika ada weight edge_labels = nx.get_edge_attributes(G, 'weight') # Fungsi untuk menggambar node nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, # Data weight font_color='blue', # Warna font label edge font_weight="bold", # Berat font label edge font_size=12, # Ukuran font label edge) plt.margins(0.2) # Memberikan margin pada plot plt.title(title) # Menampilkan judul graf jika diberikan</pre>

	<code>plt.show()</code> # Menampilkan graf menggunakan matplotlib
Posisi node `pos` agar sesuai dengan gambar contoh	
[3]:	<pre>pos = { 'A': (0, 3), 'B': (-10, 2), 'C': (10, 2), 'D': (-15, 1), 'E': (-5, 1), 'F': (5, 1), 'G': (15, 1), 'H': (-18, 0), 'I': (-12, 0), 'J': (-8, 0), 'K': (-2, 0), 'L': (2, 0), 'M': (8, 0), 'N': (12, 0), 'O': (18, 0) }</pre>
Membuat tree dengan NetworkX	
[4]:	<pre># Inisialisasi graf kosong G = nx.Graph()</pre>
[5]:	<pre># Definisi node dan edge sesuai dengan gambar edges = [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'F'), ('C', 'G'), ('D', 'H'), ('D', 'I'), ('E', 'J'), ('E', 'K'), ('F', 'L'), ('F', 'M'), ('G', 'N'), ('G', 'O')] # Menambahkan edge pada graf G.add_edges_from(edges)</pre>
Evaluasi elemen pada graf	
[6]:	<pre>print("Elemen pada Graf:") print("Daftar Node pada Graf:", G.nodes()) print("Daftar Edge pada Graf:", G.edges()) print("Jumlah Node pada Graf", G.number_of_nodes()) print("Jumlah Edge pada Graf", G.number_of_edges())</pre>
	<pre>Output: Elemen pada Graf: Daftar Node pada Graf: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O'] Daftar Edge pada Graf: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'F'), ('C', 'G'), ('D', 'H'), ('D', 'I'), ('E', 'J'), ('E', 'K'), ('F', 'L'), ('F', 'M'), ('G', 'N'), ('G', 'O')] Jumlah Node pada Graf 15 Jumlah Edge pada Graf 14</pre>
Visualisasi graf	
[7]:	<pre># Mencetak graf show_graph(G, pos=pos)</pre>
	Output:



Urutan Breadth-First Search

```
[8]: # Buat loop untuk urutan Breadth First Search
print("Urutan node yang dikunjungi:")
i = 1
for nodes in nx.bfs_edges(G, source='A'):
    print(f"{i}. {nodes[0]} - {nodes[1]}")
    i += 1
```

Output:
Urutan node yang dikunjungi:

1. A - B
2. A - C
3. B - D
4. B - E
5. C - F
6. C - G
7. D - H
8. D - I
9. E - J
10. E - K
11. F - L
12. F - M
13. G - N
14. G - O

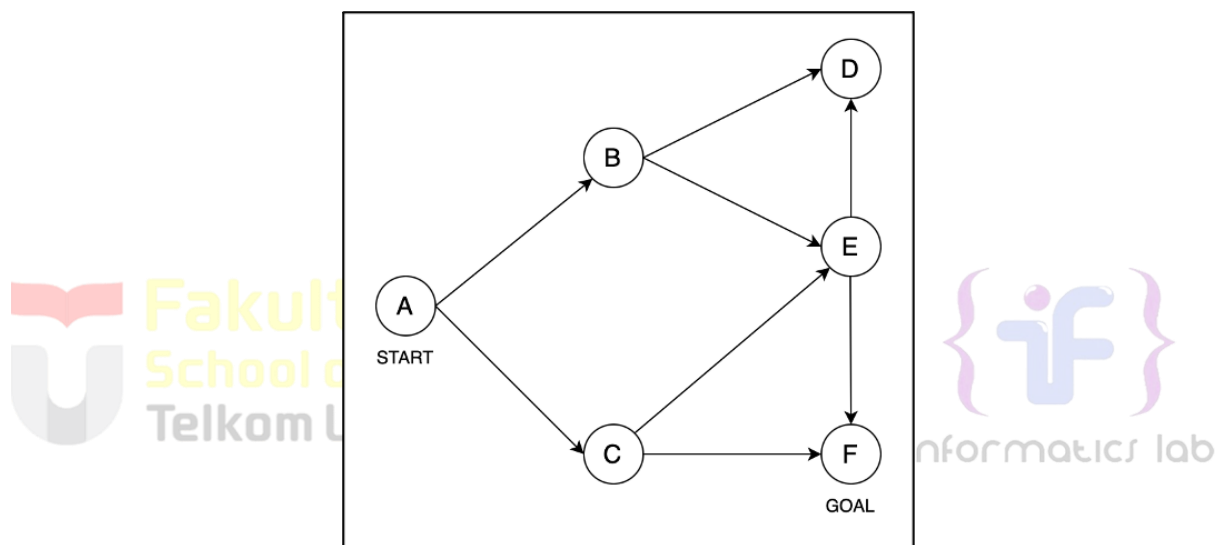
```
[9]: # loop untuk urutan Breadth First Search dengan depth 2
print("Urutan node yang dikunjungi:")
i = 1
for nodes in nx.bfs_edges(G, source='A', depth_limit=2):
    print(f"{i}. {nodes[0]} - {nodes[1]}")
    i += 1
```

Output:
Urutan node yang dikunjungi:

1. A - B
2. A - C
3. B - D
4. B - E
5. C - F
6. C - G

5.1.2. Implementasi BFS Pada *Directed Graph*

BFS pada *directed graph* dimulai dari sebuah node awal (*source node*) dan mengeksplorasi semua tetangga yang dapat dicapai langsung dari *node* tersebut. Setelah semua tetangga pada level pertama dijelajahi, BFS melanjutkan ke tetangga dari seluruh *node* pada tingkat ini, tingkat berikutnya, dan seterusnya. Pada *directed graph*, BFS hanya mengikuti arah *edge* saat menjelajahi graf. BFS pada *directed graph* dapat digunakan untuk menentukan jalur terpendek dari satu *node* ke *node* lain dalam graf tidak berbobot.

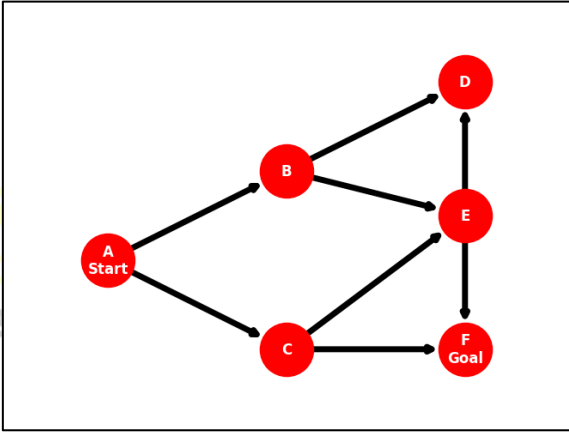


Gambar 11. Contoh directed graf untuk implementasi DFS.

Berikut adalah kode untuk implementasi BFS pada *directed graph* untuk Gambar 11:

Import library yang digunakan	
[1]:	<pre>import networkx as nx # untuk membuat grafik import matplotlib.pyplot as plt # untuk plot grafik</pre>
Function untuk cetak plot	
[2]:	<pre># Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='', labels=labels) : # Membuat pos jika pos tidak diberikan if pos is None: pos = nx.spring_layout(G) # Fungsi untuk menggambar node nx.draw(G, # Graf NetworkX pos, # Posisi node with_labels=True, # Menampilkan nama node labels=labels # Menampilkan label setiap node</pre>

	<pre> node_color='red', # Warna node node_size=2000, # Ukuran node font_color="white", # Warna font label node font_weight="bold", # Ketebalan font label node width=5 # Ketebalan garis edge) # Mengambil label edge jika ada weight edge_labels = nx.get_edge_attributes(G, 'weight') # Fungsi untuk menggambar node nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, # Data weight font_color='blue', # Warna font label edge font_weight="bold", # Berat font label edge font_size=12, # Ukuran font label edge) plt.margins(0.2) # Memberikan margin pada plot plt.title(title) # Menampilkan judul graf jika diberikan plt.show() # Menampilkan graf menggunakan matplotlib </pre>
Posisi node `pos` dan label start dan goal agar sesuai dengan gambar contoh	
[3]:	<pre> pos = { 'A': (0, 1), 'B': (1, 2), 'C': (1, 0), 'D': (2, 3), 'E': (2, 1.5), 'F': (2, 0) } # Mencatat start dan goal node labels = {node: node for node in G.nodes()} labels['A'] = 'A\nStart' labels['F'] = 'F\nGoal' </pre>
Membuat <i>directed graph</i> dengan NetworkX	
[4]:	<pre> # Inisialisasi graf berarah kosong G = nx.DiGraph() </pre>
[5]:	<pre> # Definisi node dan edge sesuai dengan gambar edges = [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'F'), ('C', 'G'), ('D', 'H'), ('D', 'I'), ('E', 'J'), ('E', 'K'), ('F', 'L'), ('F', 'M'), ('G', 'N'), ('G', 'O')] # Menambahkan edge pada graf G.add_edges_from(edges) </pre>
Menambahkan label <i>start</i> dan <i>goal</i>	
[6]:	<pre> # Mencatat start dan goal node labels = {node: node for node in G.nodes()} </pre>

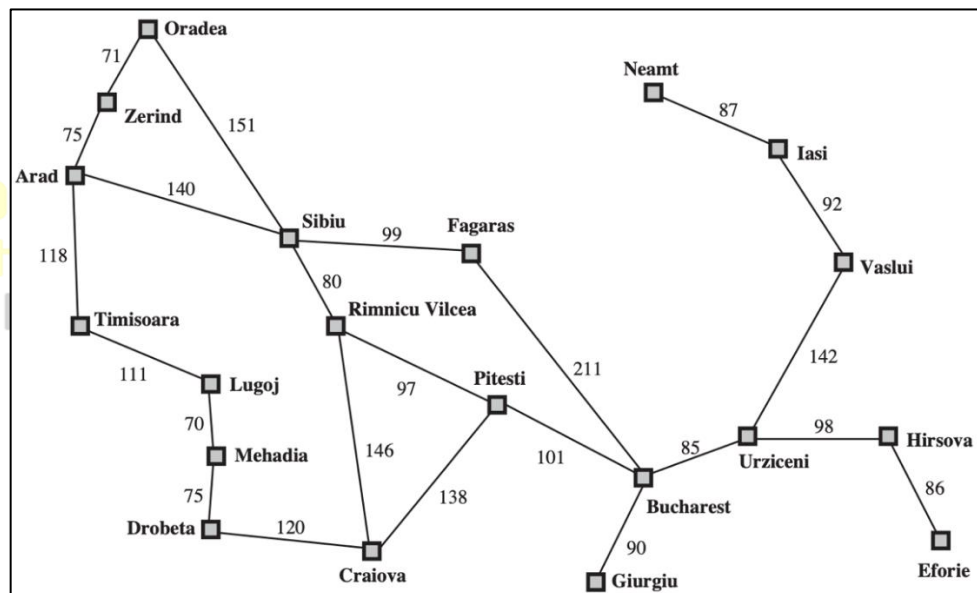
	<pre> labels['A'] = 'A\nStart' labels['F'] = 'F\nGoal' </pre>
Evaluasi elemen pada graf	
[7]:	<pre> print("Elemen pada Graf:") print("Daftar Node pada Graf:", G.nodes()) print("Daftar Edge pada Graf:", G.edges()) print("Jumlah Node pada Graf", G.number_of_nodes()) print("Jumlah Edge pada Graf", G.number_of_edges()) </pre>
	Output: Elemen pada Graf: Daftar Node pada Graf: ['A', 'B', 'C', 'D', 'E', 'F'] Daftar Edge pada Graf: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'E'), ('C', 'F'), ('E', 'D'), ('E', 'F')] Jumlah Node pada Graf 6 Jumlah Edge pada Graf 8
Visualisasi graf	
[8]:	<pre> # Mencetak graf show_graph(G, pos=pos, labels=labels) </pre>
	Output: 
Urutan Breadth-First Search	
[9]:	<pre> # Buat loop untuk urutan Breadth First Search print("Urutan node yang dikunjungi:") i = 1 for nodes in nx.bfs_edges(G, source='A'): print(f"{i}. {nodes[0]} - {nodes[1]}") i += 1 </pre>
	Output: Urutan node yang dikunjungi: 1. A - B 2. A - C 3. B - D 4. B - E 5. C - F
[10]:	<pre> # loop untuk urutan Breadth First Search dengan depth 1 print("Urutan node yang dikunjungi:") i = 1 for nodes in nx.bfs_edges(G, source='A', depth_limit=1): print(f"{i}. {nodes[0]} - {nodes[1]}") i += 1 </pre>
	Output: Urutan node yang dikunjungi:

- | |
|----------|
| 1. A - B |
| 2. A - C |

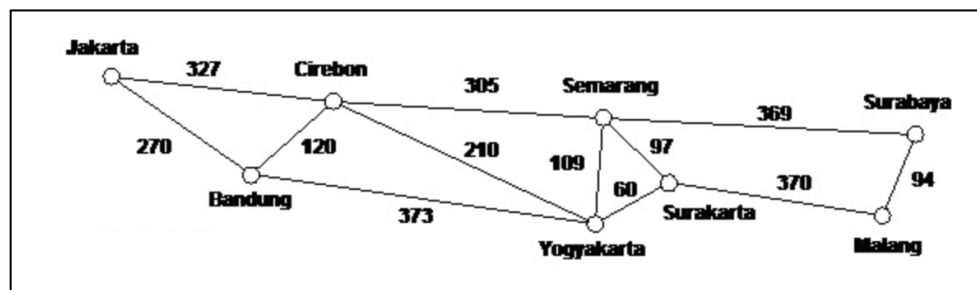
5.2. Soal Latihan

1. Buatlah graf berbobot (*weighted graf*) dari 2 gambar graf berikut ini menggunakan library NetworkX. Kemudian lakukan penelusuran *Breadth-First Search* menggunakan library NetworkX! Kemudian periksalah hasilnya secara manual! Untuk graf Eropa, kota asal (*source*)-nya adalah Arad. Untuk graf Pulau Jawa, kota asal (*source*)-nya adalah Bandung. Periksa apakah informasi weight digunakan atau tidak pada algoritma Breadth First Search ini!

a. Graf peta kota-kota besar di Eropa:



b. Graf peta kota-kota besar di Indonesia (pulau Jawa):



Modul 6 IMPLEMENTASI *DEPTH-FIRST SEARCH*

Tujuan Praktikum:

1. Mahasiswa memahami dan mampu mengimplementasikan algoritma *Depth-First Search* pada Python dengan menggunakan *library* NetworkX.

Referensi:

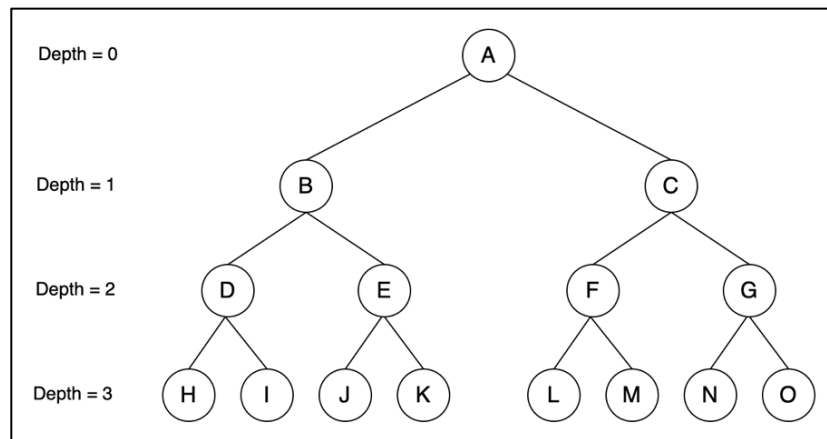
https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.traversal.depth_first_search.dfs_edges.html#networkx.algorithms.traversal.depth_first_search.dfs_edges

Depth-First Search (DFS) adalah algoritma pencarian atau traversal yang dapat digunakan pada struktur graf atau *tree*. DFS bekerja dengan mengeksplorasi satu cabang atau jalur sedalam mungkin sebelum kembali dan mengeksplorasi jalur lain. DFS digunakan untuk mencari semua jalur yang mungkin untuk menyelesaikan masalah yang terlibat pencarian mendalam.

Dalam NetworkX, DFS dapat dengan mudah diimplementasikan menggunakan fungsi `nx.dfs_edges()` untuk mengembalikan urutan *node* yang dieksplorasi oleh DFS. Algoritma DFS memungkinkan untuk menjelajahi graf secara sistematis dan mendapatkan urutan eksplorasi yang dihasilkan berdasarkan pencarian mendalam pada setiap *node* sebelum pindah ke *node* berikutnya.

6.1. Implementasi DFS Pada *Tree*

DFS dimulai dari *node* akar (*root*) dan mengikuti jalur hingga mencapai daun (*leaf*) atau titik terakhir, lalu kembali ke *node* sebelumnya untuk mengeksplorasi jalur lain yang belum dikunjungi.

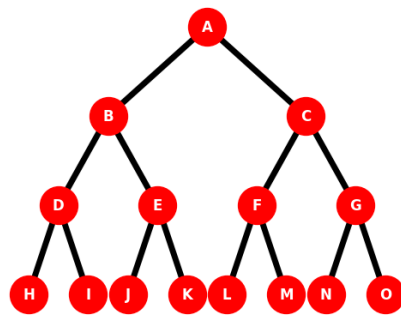


Gambar 12. Contoh tree untuk DFS.

Berikut adalah kode implementasi untuk DFS pada *tree* untuk Gambar 12:

Import library yang digunakan	
[1]:	<pre>import networkx as nx # untuk membuat grafik import matplotlib.pyplot as plt # untuk plot grafik</pre>
Function untuk cetak plot	
[2]:	<pre># Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='') : # Membuat pos jika pos tidak diberikan if pos is None: pos = nx.spring_layout(G) # Fungsi untuk menggambar node nx.draw(G, # Graf NetworkX pos, # Posisi node with_labels=True, # Menampilkan nama node node_color='red', # Warna node node_size=2000, # Ukuran node font_color="white", # Warna font label node font_weight="bold", # Ketebalan font label node width=5 # Ketebalan garis edge) # Mengambil label edge jika ada weight edge_labels = nx.get_edge_attributes(G, 'weight') # Fungsi untuk menggambar node nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, # Data weight font_color='blue', # Warna font label edge font_weight="bold", # Berat font label edge font_size=12, # Ukuran font label edge) plt.margins(0.2) # Memberikan margin pada plot plt.title(title) # Menampilkan judul graf jika diberikan</pre>

	<code>plt.show()</code> # Menampilkan graf menggunakan matplotlib
Posisi node `pos` agar sesuai dengan gambar contoh	
[3]:	<pre>pos = { 'A': (0, 3), 'B': (-10, 2), 'C': (10, 2), 'D': (-15, 1), 'E': (-5, 1), 'F': (5, 1), 'G': (15, 1), 'H': (-18, 0), 'I': (-12, 0), 'J': (-8, 0), 'K': (-2, 0), 'L': (2, 0), 'M': (8, 0), 'N': (12, 0), 'O': (18, 0) }</pre>
Membuat tree dengan NetworkX	
[4]:	<pre># Inisialisasi graf kosong G = nx.Graph()</pre>
[5]:	<pre># Definisi node dan edge sesuai dengan gambar edges = [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'F'), ('C', 'G'), ('D', 'H'), ('D', 'I'), ('E', 'J'), ('E', 'K'), ('F', 'L'), ('F', 'M'), ('G', 'N'), ('G', 'O')] # Menambahkan edge pada graf G.add_edges_from(edges)</pre>
Evaluasi elemen pada graf	
[5]:	<pre>print("Elemen pada Graf:") print("Daftar Node pada Graf:", G.nodes()) print("Daftar Edge pada Graf:", G.edges()) print("Jumlah Node pada Graf", G.number_of_nodes()) print("Jumlah Edge pada Graf", G.number_of_edges())</pre>
	Output: Elemen pada Graf: Daftar Node pada Graf: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O'] Daftar Edge pada Graf: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'F'), ('C', 'G'), ('D', 'H'), ('D', 'I'), ('E', 'J'), ('E', 'K'), ('F', 'L'), ('F', 'M'), ('G', 'N'), ('G', 'O')] Jumlah Node pada Graf 15 Jumlah Edge pada Graf 14
Visualisasi graf	
[6]:	<pre># Mencetak graf show_graph(G, pos=pos)</pre>
	Output:



Urutan Depth-First Search

```
[7]: # Buat loop untuk urutan Depth First Search
print("Urutan node yang dikunjungi:")
i = 1
for nodes in nx.dfs_edges(G, source='A'):
    print(f"{i}. {nodes[0]} - {nodes[1]}")
    i += 1
```

Output:

Urutan node yang dikunjungi:

1. A - B
2. B - D
3. D - H
4. D - I
5. B - E
6. E - J
7. E - K
8. A - C
9. C - F
10. F - L
11. F - M
12. C - G
13. G - N
14. G - O

```
[8]: # loop untuk urutan Depth First Search dengan depth 2
print("Urutan node yang dikunjungi:")

i = 1
for nodes in nx.dfs_edges(G, source='A', depth_limit=2):
    print(f"{i}. {nodes[0]} - {nodes[1]}")
    i += 1
```

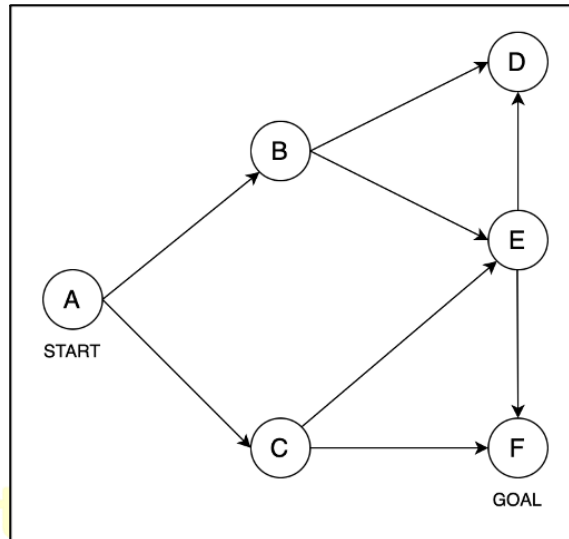
Output:

Urutan node yang dikunjungi:

1. A - B
2. B - D
3. B - E
4. A - C
5. C - F
6. C - G

6.2. Implementasi DFS Pada *Directed Graph*

Implementasi DFS pada *directed graph* melibatkan eksplorasi graf berarah dengan menyelidiki setiap jalur atau cabang dari *node* awal sedalam mungkin sebelum beralih jalur lain. Dalam *directed graph*, arah dari setiap *edge* diperhatikan, sehingga DFS hanya mengikuti arah *edge* ketika menjelajahi *node*.

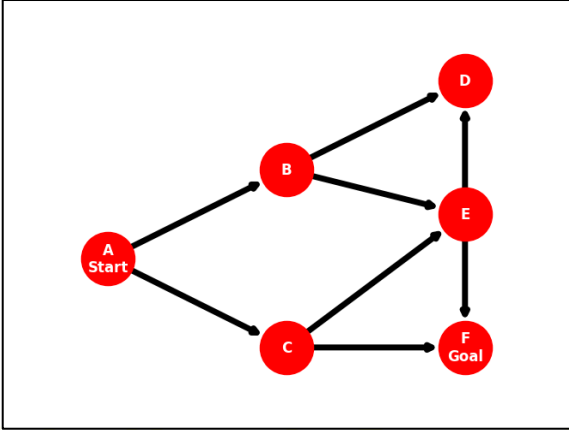


Gambar 13. Contoh *directed graph* untuk DFS.

Berikut adalah kode implementasi untuk DFS pada *directed graph* untuk Gambar 13:

Import <i>library</i> yang digunakan	
[1]:	<pre>import networkx as nx # untuk membuat grafik import matplotlib.pyplot as plt # untuk plot grafik</pre>
Function untuk cetak plot	
[2]:	<pre># Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='', labels={}): # Membuat pos jika pos tidak diberikan if pos is None: pos = nx.spring_layout(G) # Fungsi untuk menggambar node nx.draw(G, # Graf NetworkX pos, # Posisi node with_labels=True, # Menampilkan nama node labels=labels, # Menampilkan label setiap node node_color='red', # Warna node node_size=2000, # Ukuran node font_color="white", # Warna font label node font_weight="bold", # Ketebalan font label node width=5 # Ketebalan garis edge</pre>

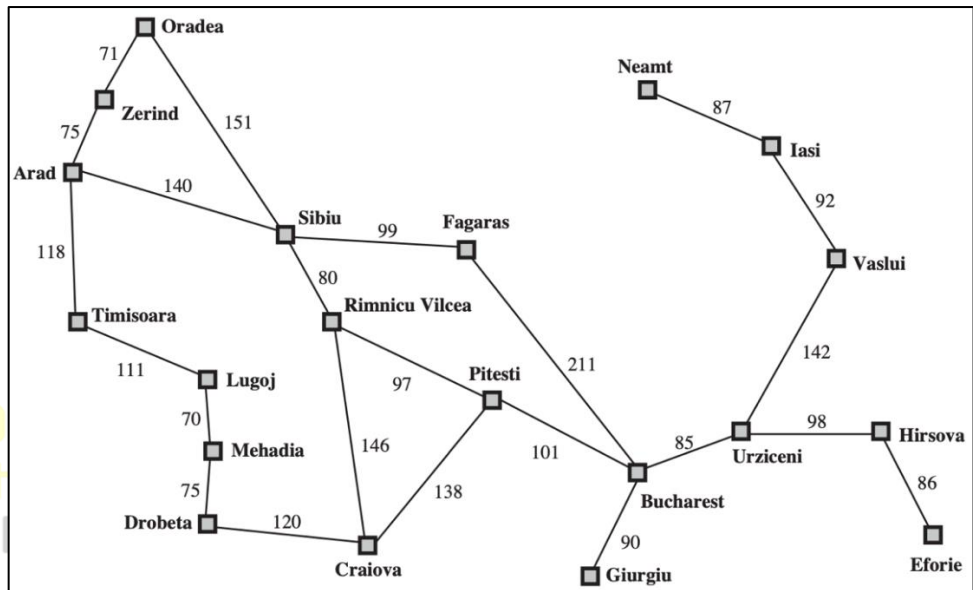
	<pre>) # Mengambil label edge jika ada weight edge_labels = nx.get_edge_attributes(G, 'weight') # Fungsi untuk menggambar node nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, # Data weight font_color='blue', # Warna font label edge font_weight="bold", # Berat font label edge font_size=12, # Ukuran font label edge) plt.margins(0.2) # Memberikan margin pada plot plt.title(title) # Menampilkan judul graf jika diberikan plt.show() # Menampilkan graf menggunakan matplotlib </pre>
Posisi node `pos` dan label <i>start</i> dan <i>goal</i> agar sesuai dengan gambar contoh	
[3]:	<pre> pos = { 'A': (0, 1), 'B': (1, 2), 'C': (1, 0), 'D': (2, 3), 'E': (2, 1.5), 'F': (2, 0) } </pre>
Membuat <i>directed graph</i> dengan NetworkX	
[4]:	<pre> # Inisialisasi graf berarah kosong G = nx.DiGraph() </pre>
[5]:	<pre> # Definisi node dan edge sesuai dengan gambar edges = [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'F'), ('C', 'G'), ('D', 'H'), ('D', 'I'), ('E', 'J'), ('E', 'K'), ('F', 'L'), ('F', 'M'), ('G', 'N'), ('G', 'O')] # Menambahkan edge pada graf G.add_edges_from(edges) </pre>
Evaluasi elemen pada graf	
[6]:	<pre> print("Elemen pada Graf:") print("Daftar Node pada Graf:", G.nodes()) print("Daftar Edge pada Graf:", G.edges()) print("Jumlah Node pada Graf", G.number_of_nodes()) print("Jumlah Edge pada Graf", G.number_of_edges()) </pre>
	<pre> Output: Elemen pada Graf: Daftar Node pada Graf: ['A', 'B', 'C', 'D', 'E', 'F'] Daftar Edge pada Graf: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'E'), ('C', 'F'), ('E', 'D'), ('E', 'F')] </pre>

	Jumlah Node pada Graf 6 Jumlah Edge pada Graf 8
Menambahkan label start dan goal	
[7]:	<pre># Mencatat start dan goal node labels = {node: node for node in G.nodes()} labels['A'] = 'A\nStart' labels['F'] = 'F\nGoal'</pre>
Visualisasi graf	
[8]:	<pre># Mencetak graf show_graph(G, pos=pos, labels=labels)</pre>
	Output:  <pre>graph LR A((A Start)) --> B((B)) A --> C((C)) B --> D((D)) B --> E((E)) C --> E C --> F((F Goal)) D --> E E --> F</pre>
Urutan Depth-First Search	
[9]:	<pre># Buat loop untuk urutan Depth First Search print("Urutan node yang dikunjungi:") i = 1 for nodes in nx.dfs_edges(G, source='A'): print(f"{i}. {nodes[0]} - {nodes[1]}") i += 1</pre>
	Output: Urutan node yang dikunjungi: 1. A - B 2. B - D 3. B - E 4. E - F 5. A - C
[10]:	<pre># loop untuk urutan Depth First Search dengan depth 1 print("Urutan node yang dikunjungi:") i = 1 for nodes in nx.dfs_edges(G, source='A', depth_limit=1): print(f"{i}. {nodes[0]} - {nodes[1]}") i += 1</pre>
	Output: Urutan node yang dikunjungi: 1. A - B 2. A - C

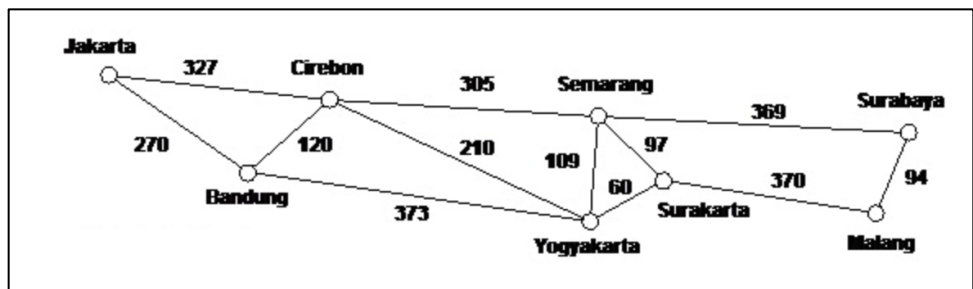
6.3. Soal Latihan

1. Buatlah graf berbobot (*weighted graf*) dari 2 gambar graf berikut ini menggunakan library NetworkX. Kemudian lakukan penelusuran *Depth First Search* menggunakan library NetworkX! Kemudian periksalah hasilnya secara manual! Untuk graf kota-kota besar di Eropa, kota asal (*source*)-nya adalah Arad. Untuk graf kota-kota Pulau Jawa, kota asal (*source*)-nya adalah Bandung. Periksa apakah informasi weight digunakan atau tidak pada algoritma Breadth First Search ini!

a. Graf peta kota-kota besar di Eropa:



b. Graf peta kota-kota besar di Indonesia (pulau Jawa):



Modul 7 IMPLEMENTASI *UNIFORM COST SEARCH*

Tujuan Praktikum:

1. Mahasiswa memahami dan mampu mengimplementasikan algoritma *Uniform Cost Search* pada Python dengan menggunakan *library* NetworkX.

Referensi:

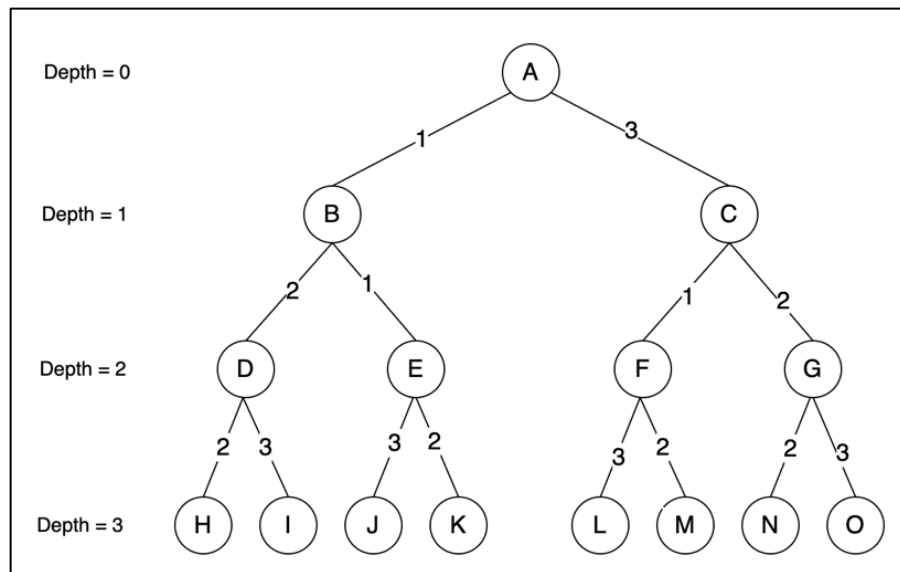
https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.shortest_paths.astar.astar_path.html

Uniform Cost Search (UCS) adalah algoritma pencarian jalur optimal dalam graf yang mencari jalur dengan *cost* terendah dari *node* awal (*start*) ke *node* tujuan (*goal*). UCS merupakan varian dari BFS, tetapi mempertimbangkan *weight* dari setiap *edge* dalam graf. Algoritma ini berguna untuk menemukan jalur dengan total *weight* terendah.

Dalam NetworkX, UCS diimplementasikan dengan memanfaatkan fungsi ``nx.astar_path()`` dengan menetapkan nilai heuristik (``heuristic``) menjadi 0, algoritma A* pada dasarnya berfungsi sebagai UCS karena hanya mempertimbangkan biaya sebenarnya tanpa menambahkan estimasi ke tujuan.

7.1. Implementasi UCS Pada *Tree*

UCS pada *tree* merupakan pencarian yang bertujuan untuk menemukan jalur dari *root* ke *node* tujuan dengan biaya total terendah. Pada *tree*, UCS menjelajahi dengan memilih *node* yang memiliki biaya kumulatif terendah dari *root*.

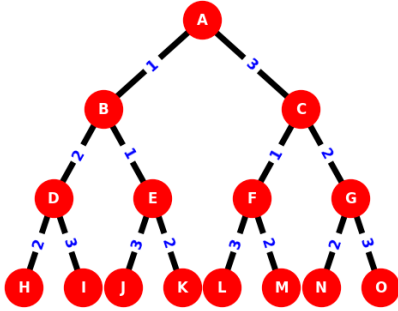


Gambar 14. Contoh tree untuk UCS.

Berikut adalah kode implementasi untuk UCS pada *tree* untuk Gambar 14:

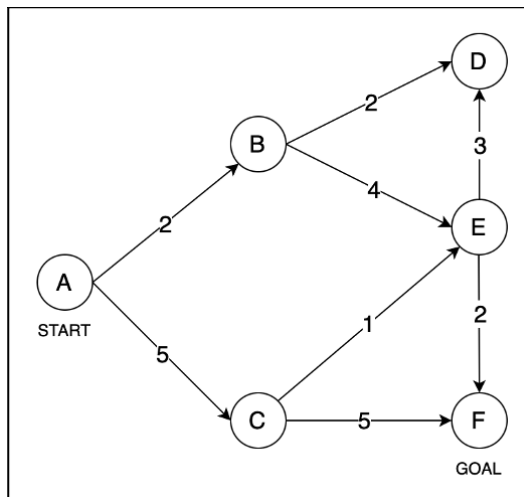
Import library yang digunakan	
[1]:	<pre>import networkx as nx # untuk membuat grafik import matplotlib.pyplot as plt # untuk plot grafik</pre>
Function untuk cetak plot	
[2]:	<pre># Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='') : # Membuat pos jika pos tidak diberikan if pos is None: pos = nx.spring_layout(G) # Fungsi untuk menggambar node nx.draw(G, # Graf NetworkX pos, # Posisi node with_labels=True, # Menampilkan nama node node_color='red', # Warna node node_size=2000, # Ukuran node font_color="white", # Warna font label node font_weight="bold", # Ketebalan font label node width=5 # Ketebalan garis edge) # Mengambil label edge jika ada weight edge_labels = nx.get_edge_attributes(G, 'weight') # Fungsi untuk menggambar node nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, # Data weight font_color='blue', # Warna font label edge font_weight="bold", # Berat font label edge font_size=12, # Ukuran font label edge)</pre>

	<pre>) plt.margins(0.2) # Memberikan margin pada plot plt.title(title) # Menampilkan judul graf jika diberikan plt.show() # Menampilkan graf menggunakan matplotlib</pre>
Posisi node `pos` agar sesuai dengan gambar contoh	
[3]:	<pre>pos = { 'A': (0, 3), 'B': (-10, 2), 'C': (10, 2), 'D': (-15, 1), 'E': (-5, 1), 'F': (5, 1), 'G': (15, 1), 'H': (-18, 0), 'I': (-12, 0), 'J': (-8, 0), 'K': (-2, 0), 'L': (2, 0), 'M': (8, 0), 'N': (12, 0), 'O': (18, 0) }</pre>
Membuat tree dengan NetworkX	
[4]:	<pre># Inisialisasi graf kosong G = nx.Graph()</pre>
[5]:	<pre># Definisi node dan edge sesuai dengan gambar edges = [('A', 'B', 1), ('A', 'C', 3), ('B', 'D', 2), ('B', 'E', 1), ('C', 'F', 1), ('C', 'G', 2), ('D', 'H', 2), ('D', 'I', 3), ('E', 'J', 3), ('E', 'K', 2), ('F', 'L', 3), ('F', 'M', 2), ('G', 'N', 2), ('G', 'O', 3)] # Menambahkan edge pada graf G.add_weighted_edges_from(edges)</pre>
Evaluasi elemen pada graf	
[5]:	<pre>print("Elemen pada Graf:") print("Daftar Node pada Graf:", G.nodes()) print("Daftar Edge pada Graf:", G.edges()) print("Jumlah Node pada Graf", G.number_of_nodes()) print("Jumlah Edge pada Graf", G.number_of_edges())</pre>
	<pre>Output: Elemen pada Graf: Daftar Node pada Graf: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O'] Daftar Edge pada Graf: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'F'), ('C', 'G'), ('D', 'H'), ('D', 'I'), ('E', 'J'), ('E', 'K'), ('F', 'L'), ('F', 'M'), ('G', 'N'), ('G', 'O')] Jumlah Node pada Graf 15 Jumlah Edge pada Graf 14</pre>
Visualisasi graf	
[6]:	<pre># Mencetak graf show_graph(G, pos=pos)</pre>
	<pre>Output:</pre>

		
	Membuat fungsi heuristik	
[7]:	<pre>def heuristable(a, b): return 0</pre>	
	Menentukan jalur dari node A ke node M dengan UCS	
[8]:	<pre># Mencari node yang dikunjungi dengan weight terkecil dari A ke M path_node = nx.astar_path(G, 'A', 'M', heuristic=heuristable, weight='weight') # Mencari weight terkecil dari A ke M path_length = nx.astar_path_length(G, 'A', 'M', heuristic=heuristable, weight='weight') # Cetak hasil print("Node yang dikunjungi dari A ke M:", path_node) print("Besar weight terkecil dari A ke M:", path_length)</pre>	
	Output: Node yang dikunjungi dari A ke M: ['A', 'C', 'F', 'M'] Besar weight terkecil dari A ke M: 6	

7.2. Implementasi UCS Pada *Directed Graph*

UCS pada graf digunakan untuk menemukan jalur dengan biaya terendah dari *node* awal (*start*) ke *node* tujuan (*goal*). UCS selalu memperluas *node* dengan biaya kumulatif terendah terlebih dahulu. Pada *directed graph* arah *edge* mempengaruhi perluasan *node*.

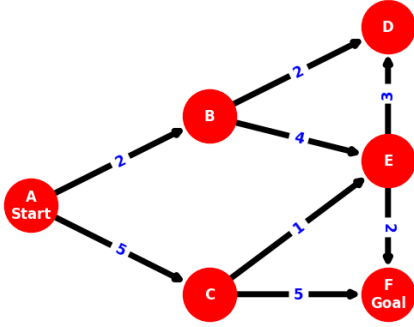


Gambar 15. Contoh directed graph untuk UCS.

Berikut adalah kode implementasi untuk UCS pada *directed graph* untuk Gambar 15:

Import library yang digunakan	
[1]:	<pre>import networkx as nx # untuk membuat grafik import matplotlib.pyplot as plt # untuk plot grafik</pre>
Function untuk cetak plot	
[2]:	<pre># Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='', labels={}): # Membuat pos jika pos tidak diberikan if pos is None: pos = nx.spring_layout(G) # Fungsi untuk menggambar node nx.draw(G, # Graf NetworkX pos, # Posisi node with_labels=True, # Menampilkan nama node labels=labels, # Menampilkan label setiap node node_color='red', # Warna node node_size=2000, # Ukuran node font_color="white", # Warna font label node font_weight="bold", # Ketebalan font label node width=5 # Ketebalan garis edge) # Mengambil label edge jika ada weight edge_labels = nx.get_edge_attributes(G, 'weight') # Fungsi untuk menggambar node nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, # Data weight font_color='blue', # Warna font label edge font_weight="bold", # Berat font label edge font_size=12, # Ukuran font label edge)</pre>

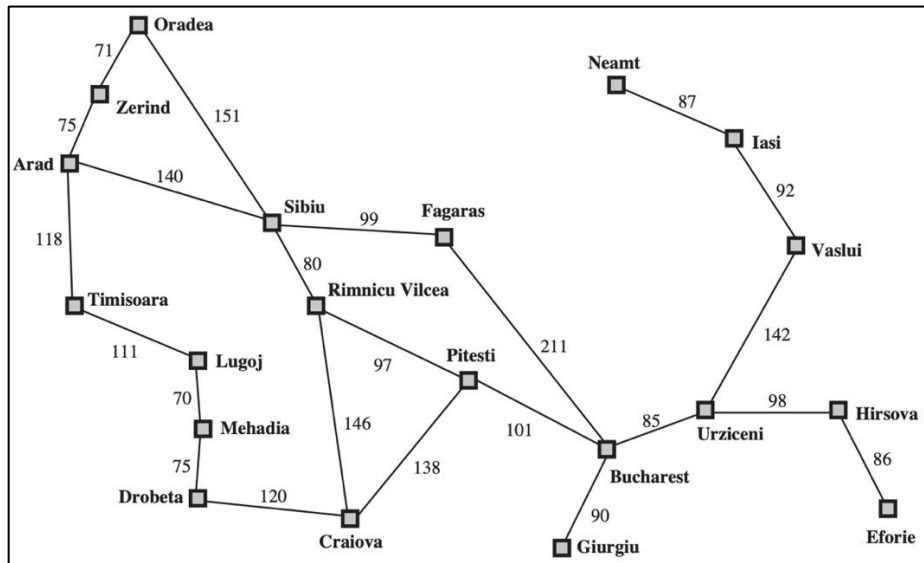
	<pre>plt.margins(0.2) # Memberikan margin pada plot plt.title(title) # Menampilkan judul graf jika diberikan plt.show() # Menampilkan graf menggunakan matplotlib</pre>
Posisi node `pos` dan label <i>start</i> dan <i>goal</i> agar sesuai dengan gambar contoh	
[3]:	<pre>pos = { 'A': (0, 1), 'B': (1, 2), 'C': (1, 0), 'D': (2, 3), 'E': (2, 1.5), 'F': (2, 0) }</pre>
Membuat <i>directed graph</i> dengan NetworkX	
[4]:	<pre># Inisialisasi graf berarah kosong G = nx.DiGraph()</pre>
[5]:	<pre># Definisi node dan edge sesuai dengan gambar edges = [('A', 'B', 2), ('A', 'C', 5), ('B', 'D', 2), ('B', 'E', 4), ('C', 'E', 1), ('C', 'F', 5), ('E', 'D', 3), ('E', 'F', 2)] # Menambahkan edge pada graf G.add_weighted_edges_from(edges)</pre>
Evaluasi elemen pada graf	
[6]:	<pre>print("Elemen pada Graf:") print("Daftar Node pada Graf:", G.nodes()) print("Daftar Edge pada Graf:", G.edges()) print("Jumlah Node pada Graf", G.number_of_nodes()) print("Jumlah Edge pada Graf", G.number_of_edges())</pre>
	<pre>Output: Elemen pada Graf: Daftar Node pada Graf: ['A', 'B', 'C', 'D', 'E', 'F'] Daftar Edge pada Graf: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'E'), ('C', 'F'), ('E', 'D'), ('E', 'F')] Jumlah Node pada Graf 6 Jumlah Edge pada Graf 8</pre>
Menambahkan label <i>start</i> dan <i>goal</i>	
[7]:	<pre># Mencatat start dan goal node labels = {node: node for node in G.nodes()} labels['A'] = 'A\nStart' labels['F'] = 'F\nGoal'</pre>
Visualisasi graf	
[8]:	<pre># Mencetak graf show_graph(G, pos=pos, labels=labels)</pre>
	<pre>Output:</pre>

		
Membuat fungsi heuristik		
[7]:	<pre>def heuristable(a, b): return 0</pre>	
Menentukan jalur dari node A ke node F dengan UCS		
[8]:	<pre># Mencari node yang dikunjungi dengan weight terkecil dari A ke M path_node = nx.astar_path(G, 'A', 'F', heuristic=heuristable, weight='weight') # Mencari weight terkecil dari A ke M path_length = nx.astar_path_length(G, 'A', 'F', heuristic=heuristable, weight='weight') # Cetak hasil print("Node yang dikunjungi dari A ke M:", path_node) print("Besar weight terkecil dari A ke M:", path_length)</pre>	
	Output: Node yang dikunjungi dari A ke M: ['A', 'B', 'E', 'F'] Besar weight terkecil dari A ke M: 8	

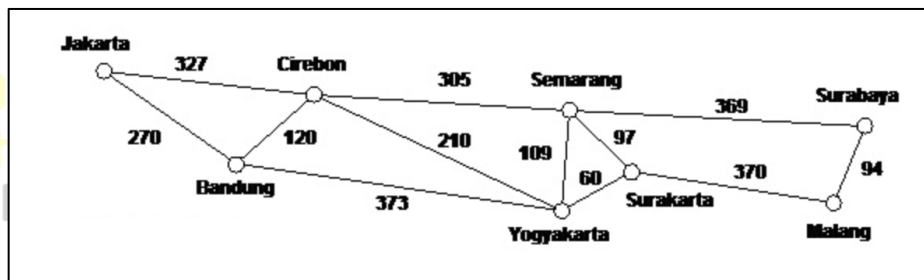
7.3. Soal Latihan

1. Buatlah graf berbobot (*weighted graf*) dari 2 gambar graf berikut ini menggunakan library NetworkX. Kemudian lakukan penelusuran Uniform Cost Search menggunakan library NetworkX! Kemudian periksalah hasilnya secara manual! Untuk graf Eropa, kota asal (source)-nya adalah Arad, dan kota tujuan (goal)-nya adalah Bucharest. Untuk graf Pulau Jawa, kota asal (source)-nya adalah Bandung, dan kota tujuan (goal)-nya adalah Malang. Nilai returned value pada tabel heuristicnya disetting sama dengan nol. Gunakan tipe data graf biasa (bukan directed graph).

a. Graf peta kota-kota besar di Eropa:



b. Graf peta kota-kota besar di Indonesia (pulau Jawa):



Modul 8 IMPLEMENTASI *HILL CLIMBING*

Tujuan Praktikum:

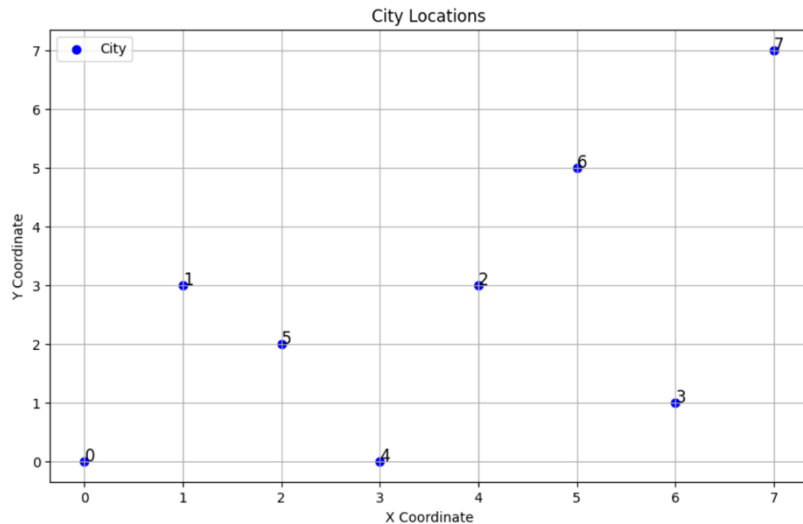
- | |
|---|
| 1. Mahasiswa memahami dan mampu mengimplementasikan algoritma <i>Hill Climbing</i> pada Python. |
|---|

Hill Climbing adalah salah satu algoritma *heuristic search* yang digunakan untuk mencari solusi terbaik dalam ruang pencarian. Tujuan dari algoritma ini adalah untuk mencari titik dalam ruang pencarian yang memberikan nilai tertinggi atau terendah tergantung pada jenis masalah yang sedang diselesaikan.

Cara kerja algoritma ini dapat dianalogikan sebagai berikut, bayangkan terdapat pendaki yang sedang mendaki gunung dalam kegelapan. pendaki tersebut tidak memiliki peta, tetapi anda selalu memilih jalur yang terlihat paling menanjak untuk mencapai puncak. Algoritma *hill climbing* bekerja dengan prinsip yang serupa. Pendaki memulai pencarian dari titik awal yang acak, lalu terus bergerak ke arah yang meningkatkan nilai suatu fungsi (seperti ketinggian gunung) hingga mencapai titik di mana tidak ada lagi peningkatan.

8.1. Contoh Implementasi *Hill Climbing*

[Traveling Salesman Problem] Seorang *salesman* harus mengunjungi 8 kota, di mana setiap kota hanya boleh dikunjungi satu kali. Tujuannya adalah menemukan rute dengan jarak total yang paling minimum dari persebaran kota pada Gambar 16.



Gambar 16. Grafik persebaran kota yang akan dikunjungi oleh salesman pada soal Hill Climbing.

Berikut adalah kode implementasi algoritma *Hill Climbing* untuk menyelesaikan masalah tersebut:

Import library yang digunakan	
[1]:	<pre>import random import math</pre>
Deklarasi function yang akan digunakan	
[2]:	<pre># Menghitung jarak 2 titik dengan menggunakan rumus euclidean def distance(city1, city2): return math.sqrt((city1[0] - city2[0]) ** 2 + (city1[1] - city2[1]) ** 2) # Menjumlahkan seluruh jarak antar titik pada rute tour def total_distance(tour, cities): dist = 0 for i in range(len(tour)): dist += distance(cities[tour[i]], cities[tour[(i + 1) % len(tour)]]) return dist # Memberikan semua kombinasi rute yang mungkin dengan cara menukar # dua titik dalam rute tour yang diberikan def get_neighbors(tour): neighbors = [] for i in range(len(tour)): for j in range(i + 1, len(tour)): neighbor = tour[:] neighbor[i], neighbor[j] = neighbor[j], neighbor[i] neighbors.append(neighbor) return neighbors</pre>
Hill Climbing Algorithm	
[3]:	<pre># Deklarasi seluruh titik kota (x, y) dalam bentuk tuple kedalam list cities cities = [(0, 0), (1, 3), (4, 3), (6, 1), (3, 0), (2, 2), (5, 5), (7, 7)]</pre>

	<pre> current_solution = list(range(len(cities))) # deklarasi rute kota awal random.shuffle(current_solution) # mengacak rute awal menggunakan fungsi random # Menetapkan total jarak untuk rute awal current_distance = total_distance(current_solution, cities) # Program akan terus melakukan loop hingga tidak menemukan nilai distance # yang lebih kecil dari current distance while True: # Mendapatkan seluruh kombinasi rute dengan cara menukar 2 titik # pada current solution neighbors = get_neighbors(current_solution) # Menggunakan fungsi lambda untuk mencari nilai minimum # berdasarkan total distance dalam list neighbors best_neighbor = min(neighbors, key=lambda tour: total_distance(tour, cities)) # Menghitung total distance berdasarkan best neighbor yang didapat best_distance = total_distance(best_neighbor, cities) # Jika tidak ada neighbor yang lebih baik, keluar dari loop if best_distance >= current_distance: break # Update solusi saat ini dengan solusi terbaik dari tetangga current_solution = best_neighbor current_distance = best_distance print("Solusi terbaik (urutan kota):", current_solution) print("Total jarak terbaik:", current_distance) </pre>
	Output: Solusi terbaik (urutan kota): [5, 2, 6, 7, 3, 4, 0, 1] Total jarak terbaik: 24.122094492753842

Algoritma *hill climbing* tidak selalu memberikan solusi optimal karena cenderung terjebak pada *local optimum* (solusi terbaik di sekitarnya), namun bukan solusi terbaik secara keseluruhan (*global optimum*). Oleh karena itu, untuk masalah kompleks, algoritma ini sering kali tidak menemukan solusi paling optimal tanpa tambahan teknik seperti *simulated annealing* atau *genetic algorithm*.

8.2. Soal Latihan

Seorang *salesman* ingin mengunjungi 5 kota berbeda, dan setiap kota hanya boleh dikunjungi satu kali. *Salesman* harus kembali ke kota awal setelah mengunjungi semua kota. Tujuannya adalah mencari rute terpendek untuk mengunjungi semua kota dengan menggunakan algoritma *Hill Climbing* dengan jarak yang tertera pada Tabel 6.

Tabel 6. Jarak antar kota yang harus dikunjungi salesman.

Kota	A	B	C	D	E
A	0	10	15	20	25

B	10	0	35	25	15
C	15	35	0	30	20
D	20	25	30	0	10
E	25	15	20	10	0

1. Buat representasi solusi awal dengan urutan kota yang akan dikunjungi.
2. Jelaskan bagaimana cara menghasilkan *neighbors* dari solusi awal tersebut dengan menukar dua kota.
3. Gunakan algoritma *Hill Climbing* untuk mencari rute yang paling pendek. Berikan proses perubahan rute dari solusi awal ke solusi yang lebih baik.
4. Apa rute terpendek yang ditemukan oleh algoritma? Apakah rute tersebut merupakan solusi paling optimal atau masih merupakan *local optimum*?

Modul 9 IMPLEMENTASI *SIMULATED ANNEALING*

Tujuan Praktikum:

1. Mahasiswa memahami dan mampu mengimplementasikan algoritma *Simulated Annealing* pada Python.

Simulated Annealing adalah teknik yang digunakan untuk mencari solusi terbaik dari berbagai kemungkinan. Algoritma ini terinspirasi dari proses dalam dunia fisika yang disebut *annealing*, yaitu ketika suatu material dipanaskan hingga sangat panas, lalu didinginkan perlahan. Proses ini memungkinkan material membentuk struktur yang **kuat dan stabil**. Algoritma *Simulated Annealing* bekerja dengan cara yang mirip dengan konsep *annealing* tersebut, algoritma memulai dengan menjelajahi berbagai solusi, bahkan jika beberapa solusi awalnya tampak buruk, lalu secara bertahap algoritma menjadi lebih "pintar" dalam memilih solusi yang lebih baik seiring berjalannya waktu.

Jika dibandingkan dengan algoritma *Hill Climbing* yang selalu memilih solusi yang lebih baik di setiap langkah, *Simulated Annealing* lebih fleksibel. *Simulated Annealing* kadang-kadang bisa memilih solusi yang lebih buruk, agar bisa menjelajahi lebih banyak kemungkinan solusi. Hal ini berguna agar tidak terjebak dalam solusi lokal (*local optimum*) yang terlihat baik tapi sebenarnya bukan yang terbaik. Jadi, *Simulated Annealing* lebih berpeluang untuk menemukan solusi terbaik dari semua kemungkinan, bukan hanya yang terlihat baik di sekitar langkah yang diambil saat ini.

9.1. Cara Kerja Algoritma *Simulated Annealing*

Beirkut adalah cara kerja algoritma *simulated annealing* dalam memecahkan masalah TSP:

1. Inisialisasi Nilai Awal:

Algoritma akan menghasilkan solusi awal berupa urutan acak dari kota kota yang perlu dikunjungi. Misal kita memiliki kota 0 hingga 3, maka urutan tersebut akan diacak menjadi solusi awal seperti `[3, 1, 0, 2]`.

2. Inisialisasi Parameter Suhu:

Algoritma akan menetapkan parameter **suhu awal** yang besar, misalnya 10000. Nilai tersebut akan menurun seiring berjalannya iterasi. Seberapa cepat suhu tersebut turun akan diatur berdasarkan **cooling rate** (laju penurunan suhu). Selain suhu awal dan **cooling rate**, algoritma akan menentukan **stopping temp** sebagai batas akhir iterasi.

3. Loop Berdasarkan Suhu:

Algoritma akan masuk ke dalam *loop* yang akan terus berjalan selama suhu masih di atas **stopping temp**. Pada setiap iterasi, suhu akan berkurang perlahan berdasarkan *cooling rate* dengan cara mengalikan kedua nilai tersebut (``temp *= cooling_rate``).

4. Buat Solusi Baru:

Pada setiap iterasi, algoritma akan membuat solusi baru dengan sedikit perubahan dari solusi sebelumnya. Perubahan ini bisa dilakukan dengan cara **menukar dua nilai secara acak** dalam solusi sebelumnya.

5. Evaluasi Solusi Baru:

- Jika solusi baru lebih baik, algoritma akan langsung menerima solusi baru tersebut dengan cara melakukan *update* nilai solusi saat ini.
- Jika solusi baru lebih buruk, algoritma bisa menerima solusi buruk tersebut dengan pertimbangan probabilitas tertentu berdasarkan nilai temp saat ini. Algoritma akan menghitung probabilitas dengan menggunakan **Metropolis Criterion** yang memiliki persamaan sebagai berikut:

$$P = e^{\frac{\text{current_distance} - \text{new_distance}}{\text{temp}}}$$

Jika bilangan acak lebih kecil dari nilai P, maka solusi buruk akan diterima.

6. Update Solusi Terbaik:

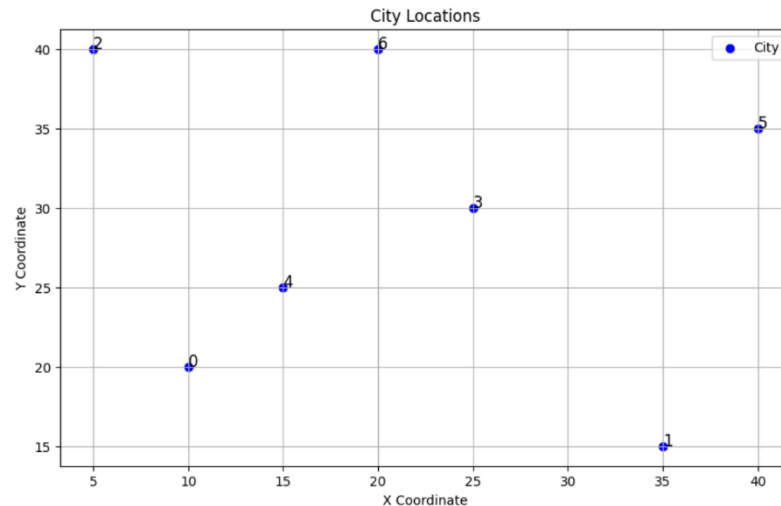
Di setiap iterasi, algoritma akan membandingkan solusi saat ini dengan solusi terbaik yang pernah ditemukan. Jika solusi saat ini lebih baik, maka solusi terbaik akan diganti dengan solusi saat ini.

7. Hasil Akhir:

Algoritma akan terus menurunkan suhu berdasarkan *cooling rate*, iterasi akan berhenti ketika suhu turun di bawah batas yang sudah ditentukan, kemudian algoritma akan memberikan solusi terbaik yang ditemukan selama iterasi sebagai solusi optimal.

9.2. Contoh Implementasi

[Traveling Salesman Problem] Seorang *salesman* harus mengunjungi 7 kota yang terdapat pada Gambar 17, di mana setiap kota hanya boleh dikunjungi satu kali. Tujuannya adalah menemukan rute dengan jarak total yang paling minimum. Buatlah solusi untuk kasus TSP tersebut menggunakan pendekatan *Simulated Annealing*.



Gambar 17. Grafik persebaran kota yang akan dikunjungi oleh salesman pada contoh soal Simulated Annealing.

Berikut adalah kode implementasi algoritma *simulated annealing* untuk menyelesaikan masalah tersebut:

Import library yang digunakan	
[1]:	<pre>import random import math</pre>
Deklarasi function yang akan digunakan	
[2]:	<pre># Fungsi untuk menghitung jarak 2 titik dengan menggunakan rumus euclidean def distance(city1, city2): return math.sqrt((city1[0] - city2[0]) ** 2 + (city1[1] - city2[1]) ** 2) # Fungsi untuk menjumlah seluruh jarak antar titik pada rute tour def total_distance(tour, cities): dist = 0 for i in range(len(tour)): dist += distance(cities[tour[i]], cities[tour[(i + 1) % len(tour)]]) return dist # Fungsi untuk melakukan random swap dua titik dalam rute def random_swap(tour): new_tour = tour[:] i, j = random.sample(range(len(tour)), 2) new_tour[i], new_tour[j] = new_tour[j], new_tour[i] return new_tour</pre>
Simulated Annealing Algorithm	
Algoritma akan menyimpan 2 nilai solusi. <i>Best solution</i> akan menyimpan solusi terbaik, dan <i>current solution</i> akan menyimpan solusi untuk melakukan eksplorasi lebih lanjut untuk menghindari <i>local optimum</i>.	
[3]:	<pre>def simulated_annealing(cities, initial_temp, cooling_rate, stopping_temp): # Inisialisasi solusi awal current_solution = list(range(len(cities))) # Rute awal (urut) random.shuffle(current_solution) # Mengacak rute awal # Menghitung total jarak untuk rute awal current_distance = total_distance(current_solution, cities)</pre>

	<pre> # Set best solution dan best distance berdasarkan current solution best_solution = current_solution[:] best_distance = current_distance # Set suhu awal berdasarkan parameter yang diterima oleh function temp = initial_temp # Program akan terus melakukan loop hingga suhu kurang dari suhu stopping while temp > stopping_temp: # Buat solusi baru dengan menukar dua titik acak new_solution = random_swap(current_solution) new_distance = total_distance(new_solution, cities) # Jika solusi baru lebih baik, terima solusi tersebut if new_distance < current_distance: current_solution = new_solution[:] current_distance = new_distance else: # jika solusi baru lebih buruk, terima dengan probabilitas tertentu if random.random() < math.exp(-(current_distance - new_distance)/temp): current_solution = new_solution[:] current_distance = new_distance # Simpan solusi terbaik yang ditemukan if current_distance < best_distance: best_solution = current_solution[:] best_distance = current_distance # Kurangi suhu sesuai cooling rate temp *= cooling_rate return best_solution, best_distance </pre>
[4]:	<pre> # Deklarasi titik-titik kota (x, y) cities = [(0, 0), (1, 3), (4, 3), (6, 1), (3, 0), (2, 2), (5, 5), (7, 7)] # Parameter algoritma init = 10000 # Suhu awal Cooling = 0.995 # Laju pendinginan Stop = 1 # Suhu ketika berhenti # Menjalankan algoritma Simulated Annealing best_solution, best_distance = simulated_annealing(cities, init, cooling, stop) print("Solusi terbaik (urutan kota):", best_solution) print("Total jarak terbaik:", best_distance) </pre>
	Output: Solusi terbaik (urutan kota): [6, 2, 0, 4, 1, 5, 3] Total jarak terbaik: 112.65453203138081

Catatan penting!

```
`[if random.random() < math.exp((current_distance - new_distance)/temp)]`
```

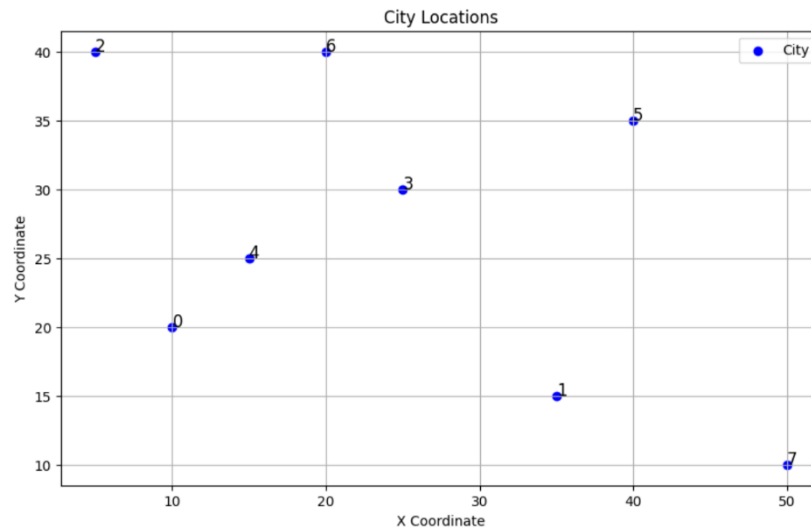
Tahap tersebut merupakan inti dari algoritma *Simulated Annealing* dimana tahap tersebut memungkinkan algoritma untuk menerima solusi yang lebih buruk berdasarkan probabilitas **Metropolis Criterion**. Hal ini bertujuan untuk menghindari *local optimum* dan memungkinkan eksplorasi yang lebih luas dari solusi.

- Jika solusi baru lebih buruk, algoritma akan menerima solusi tersebut dengan probabilitas **Metropolis Criterion**
- `random.random()` akan menghasilkan bilangan acak antara 0 hingga 1
- Sedangkan `math.exp((current_distance - new_distance) / temp)` merupakan persamaan probabilitas yang dikenal sebagai **Metropolis Criterion** dalam *Simulated Annealing*. Persamaan ini akan menentukan seberapa besar kemungkinan solusi buruk diterima berdasarkan *temp*.

Algoritma *Simulated Annealing* digunakan untuk mencari solusi *global optimum* dengan mengeksplorasi berbagai kemungkinan solusi dan menghindari jebakan *local optimum*, berbeda dengan algoritma *Hill Climbing* yang cenderung terjebak pada solusi terbaik lokal karena hanya menerima perbaikan langsung tanpa mengeksplorasi solusi yang lebih buruk. Dengan probabilitas yang dikontrol oleh suhu (*temperature*), *Simulated Annealing* memungkinkan penerimaan solusi yang lebih buruk pada awal iterasi untuk **memperluas ruang pencarian**, namun secara bertahap menjadi lebih selektif seiring turunnya nilai dari suhu (*cooling process*), sehingga meningkatkan peluang mencapai solusi global yang lebih baik.

9.3. Soal Latihan

[Multiple Traveling Saleman Problem] Terdapat dua orang *salesman* yang ingin mengunjungi 8 kota. Setiap salesman hanya diperbolehkan mengunjungi setiap kota satu kali, dan tidak ada kota yang dikunjungi lebih dari satu kali oleh kedua *salesman*. Koordinat 8 kota tersebut telah ditentukan berdasarkan titik-titik pada .



Gambar 18. Grafik persebaran kota yang akan dikunjungi oleh salesman pada soal latihan Simulated Annealing.

Buatlah solusi untuk menentukan rute perjalanan kedua *salesman* tersebut dengan menggunakan pendekatan Algoritma *Simulated Annealing*. Rute yang dihasilkan harus meminimalkan total jarak yang ditempuh oleh kedua *salesman* secara bersama-sama.



Fakultas Informatika
School of Computing
Telkom University



Modul 10 IMPLEMENTASI *GREEDY BEST-FIRST SEARCH*

Tujuan Praktikum:

1. Mahasiswa memahami dan mampu mengimplementasikan algoritma *Greedy Best-First Search* pada Python dengan menggunakan *library* NetworkX.

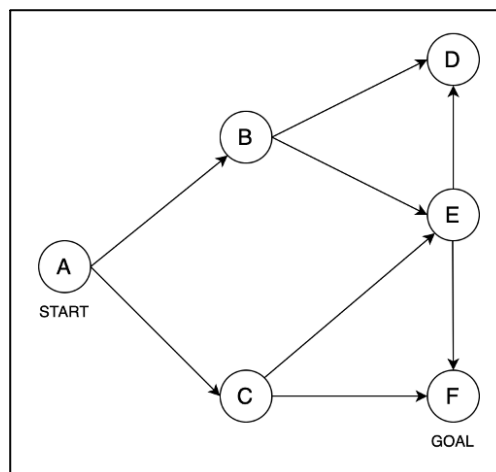
REFERENSI:

https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html#module-networkx.algorithms.shortest_paths.astar

Greedy Best-First Search (*Greedy BFS*) adalah algoritma pencarian yang memprioritaskan eksplorasi node yang dianggap paling menjanjikan untuk mencapai tujuan secepat mungkin berdasarkan fungsi heuristik. *Greedy BFS* hanya mempertimbangkan nilai heuristik dari *node* saat ini ke tujuan. Algoritma ini menggunakan pendekatan *greedy*, di mana setiap langkah, *node* dengan nilai heuristik terkecil dieksplorasi terlebih dahulu. Namun, *Greedy BFS* selalu menemukan jalur optimal karena hanya fokus pada pendekatan langsung tanpa mempertimbangkan biaya total.

Dalam NetworkX, *Greedy BFS* dapat diimplementasikan dengan memodifikasi fungsi `nx.astar_path()` dengan menetapkan nilai bobot pada setiap *edge* menjadi 0, sehingga hanya fungsi heuristik yang digunakan untuk memandu pencarian. Namun, pada implementasi *Greedy BFS* ini, `nx.astar_path_length()` tidak dapat digunakan karena setiap bobot pada *edge* memiliki nilai 0.

10.1. Implementasi *Greedy BFS* pada *Directed Graph*



Gambar 19. Contoh *directed graph* untuk *Greedy BFS*.

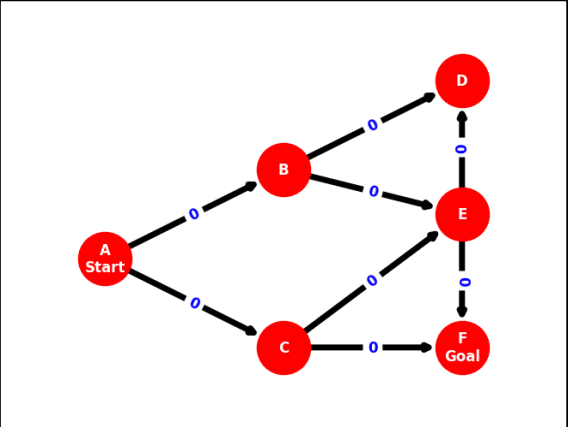
Tabel 7. Nilai heuristik untuk *directed graph Greedy BFS*.

Node	Nilai Heuristik
A	10
B	2
C	3
D	1
E	4
F	0

Berikut adalah kode implementasi untuk *Greedy BFS* pada *directed graph* untuk Gambar 19 dengan nilai heuristik yang tertera pada Tabel 7:

Import library yang digunakan	
[1]:	<code>import network as ns # untuk membuat grafik import matplotlib.pyplot as plt # untuk plot grafik</code>
Function untuk cetak plot	
[2]:	<code># Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='', labels={}): # Membuat pos jika pos tidak diberikan if pos is None: pos = nx.spring_layout(G) # Fungsi untuk menggambar node nx.draw(G, # Graf NetworkX pos, # Posisi node with_labels=True, # Menampilkan nama node labels=labels, # Menampilkan label setiap node node_color='red', # Warna node node_size=2000, # Ukuran node font_color="white", # Warna font label node font_weight="bold", # Ketebalan font label node width=5 # Ketebalan garis edge) # Mengambil label edge jika ada weight edge_labels = nx.get_edge_attributes(G, 'weight') # Fungsi untuk menggambar node nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, # Data weight</code>

	<pre> font_color='blue', # Warna font label edge font_weight="bold", # Berat font label edge font_size=12, # Ukuran font label edge) plt.margins(0.2) # Memberikan margin pada plot plt.title(title) # Menampilkan judul graf jika diberikan plt.show() # Menampilkan graf menggunakan matplotlib </pre>
Posisi node `pos` dan label <i>start</i> dan <i>goal</i> agar sesuai dengan gambar contoh	
[3]:	<pre> pos = { 'A': (0, 1), 'B': (1, 2), 'C': (1, 0), 'D': (2, 3), 'E': (2, 1.5), 'F': (2, 0) } </pre>
Membuat <i>directed graph</i> dengan NetworkX	
[4]:	<pre> # Inisialisasi graf berarah kosong G = nx.DiGraph() </pre>
[5]:	<pre> # Definisi node dan edge sesuai dengan gambar edges = [('A', 'B', 0), ('A', 'C', 0), ('B', 'D', 0), ('B', 'E', 0), ('C', 'E', 0), ('C', 'F', 0), ('E', 'D', 0), ('E', 'F', 0)] # Menambahkan edge pada graf G.add_weighted_edges_from(edges) </pre>
Evaluasi elemen pada graf	
[6]:	<pre> print("Elemen pada Graf:") print("Daftar Node pada Graf:", G.nodes()) print("Daftar Edge pada Graf:", G.edges()) print("Jumlah Node pada Graf", G.number_of_nodes()) print("Jumlah Edge pada Graf", G.number_of_edges()) </pre>
	Output: Elemen pada Graf: Daftar Node pada Graf: ['A', 'B', 'C', 'D', 'E', 'F'] Daftar Edge pada Graf: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'E'), ('C', 'F'), ('E', 'D'), ('E', 'F')] Jumlah Node pada Graf 6 Jumlah Edge pada Graf 8
Menambahkan label <i>start</i> dan <i>goal</i>	
[7]:	<pre> # Mencatat start dan goal node labels = {node: node for node in G.nodes()} labels['A'] = 'A\nStart' labels['F'] = 'F\nGoal' </pre>
Visualisasi graf	
[8]:	<pre> # Mencetak graf show_graph(G, pos=pos, labels=labels) </pre>

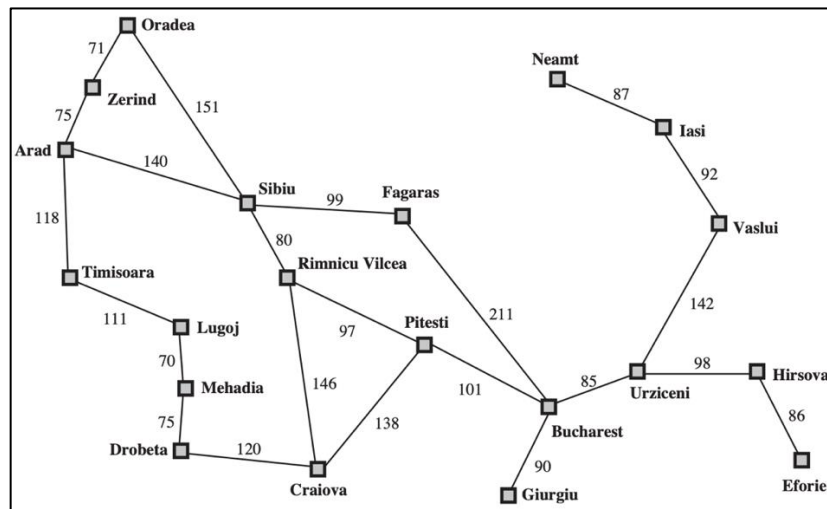
	Output: 
Membuat fungsi heuristik	
[7]:	<pre>def heuristable(a, b): if a == 'A' : return 10 elif a == 'B' : return 2 elif a == 'C' : return 3 elif a == 'D' : return 1 elif a == 'E' : return 4 elif a == 'F' : return 0</pre>
Menentukan jalur dari node A ke node F dengan UCS	
[8]:	<pre># Mencari node yang dikunjungi dengan weight terkecil dari A ke F path_node = nx.astar_path(G, 'A', 'F', heuristic=heuristable, weight='weight') # Cetak hasil print("Node yang dikunjungi dari A ke F:", path_node)</pre>
	Output: Node yang dikunjungi dari A ke F: ['A', 'C', 'F']

10.2. Soal Latihan

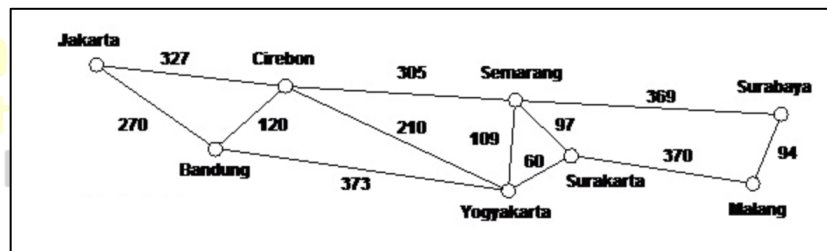
1. Buatlah graf berbobot (*weighted graf*) dari 2 gambar graf berikut ini menggunakan library NetworkX. Kemudian lakukan penelusuran *Greedy Best-First Search* menggunakan library NetworkX! Jangan lupa menghilangkan data *weight*-nya terlebih dahulu! Untuk graf Eropa, kota asal (*source*)-nya adalah Arad, dan kota tujuan (*goal*)-nya adalah Bucharest. Untuk graf Pulau Jawa, kota asal (*source*)-nya adalah Bandung, dan kota tujuan (*goal*)-nya adalah Malang. Untuk menentukan nilai *returned value* pada tabel heuristiknya, gunakan rumus *Euclidan Distance* untuk menghitung jarak antara dua titik (yaitu panjang garis lurus yang

menghubungkan kedua titik tersebut). Gunakan tipe data graf biasa (bukan *directed graph*).

a. Graf peta kota-kota besar di Eropa:



b. Graf peta kota-kota besar di Indonesia (pulau Jawa):



Modul 11 IMPLEMENTASI A*

Tujuan Praktikum:

1. Mahasiswa memahami dan mampu mengimplementasikan algoritma A* pada Python dengan menggunakan *library* NetworkX.

REFERENSI:

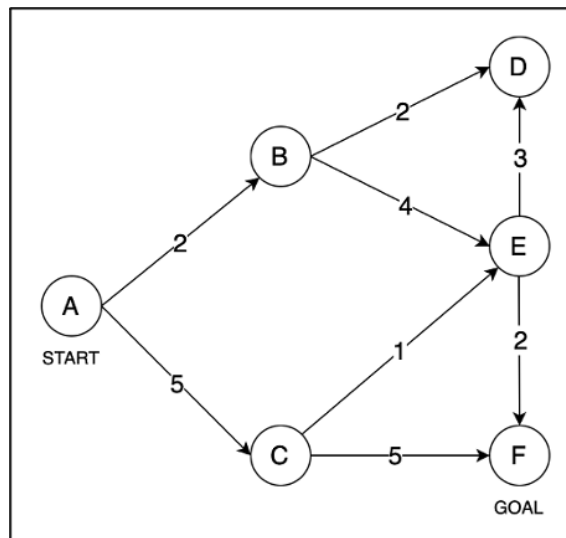
https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html#module-networkx.algorithms.shortest_paths.astar

A* adalah algoritma pencarian jalur yang digunakan untuk menemukan jalur terpendek dari *node* awal (*start*) ke *node* tujuan (*goal*) dalam graf. A* menggabungkan dari UCS dan *Greedy* BFS dengan mempertimbangkan biaya sebenarnya ($g(n)$) dan jalur yang telah dilalui, maupun perkiraan biaya ($h(n)$) dari *node* saat ini ke *node* tujuan berdasarkan fungsi heuristik.

A* menggunakan fungsi $f(n) = g(n) + h(n)$ untuk memprioritaskan *node* yang akan dieksplorasi, di mana $g(n)$ adalah biaya dari node awal ke node saat ini, dan $h(n)$ adalah estimasi biaya dari node saat ini ke tujuan. Dengan pendekatan ini, A* memastikan bahwa jalur yang ditemukan adalah jalur optimal.

Pada NetworkX, A* dapat diimplementasikan menggunakan fungsi ``nx.astar_path()``, yang secara otomatis menghitung jalur terpendek dengan mempertimbangkan biaya sebenarnya dan fungsi heuristik.

11.1. Implementasi A* Pada *Directed Graph*



Gambar 20. Contoh *directed graph* untuk A*.

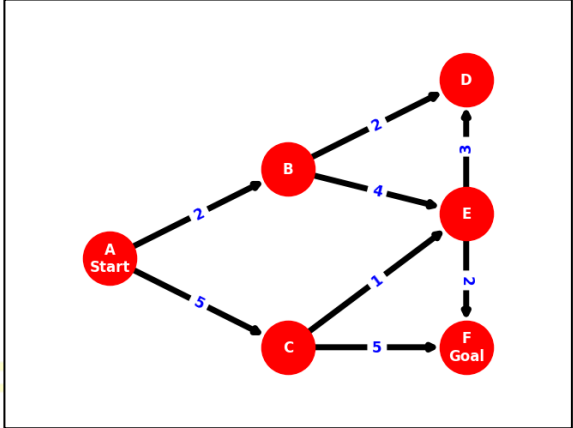
Tabel 8. Nilai heuristik untuk *directed graph* A*.

Node	Nilai heuristik
A	10
B	2
C	3
D	1
E	4
F	0

Berikut adalah kode implementasi untuk A* pada *directed graph* untuk Gambar 20 dengan nilai heuristik yang tertera pada Tabel 8:

Import library yang digunakan	
[1]:	<pre>import networkx as nx # untuk membuat grafik import matplotlib.pyplot as plt # untuk plot grafik</pre>
Function untuk cetak plot	
[2]:	<pre># Fungsi pendukung untuk mencetak graf def show_graph(G, pos=None, title='', labels={}): # Membuat pos jika pos tidak diberikan if pos is None: pos = nx.spring_layout(G) # Fungsi untuk menggambar node nx.draw(G, # Graf NetworkX pos, # Posisi node with_labels=True, # Menampilkan nama node</pre>

	<pre> labels=labels # Menampilkan label setiap node node_color='red', # Warna node node_size=2000, # Ukuran node font_color="white", # Warna font label node font_weight="bold", # Ketebalan font label node width=5 # Ketebalan garis edge) # Mengambil label edge jika ada weight edge_labels = nx.get_edge_attributes(G, 'weight') # Fungsi untuk menggambar node nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, # Data weight font_color='blue', # Warna font label edge font_weight="bold", # Berat font label edge font_size=12, # Ukuran font label edge) plt.margins(0.2) # Memberikan margin pada plot plt.title(title) # Menampilkan judul graf jika diberikan plt.show() # Menampilkan graf menggunakan matplotlib </pre>
Posisi node `pos` dan label start dan goal agar sesuai dengan gambar contoh	
[3]:	<pre> pos = { 'A': (0, 1), 'B': (1, 2), 'C': (1, 0), 'D': (2, 3), 'E': (2, 1.5), 'F': (2, 0) } </pre>
Membuat <i>directed graph</i> dengan NetworkX	
[4]:	<pre> # Inisialisasi graf berarah kosong G = nx.DiGraph() </pre>
[5]:	<pre> # Definisi node dan edge sesuai dengan gambar edges = [('A', 'B', 2), ('A', 'C', 5), ('B', 'D', 2), ('B', 'E', 4), ('C', 'E', 1), ('C', 'F', 5), ('E', 'D', 3), ('E', 'F', 2)] # Menambahkan edge pada graf G.add_weighted_edges_from(edges) </pre>
Evaluasi elemen pada graf	
[6]:	<pre> print("Elemen pada Graf:") print("Daftar Node pada Graf:", G.nodes()) print("Daftar Edge pada Graf:", G.edges()) print("Jumlah Node pada Graf", G.number_of_nodes()) print("Jumlah Edge pada Graf", G.number_of_edges()) </pre>
	<pre> Output: Elemen pada Graf: Daftar Node pada Graf: ['A', 'B', 'C', 'D', 'E', 'F'] </pre>

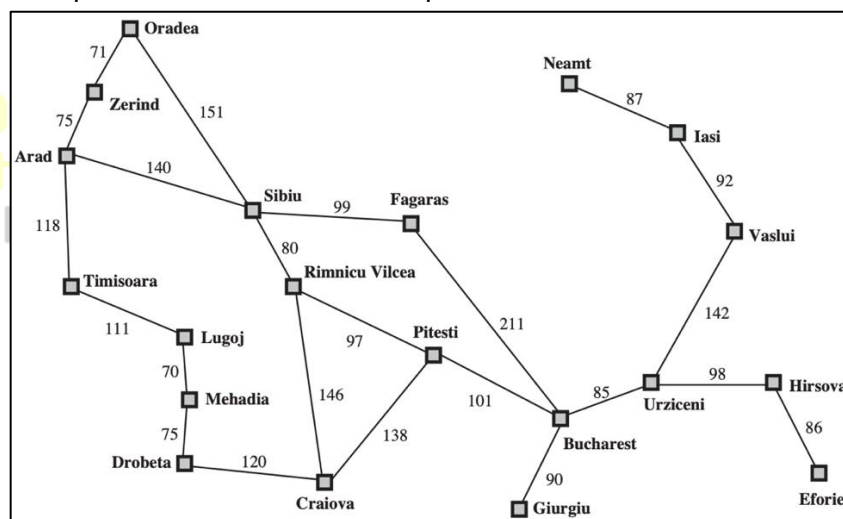
	Daftar Edge pada Graf: [('A', 'B'), ('A', 'C'), ('B', 'D'), ('B', 'E'), ('C', 'E'), ('C', 'F'), ('E', 'D'), ('E', 'F')] Jumlah Node pada Graf 6 Jumlah Edge pada Graf 8
Menambahkan label <i>start</i> dan <i>goal</i>	
[7]:	<pre># Mencatat start dan goal node labels = {node: node for node in G.nodes()} labels['A'] = 'A\nStart' labels['F'] = 'F\nGoal'</pre>
Visualisasi graf	
[8]:	<pre># Mencetak graf show_graph(G, pos=pos, labels=labels)</pre> Output: 
Membuat fungsi heuristik	
[7]:	<pre>def heuristable(a, b): if a == 'A' : return 10 elif a == 'B' : return 2 elif a == 'C' : return 3 elif a == 'D' : return 1 elif a == 'E' : return 4 elif a == 'F' : return 0</pre>
Menentukan jalur dari <i>node</i> A ke <i>node</i> F dengan UCS	
[8]:	<pre># Mencari node yang dikunjungi dengan weight terkecil dari A ke F path_node = nx.astar_path(G, 'A', 'F', heuristic=heuristable, weight='weight') # Mencari weight terkecil dari A ke F path_length = nx.astar_path_length(G, 'A', 'F', heuristic=heuristable, weight='weight') # Cetak hasil print("Node yang dikunjungi dari A ke F:", path_node) print("Besarnya weight hasil A* dari A ke F:", path_length)</pre>

Output:	Node yang dikunjungi dari A ke F: ['A', 'B', 'E', 'F'] Besarnya weight hasil A* dari A ke F: 8
---------	---

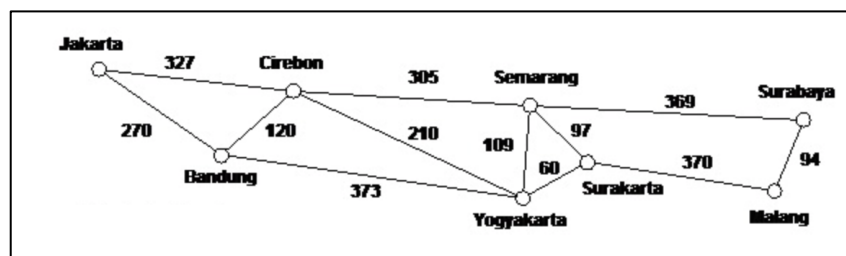
11.2. Soal Latihan

- Buatlah graf berbobot (*weighted graf*) dari 2 gambar graf berikut ini menggunakan *library* NetworkX. Kemudian lakukan penelusuran A* menggunakan *library* NetworkX! Untuk graf Eropa, kota asal (*source*)-nya adalah Arad, dan kota tujuan (*goal*)-nya adalah Bucharest. Untuk graf Pulau Jawa, kota asal (*source*)-nya adalah Bandung, dan kota tujuan (*goal*)-nya adalah Malang. Untuk menentukan nilai returned value pada tabel heuristiknya, gunakan rumus *Euclidan Distance* untuk menghitung jarak antara dua titik (yaitu panjang garis lurus yang menghubungkan kedua titik tersebut). Gunakan tipe data graf biasa (bukan *directed graph*). Ingat pula bahwa baik nilai weight dan nilai tabel heuristik sama-sama digunakan dalam perhitungan algoritma A*.

- Graf peta kota-kota besar di Eropa:



- Graf peta kota-kota besar di Indonesia (pulau Jawa):



Modul 12 PENGENALAN *LIBRARY* SYMPY

Tujuan Praktikum:

1. Mahasiswa memahami cara pembuatan program dengan *library* Sympy.
2. Mahasiswa mengetahui fitur-fitur utama yang tersedia di *library* Sympy.

REFERENSI UTAMA:

<https://docs.sympy.org/latest/tutorials/intro-tutorial/index.html>

<https://certik.github.io/scipy-2013-tutorial/html/index.html>

REFERENSI TAMBAHAN:

<https://docs.sympy.org/latest/guides/index.html>

<https://docs.sympy.org/latest/reference/index.html>

SymPy adalah *library* Python yang digunakan untuk melakukan perhitungan simbolik dalam matematika. Dengan SymPy, ekspresi matematis dapat direpresentasikan, disederhanakan, dievaluasi, dan dimanipulasi secara simbolis, bukan hanya secara numerik.

Tujuan dari SymPy adalah untuk menyediakan alat-alat yang memungkinkan perhitungan matematika dilakukan dengan akurasi simbolik yang tinggi, sehingga perhitungan seperti aljabar, kalkulus, dan operasi matriks dapat diselesaikan secara analitis.

12.1. Cara *Import* SymPy

SymPy seharusnya sudah terdapat pada instalasi Anaconda. Untuk menggunakan modul SymPy, harus dilakukan *import* terlebih dahulu. Berikut adalah cara standar untuk mengimport *library* SymPy:

```
import sympy as sp
```

12.2. Pendefinisian Simbol Pada SymPy

SymPy memungkinkan perhitungan matematika dilakukan secara simbolik, yang berarti bahwa variabel dan ekspresi matematika dapat didefinisikan dan dimanipulasi sebagai simbol aljabar. Dengan simbol, berbagai perhitungan dan manipulasi matematika dapat dilakukan secara simbolis. Untuk melakukan ini, langkah pertama adalah mendefinisikan simbol menggunakan ``symbols()``. Berikut adalah beberapa contoh dalam penggunaan ``symbols()``:

```
[1]: # Penggunaan symbols() untuk mendefinisikan satu simbol
      x = sp.symbols('x')
```

	x
	Output: x
[2]:	<pre># Pendefinisian beberapa simbol # Cara 1 a, b, c = sp.symbols("a b c") # Cara 2 a, b, c = sp.symbols("a, b, c") a + b + c</pre>
	Output: a + b + c
[3]:	<pre># Pendefinisian simbol dengan list my_symbols = ['a', 'b', 'c'] a, b, c = sp.symbols(my_symbols) a + b + c</pre>
	Output: a + b + c
[4]:	<pre># Pendefinisian simbol variabel dengan indeks x1, x2, x3 = sp.symbols("x1, x2, x3") x1 + x2 + x3</pre>
	Output: x₁ + x₂ + x₃
[5]:	<pre># Pendefinisian simbol dengan notasi range x_n = sp.symbols('x:5') x_n[0] + x_n[1] + x_n[2] + x_n[3] + x_n[4]</pre>
	Output: x₁ + x₂ + x₃ + x₄ + x₅
[6]:	<pre># Pendefinisian simbol Yunani alpha, beta, gamma = sp.symbols("alpha, beta, gamma") alpha + beta + gamma</pre>
	Output: α + β + γ

12.3. Aljabar Dasar Pada SymPy

SymPy memungkinkan untuk melakukan operasi aljabar dasar secara simbolik, yang berarti variabel dan ekspresi matematika dapat dimanipulasi seperti dalam aljabar. SymPy menyediakan alat untuk penjumlahan, pengurangan, perkalian, pembagian, pengembangan, dan penyederhanaan ekspresi matematika. Berikut adalah implementasi aljabar dasar pada SymPy:

[1]:	<pre># Penjumlahan x, y = sp.symbols('x, y') addition = x + y addition</pre>
	Output: x + y
[2]:	<pre># Pengurangan x, y = sp.symbols('x, y') subtraction = x - y subtraction</pre>
	Output:

	$x - y$
[3]:	<pre># Perkalian x, y = sp.symbols('x, y') multiplication = x * y multiplication</pre>
	Output: xy
[4]:	<pre># Pembagian x, y = sp.symbols('x, y') division = x / y division</pre>
	Output: $\frac{x}{y}$
<code>.expand()</code> Metode ini digunakan untuk mengembangkan ekspresi simbolik, yang menguraikan ekspresi menjadi bentuk yang lebih panjang.	
[1]:	<pre>x, y = sp.symbols('x, y') expr = (x + y) ** 2 expr</pre>
	Output: $(x + y)^2$
[2]:	<pre># Mengembangkan ekspresi expanded_expr = expr.expand() expanded_expr</pre>
	Output: $x^2 + xy + y^2$
<code>.simplify()</code> Metode ini digunakan untuk menyederhanakan ekspresi simbolik ke bentuk yang paling sederhana. Metode ini mungkin melibatkan penggabungan suku, pemecahan faktor, penyederhanaan trigonometri, dan lain-lain untuk meneukan ekspresi yang paling sederhana.	
[1]:	<pre>x = sp.symbols('x') expr = sp.sin(x)**2 + sp.cos(x)**2 expr</pre>
	Output: $\sin^2(x) + \cos^2(x)$
[2]:	<pre># Menyederhanakan ekspresi simplified_expr = expr.simplify() simplified_expr</pre>
	Output: 1

12.4. Kalkulus Dasar Pada SymPy

SymPy juga dapat digunakan untuk melakukan perhitungan kalkulus dasar secara simbolik. Ini termasuk diferensiasi, integrasi, dan perhitungan limit. Dengan SymPy, operasi ini dapat dilakukan pada ekspresi matematis secara analitis, memungkinkan manipulasi dan evaluasi simbolik tanpa perlu konversi langsung ke bentuk numerik. Berikut adalah contoh implementasi kalkulus dasar pada SymPy:

```
.diff()
```

Metode ini digunakan untuk melakukan difrensiasi atau menghitung turunan dari suatu fungsi simbolik terhadap variabel tertentu.	
[1]:	<pre>x = sp.symbols('x') # Pendefinisian fungsi f(x) f = x**3 + 2*x**2 + x f</pre>
	Output: $x^3 + 2x^2 + x$
[2]:	<pre># Difrensiasi fungsi f(x) terhadap x f_prime = f.diff(x) f_prime</pre>
	Output: $3x^2 + 4x + 1$
<pre>` .integrate() `</pre> Metode ini digunakan untuk melakukan integrasi dari suatu fungsi simbolik. Integrasi dapat dilakukan secara tak tentu (integral tanpa batas) atau secara tentu (integral dengan batas)	
[1]:	<pre>x = sp.symbols('x') # Pendefinisian fungsi f(x) f = sp.sin(x)</pre>
	Output: $\sin(x)$
[2]:	<pre># Integrasi tak tentu f_integral_indef = sp.integrate(f, x) f_integral_indef</pre>
	Output: $-\cos(x)$
[3]:	<pre># Integrasi tentu dari 0 sampai pi f_integral_def = sp.integrate(f, (x, 0, sp.pi)) # .integrate(fungsi, (ekspresi, batas_bawah, batas_atas)) f_integral_def</pre>
	Output: 2
<pre>` .limit() `</pre> Metode ini digunakan untuk menghitung limit dari suatu fungsi ketika variabel mendekati nilai teretntu.	
[1]:	<pre>x = sp.symbols('x') # Pendefinisian fungsi f(x) f = sp.sin(x) / x f</pre>
	Output: $\frac{\sin(x)}{x}$
[2]:	<pre># Menghitung limit f(x) saat x mendekati 0 f_limit = sp.limit(f, x, 0) f_limit</pre>
	Output: 1

12.5. Penyelesaian Persamaan Simbolik Pada SymPy

Sympy menyediakan alat yang kuat untuk menyelesaikan persamaan simbolik, baik persamaan aljabar maupun persamaan difrensial. Dua metode utama yang digunakan

dalam SymPy untuk penyelesaian simbolik adalah `.solve()` dan `.dsolve()`. Berikut adalah implementasi dan contoh dari penyelesaian persamaan simbolik:

<code>.eq()</code>	
Fungsi ini digunakan untuk mendefinisikan sebuah persamaan simbolik pada SymPy. Fungsi ini digunakan untuk menyatakan bahwa dua ekspresi matematika sama.	
[1]:	<pre>x = sp.symbols('x') # Membuat persamaan equation = sp.Eq(x**2 - 4, 0) equation</pre>
	Output: $x^2 - 4 = 0$
[2]:	<pre>x, y = sp.symbols('x, y') # Membuat persamaan equation = sp.Eq(x**3 - 2, y - 2 + 3*x) equation</pre>
	Output: $x^3 - 2 = 3x + y - 2$
<code>.solve()</code>	
Merupakan fungsi yang digunakan untuk mencari solusi dari persamaan atau sistem persamaan simbolik. Fungsi ini memecahkan persamaan aljabar dengan mencari nilai simbolik yang memenuhi persamaan tersebut.	
[1]:	<pre>x = sp.symbols('x') # Membuat persamaan equation = sp.Eq(x**2 - 4, 0) equation</pre>
	Output: $x^2 - 4 = 0$
[2]:	<pre># Memecahkan persamaan untuk x solution = sp.solve(equation, x) solution</pre>
	Output: [-2, 2]
<code>.dsolve()</code>	
Merupakan fungsi yang digunakan untuk menyelesaikan persamaan diferensial simbolik. Fungsi ini mencari Solusi umum dari persamaan difrensial dalam bentuk simbolik.	
[1]:	<pre># Pendefinisian simbol dan fungsi t = sp.symbols('t') y = sp.Function('y')(t) equation = sp.Eq(y(t).diff(t,t) + y(t), 0) equation</pre>
	Output: $\frac{d}{dt}y(t) = -t + y(t)$
[2]:	<pre># Penyelesaian persamaan difrensial solution = sp.dsolve(equation, y(t)) solution</pre>
	Output: $y(t) = C_1 e^t + t + 1$

12.6. Manipulasi Matriks Pada SymPy

SymPy menyediakan berbagai alat untuk manipulasi matriks, yang mencakup pendefinisian matriks, operasi determinan, operasi invers, dan perkalian matriks. Berikut adalah implementasi dan contoh manipulasi matriks pada SymPy:

<code>`<b>.Matrix()</b>`</code> Fungsi ini digunakan untuk mendefinisikan matriks yang dapat diisi dengan angka, variabel simbolik, atau ekspresi simbolik.	
<code>[1]:</code>	<code># Pendefinisian matriks 2x2 A = sp.Matrix([[1, 2], [3,4]]) A</code>
	Output: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
<code>[2]:</code>	<code># Pendefinisian matriks nol 3x3 A = sp.zeros(3) A</code>
	Output: $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$
<code>[3]:</code>	<code># Pendefinisian matriks identitas 3x3 A = sp.eye(3) A</code>
	Output: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
<code>`<b>.det()</b>`</code> Fungsi ini digunakan untuk menghitung determinan dari sebuah matriks persegi.	
<code>[1]:</code>	<code># Pendefinisian Matriks A = sp.Matrix([[1, 2], [3, 4]]) A</code>
	Output: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
<code>[2]:</code>	<code># Menghitung determinan dari matriks A det_A = A.det() det_A</code>
	Output: -2
<code>`<b>.inv()</b>`</code> Fungsi ini digunakan untuk menghitung invers dari sebuah matriks.	
<code>[1]:</code>	<code># Pendefinisian Matriks A = sp.Matrix([[1, 2], [3, 4]]) A</code>
	Output: $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

[2]:	# Penyelesaian persamaan difrensial <code>solution = sp.dsolve(equation, y(t))</code> <code>solution</code>
	Output: $\begin{bmatrix} -2 & 1 \\ 3 & 1 \\ 2 & -\frac{1}{2} \end{bmatrix}$
Matrix Multiplication Perkalian matriks dilakukan dengan `*`, menghasilkan matriks baru yang merupakan hasil dari perkalian dua matriks.	
[1]:	# Pendefinisian Matriks <code>A = sp.Matrix([[1, 2], [3, 4]])</code> <code>B = sp.Matrix([[5, 6], [7, 8]])</code> # Perkalian matriks A dan B <code>mat_mul = A * B</code> <code>mat_mul</code>
	Output: $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

12.7. Soal Latihan *Library* SymPy

A. Dengan menggunakan *library* SymPy, carilah akar-akar dari persamaan-persamaan berikut ini (jika akar-akarnya imajiner atau tidak ada, tuliskan output apa yang muncul di layar):

1. $x - 2 = 0$
2. $x^2 - 3x - 4 = 0$
3. $x^2 - 6x + 9 = 0$
4. $x^2 - x - 1 = 0$
5. $x^3 - 6x^2 + 11x - 6 = 0$
6. $x^3 - 7x^2 + 15x - 9 = 0$
7. $x^5 + 7x^4 - 9x^2 + 15x + 2 = 0$
8. $x^5 + 9x^2 + 2 = 0$

B. Dengan menggunakan library Sympy, carilah turunan pertama atau $\frac{dy}{dx}$ dari:

- | | |
|--------------------------|---------------------------|
| 1. $y = x^3 + 3x^2 + 6x$ | 2. $y = x^5 + x^4$ |
| 3. $y = (3x + 5)^3$ | 4. $y = (3 - 5x)^5$ |
| 5. $y = \sin(7x)$ | 6. $y = \sin(x^3)$ |
| 7. $y = \frac{1}{x - 1}$ | 8. $y = \frac{3x}{1 - x}$ |

C. Dengan menggunakan library Sympy, carilah integral tentu (*definite integral*) dari:

- | | |
|---|--|
| 1. $\int_0^2 x^3 dx$ | 2. $\int_{-1}^2 x^4 dx$ |
| 3. $\int_{-1}^2 (3x^2 - 2x + 3) dx$ | 4. $\int_1^2 (4x^3 + 7) dx$ |
| 5. $\int_1^4 \frac{1}{w^2} dw$ | 6. $\int_1^3 \frac{2}{t^3} dt$ |
| 7. $\int_0^4 \sqrt{t} dt$ | 8. $\int_1^8 \sqrt[3]{w} dw$ |
| 9. $\int_{-4}^{-2} \left(y^2 + \frac{1}{y^3} \right) dy$ | 10. $\int_1^4 \frac{s^4 - 8}{s^2} ds$ |
| 11. $\int_0^{\pi/2} \cos x dx$ | 12. $\int_{\pi/6}^{\pi/2} 2 \sin t dt$ |
| 13. $\int_0^1 (2x^4 - 3x^2 + 5) dx$ | 14. $\int_0^1 (x^{4/3} - 2x^{1/3}) dx$ |

D. Diberikan matriks A sebagai berikut. Dengan menggunakan *library* Sympy, carilah determinan dari matriks A.

$$A = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 3 & 1 & 2 & 2 \\ 1 & 0 & -2 & 1 \\ 2 & 0 & 0 & 1 \end{bmatrix}$$

Modul 13 PENGENALAN *LIBRARY* SCIKIT-FUZZY

Tujuan Praktikum:

1. Mahasiswa memahami cara pembuatan program dengan *library* Scikit-Fuzzy.
2. Mahasiswa mengetahui fitur-fitur utama yang tersedia di *library* Scikit-Fuzzy.

REFERENSI UTAMA:

https://scikit-fuzzy.github.io/scikit-fuzzy/userguide/getting_started.html
https://scikit-fuzzy.github.io/scikit-fuzzy/auto_examples/plot_tipping_problem_newapi.html#example-plot-tipping-problem-newapi-py

REFERENSI TAMBAHAN:

https://scikit-fuzzy.github.io/scikit-fuzzy/user_guide.html
https://scikit-fuzzy.github.io/scikit-fuzzy/auto_examples/index.html
<https://github.com/scikit-fuzzy/scikit-fuzzy>
<https://pypi.org/project/scikit-fuzzy/>

Fuzzy Logic adalah pendekatan komputasi yang meniru cara manusia membuat keputusan berdasarkan informasi yang tidak pasti atau ambigu. Tidak seperti logika tradisional yang hanya mengenal nilai 0 (salah) dan 1 (benar), *fuzzy logic* memperkenalkan konsep derajat kebenaran, di mana nilai bisa berada di antara 0 dan 1, mencerminkan tingkat kepastian dan ketidakpastian.

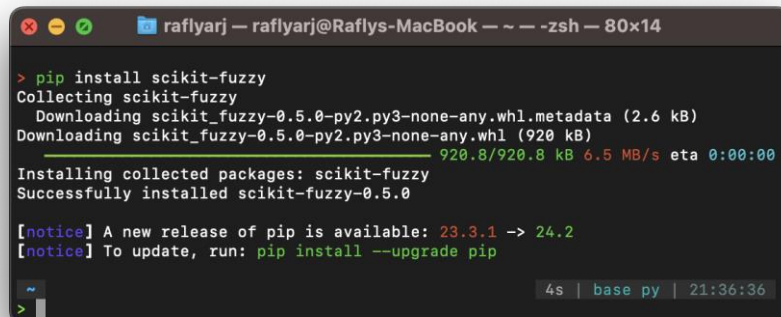
Dalam *fuzzy logic*, suatu kondisi tidak perlu sepenuhnya benar atau salah, misalnya suhu dapat dikategorikan sebagai "hangat" dengan derajat tertentu, bukan hanya "panas" atau "dingin". Hal ini memungkinkan sistem untuk menangani ketidakpatian dan variasi dengan yang lebih fleksibel.

Pada Python, perhitungan *fuzzy logic* dapat dilakukan dengan mudah menggunakan *library* scikit-fuzzy. *Library* ini memudahkan untuk membangun, menguji, dan menerapkan sistem *fuzzy logic* dengan mendefinisikan variabel *fuzzy*, fungsi keanggotaan, aturan *fuzzy*, serta melakukan proses fuzzifikasi, inferensi, dan defuzzifikasi.

13.1. Instalasi Scikit-Fuzzy

Scikit-Fuzzy tidak tersedia pada *package* Anaconda secara *default*. Berikut adalah langkah-langkah untuk menambahkan *package* Scikit-Fuzzy pada *environment* Anaconda:

1. Buka **Anaconda Prompt** untuk Windows atau **Terminal** pada MacOS.
2. Jalankan perintah ``pip install scikit-fuzzy``.
3. Jika berhasil, maka akan terlihat tampilan seperti Gambar 21.



```
rafiyarj — rafiylarj@Rafiyls-MacBook — ~ — zsh — 80x14
> pip install scikit-fuzzy
Collecting scikit-fuzzy
  Downloading scikit_fuzzy-0.5.0-py2.py3-none-any.whl.metadata (2.6 kB)
  Downloading scikit_fuzzy-0.5.0-py2.py3-none-any.whl (920 kB)
    920.8/920.8 kB 6.5 MB/s eta 0:00:00
Installing collected packages: scikit-fuzzy
Successfully installed scikit-fuzzy-0.5.0

[notice] A new release of pip is available: 23.3.1 -> 24.2
[notice] To update, run: pip install --upgrade pip

4s | base py | 21:36:36
>
```

Gambar 21. Tampilan terminal saat instalasi Scikit-Fuzzy berhasil.

13.2. Membuat Program dengan *Library* Scikit-Fuzzy

13.2.1. Inisialisasi Variabel *Fuzzy*

Dalam *fuzzy logic*, terdapat dua variabel yang mendefinisikan bagaimana *fuzzy system* beroperasi dan menghasilkan keputusan. Variabel tersebut terdiri atas **variabel input** dan **variabel output**. variabel *input* adalah nilai atau data yang diberikan kepada *fuzzy system* untuk diproses, variabel ini mewakili kondisi atau faktor-faktor yang mempengaruhi hasil akhir dalam suatu sistem. Sedangkan variabel output adalah hasil atau keputusan yang dihasilkan oleh *fuzzy system* setelah melalui aturan-aturan *fuzzy* yang telah ditentukan.

Dalam Scikit-Fuzzy, variabel *input* didefinisikan dengan ``ctrl.Antecedent()`` yang berperan sebagai *input* dalam *fuzzy logic*. Berikut adalah parameter yang digunakan dalam ``ctrl.Antecedent()``:

- ``universe``: Parameter ini adalah rentang atau domain nilai yang dapat diambil oleh variabel *fuzzy*. Parameter ini didefinisikan dengan NumPy *array* dengan menggunakan ``np.arange(start, stop, step)``, yang menunjukkan rentang nilai dari ``start`` hingga ``stop`` dengan langkah ``step``.
- ``label``: Label atau nama yang diberikan untuk variabel *fuzzy* akan digunakan untuk referensi dalam aturan *fuzzy logic*.

Berikut adalah contoh penggunaan `ctrl.Antecedent()`:

```
from skfuzzy import control as ctrl
# Pendefinisian variabel kontrol quality
quality = ctrl.Antecedent(np.arange(0, 11, 1), 'quality')
```

Pada kode di atas, `quality` adalah variabel *fuzzy* yang dapat memiliki nilai hingga 0 hingga 10 dan variabel *fuzzy* ini diberikan label `quality`.

Sedangkan variable *output*, Scikit-Fuzzy menyediakan `ctrl.Consequent()` yang digunakan untuk mendefinisikan variabel *fuzzy* yang berperan sebagai *output* dari sistem yang akan ditentukan berdasarkan *input* yang diberikan dan aturan *fuzzy logic* yang telah ditentukan. `ctrl.Consequent()` memiliki parameter yang sama seperti `ctrl.Antecedent()`.

Berikut adalah implementasi dan contoh dari `ctrl.Consequent()`:

```
from skfuzzy import control as ctrl
# Pendefinisian variabel output tip
tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')
```

Dalam contoh di atas, `tip` merupakan variabel *fuzzy* yang dapat memiliki nilai 0 hingga 25, dan diberi label "tip" sebagai penanda.

Berikut adalah hubungan antara `ctrl.Antecedent()` sebagai variabel *input* dan `ctrl.Consequent()` sebagai variabel *output*:

- `ctrl.Antecedent()` berfungsi sebagai variabel *input* yang akan dievaluasi dalam aturan *fuzzy*, sedangkan `ctrl.Consequent()` adalah variabel *output* yang nilainya akan dipengaruhi oleh evaluasi aturan-aturan tersebut.
- Dalam sebuah sistem kontrol *fuzzy*, aturan-aturan didefinisikan untuk menghubungkan nilai dari `ctrl.Antecedent()` ke `ctrl.Consequent()`. Misalnya "Jika kualitas (*Antecedent*) buruk, maka tip (*Consequent*) rendah".

13.2.2. Membuat Fungsi Keanggotaan

Fungsi keanggotaan dalam *fuzzy logic* adalah kurva yang mendefinisikan bagaimana setiap titik dalam *input* atau *output* dipetakan ke derajat keanggotaan dalam himpunan *fuzzy*. Fungsi ini mengubah nilai numerik menjadi nilai *fuzzy* yang menunjukkan seberapa besar suatu nilai termasuk dalam kategori tertentu, seperti "rendah", "sedang", atau "tinggi".

Dalam scikit-fuzzy, `.trimf()` merupakan salah satu fungsi untuk membuat keanggotaan dalam *fuzzy logic* yang digunakan untuk menentukan derajat keanggotaan suatu nilai dalam himpunan *fuzzy*. **Trimf** adalah singkatan dari *Triangular Membership Function* (Fungsi keanggotaan segitiga).

Fungsi ini digunakan untuk memetakan *fuzzy system* untuk memetakan *input* ke dalam nilai keanggotaan antara 0 dan 1. Fungsi ini paling sederhana dan banyak digunakan karena bentuknya yang mudah dipahami.

Fungsi `.trimf()` didefinisikan sebagai berikut

```
import skfuzzy as fuzz
# Pendefinisian keanggotaan pada quality
quality['membership'] = fuzz.trimf(x, [a, b, c])
```

Berikut adalah penjelasan dari parameter `.trimf()`:

- `x`: Array dari nilai input, ini bisa diambil dari *Antecedent* atau *Consequent* yang sudah dibuat.
- `a` (*Lower Bound*): Titik awal segitiga, di mana derajat keanggotaan mulai dari 0.
- `b` (*Peak / Middle Point*): Titik puncak segitiga, di mana derajat keanggotaan mencapai 1.
- `c` (*Upper Bound*): Titik akhir segitiga, di mana derajat keanggotaan kembali ke 0.

Fungsi keanggotaan yang sudah dibuat dapat divisualisasikan dengan cara berikut:

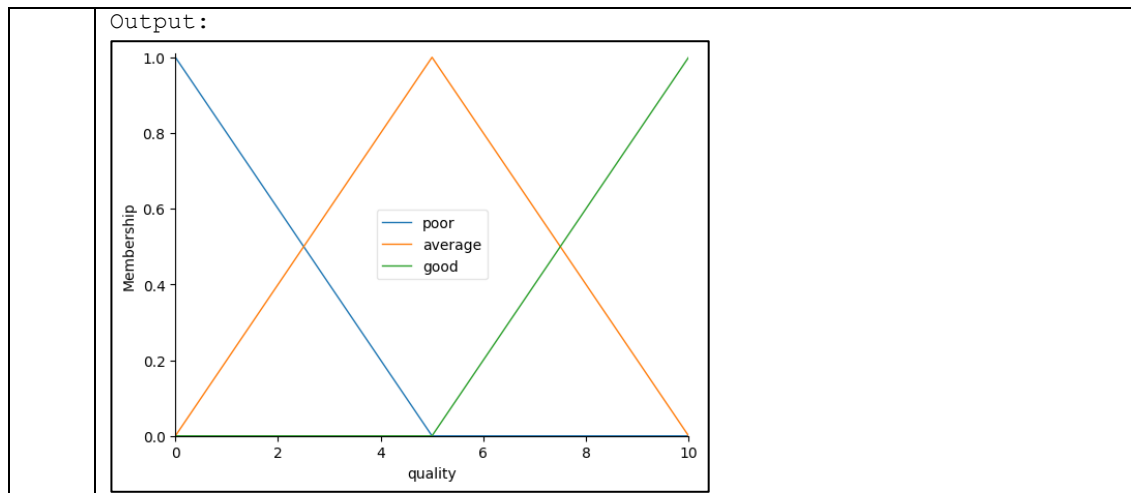
```
# Menampilkan visualisasi dari fungsi keanggotaan yang sudah diberikan
quality.show()
```

Hasilnya adalah grafik dengan bentuk segitiga yang menunjukkan derajat keanggotaan untuk setiap nilai dalam rentang yang diberikan.

Berikut adalah implementasi dan contoh untuk membuat fungsi keanggotaan:

```
[1]: import skfuzzy as fuzz
      from skfuzzy import control as ctrl
      # Mendefinisikan input variabel
      quality = ctrl.Antecedent(np.arange(0, 11, 1), 'quality')
      # Membuat fungsi keanggotaan untuk quality
      quality['poor'] = fuzz.trimf(quality.universe, [0, 0, 5])
      quality['average'] = fuzz.trimf(quality.universe, [0, 5, 10])
      quality['good'] = fuzz.trimf(quality.universe, [5, 10, 10])

      # Menampilkan visualisasi keanggotaan quality
      quality.show()
```



13.2.3. Penentuan Aturan Fuzzy

Aturan fuzzy adalah komponen penting dalam sistem *fuzzy logic* yang digunakan untuk menghubungkan *input* dengan *output* berdasarkan logika yang ditentukan. Aturan fuzzy dinyatakan dalam bentuk "**Jika ... Maka ...**" yang menghubungkan kondisi input dengan hasil *output*.

Dalam scikit-fuzzy, aturan tersebut didefinisikan dengan `.Rule()`. Paramater `.Rule()` terdiri dari *Antecedent* atau syarat / kondisi dan *Consequent* atau hasil / tindakan. Berikut adalah implementasi dari `.Rule()`:

```
# quality, service = Antecedent, tip = Consequent
rule1 = ctrl.Rule(quality['poor'] | service['poor'], tip['low'])
```

Arti dari rule di atas adalah: "**Jika quality atau service bernilai poor maka tip menjadi low**".

Berikut adalah operator logika dalam `.Rule()` pada parameter **Antecedent**.

- `&` (AND): Menghubungkan dua kondisi yang keduanya harus terpenuhi agar aturan terpicu.
- `|` (OR): Menghubungkan dua kondisi di mana salah satu kondisi yang terpenuhi sudah cukup untuk memicu aturan.
- `~` (NOT): Membalik kondisi dari variabel fuzzy.

Setelah aturan didefinisikan, aturan-aturan tersebut dimasukkan ke dalam *fuzzy system* menggunakan `.ControlSystem()`, yang kemudian dapat disimulasikan untuk menghasilkan *output* berdasarkan *input* yang diberikan. Objek untuk mensimulasikan

fuzzy system juga harus dibuat menggunakan `.ControlSystemSimulation()`.

Berikut adalah implementasi `.ControlSystemSimulation()`:

```
# Membuat control system dari beberapa aturan yang sudah dibuat
tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
tipping_simulation = ctrl.ControlSystemSimulation(tipping_ctrl)
```

Berikut adalah contoh implementasi dari penentuan aturan *fuzzy*:

```
# Pendefinisian rule
rule1 = ctrl.Rule(quality['poor'] | service['poor'], tip['low'])
rule2 = ctrl.Rule(service['average'], tip['medium'])
rule3 = ctrl.Rule(service['good'] | quality['good'], tip['high'])

# Penggabungan aturan
tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
# Pembuatan objek untuk melakukan simulasi
tipping_simulation = ctrl.ControlSystemSimulation(tipping_ctrl)
```

13.2.4. Pemberian *Input* dan Perhitungan dari *Fuzzy System*

Pemberian input kepada ***Antecedent fuzzy system*** dapat menggunakan `.input()`.

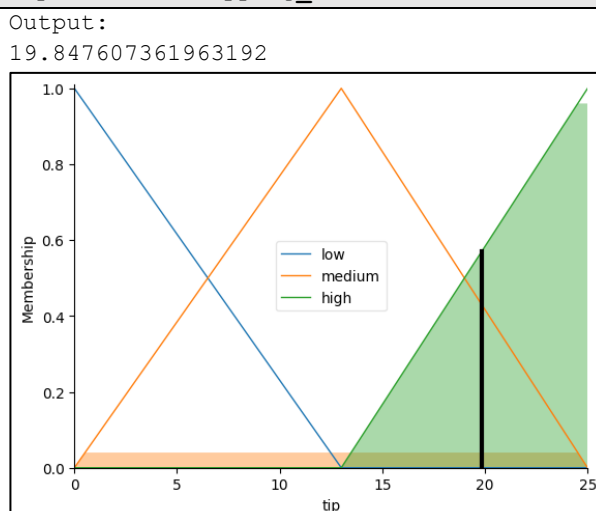
Dengan *method* ini, nilai numerik untuk variabel *input* dapat diproses *fuzzy system* untuk menghasilkan *output*. Kemudian, setelah semua *input* diberikan kepada variabel *fuzzy*, nilai tersebut dapat dihitung dengan `.compute()` untuk mendapatkan hasil.

Berikut adalah implementasi dari `.input()` dan `.compute()`:

```
[1]: # Memberikan nilai variabel quality dan service
tipping_simulation.input['quality'] = 6.5
tipping_simulation.input['service'] = 9.8

# Menghitung output berdasarkan input yang diberikan
tipping_simulation.compute()

[2]: # Mencetak output dan menghasilkan visualisasi
print(tipping_simulation.output['tip'])
tip.view(sim=tipping_simulation)
```



Pada hasil diatas, *fuzzy system* memberikan rekomendasi jika nilai *quality* 6.5 dan *service* 9.8, maka *tip* yang diberikan adalah 19.84%.

13.3. Study Case Sistem Fuzzy dengan Scikit-Fuzzy

Anda diminta untuk membuat sebuah *fuzzy system* yang memodelkan bagaimana *tip* diberikan di sebuah restoran. Ketika *tip* diberikan, dua faktor utama akan dipertimbangkan: kualitas layanan (***service***) dan kualitas makanan (***quality***). Kedua faktor ini dinilai pada skala 0 hingga 10, di mana 0 berarti sangat buruk dan 10 berarti sangat baik. Berdasarkan penilaian ini, *fuzzy system* akan menghitung rekomendasi besaran *tip* antara 0% hingga 25%. Berikut adalah informasi yang diberikan:

Input (Antecedents):

- **Quality**

- Rentang nilai : [0 – 10]
- Fuzzy set:
 - Poor: *Triangular* dengan titik [0, 0, 5]
 - Average: *Triangular* dengan titik [0, 5, 10]
 - Good: *Triangular* dengan titik [5, 10, 10]

- **Service**

- Rentang nilai: [0 – 10]
- Fuzzy set:
 - Poor: *Triangular* dengan titik [0, 0, 5]
 - Average: *Triangular* dengan titik [0, 5, 10]
 - Good: *Triangular* dengan titik [5, 10, 10]

Output (Consequent):

- **Tip**

- Rentang nilai = [0 – 25].
- Fuzzy set:
 - Low: *Triangular* dengan titik [0, 0, 13].
 - Medium: *Triangular* dengan titik [0, 13, 15].
 - High: *Triangular* dengan titik [13, 13, 25].

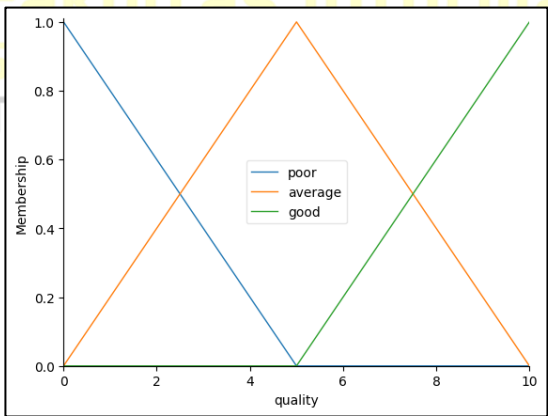
Aturan Fuzzy

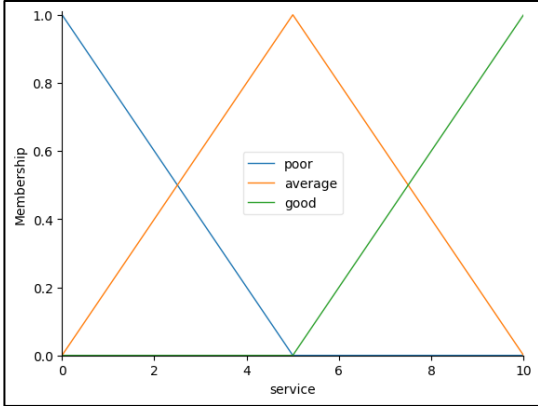
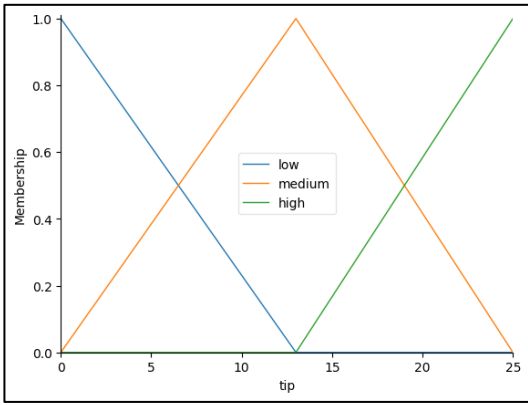
- Aturan 1: Jika ***service*** `good` dan ***quality*** `good` maka ***tip*** `high`.
- Aturan 2: Jika ***service*** `average` maka ***tip*** `medium`.

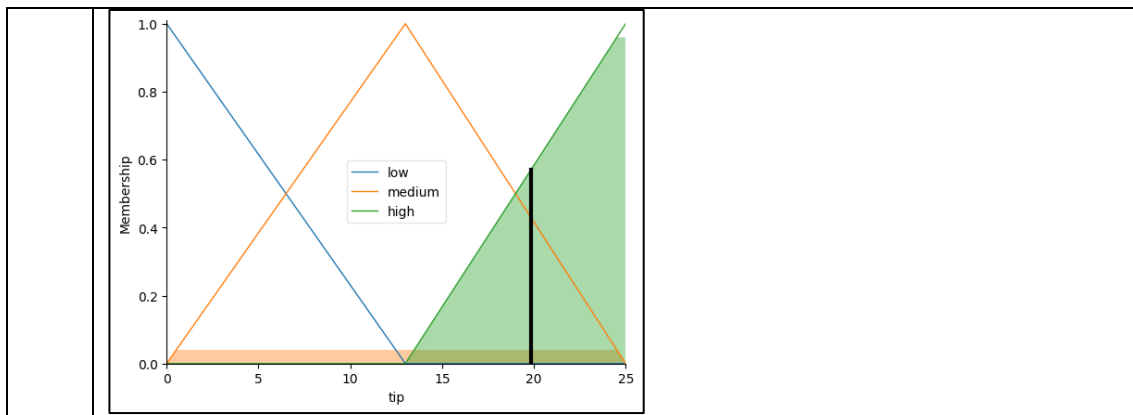
- Aturan 3: Jika **service** `poor` dan **quality** `poor` maka **tip** `low`.

Contoh Kasus

Misalkan *service* dinilai dengan 9.8 dan *quality* dinilai dengan 6.5. Besaran *fuzzy* di atas, sebuah rekomendasi akan memberikan rekomendasi *tip* sebesar 19.8%.

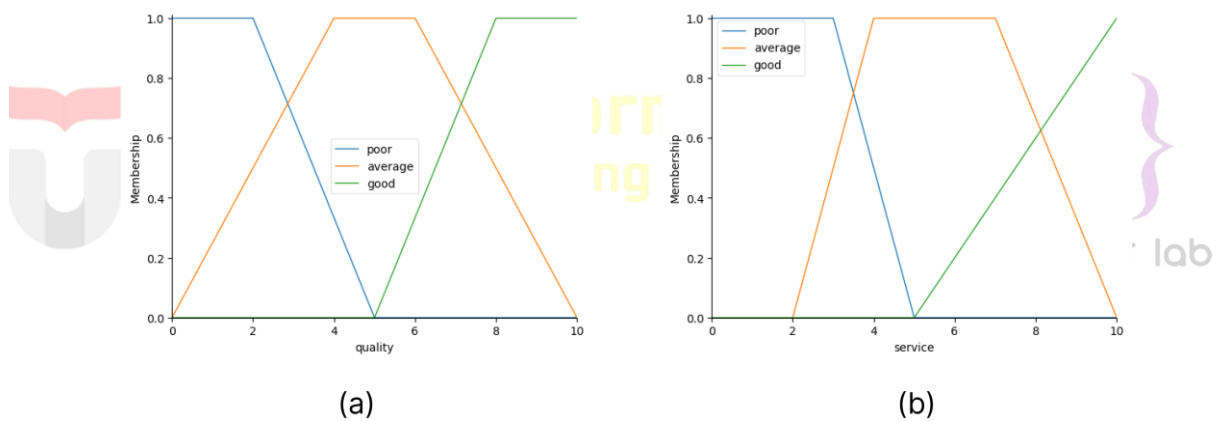
[1]:	<pre># Import module yang diperlukan import skfuzzy as fuzz import numpy as np from skfuzzy import control as ctrl import matplotlib.pyplot as plt</pre>
[2]:	<pre># Definisi varibel input quality = ctrl.Antecedent(np.arange(0, 11, 1), 'quality') service = ctrl.Antecedent(np.arange(0, 11, 1), 'service') # Definisi variabel output tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')</pre>
[3]:	<pre># Fungsi keanggotaan untuk quality quality['poor'] = fuzz.trimf(quality.universe, [0,0,5]) quality['average'] = fuzz.trimf(quality.universe, [0,5,10]) quality['good'] = fuzz.trimf(quality.universe, [5,10,10]) # Visualisasi quality.view()</pre>
	Output: 
[4]:	<pre># Fungsi keanggotaan untuk service service['poor'] = fuzz.trimf(service.universe, [0, 0, 5]) service['average'] = fuzz.trimf(service.universe, [0, 5, 10]) service['good'] = fuzz.trimf(service.universe, [5, 10, 10]) service.view()</pre>
	Output:

	
[5]:	<pre># Fungsi keanggotaan untuk tip tip['low'] = fuzz.trimf(tip.universe, [0, 0, 13]) tip['medium'] = fuzz.trimf(tip.universe, [0, 13, 25]) tip['high'] = fuzz.trimf(tip.universe, [13, 25, 25]) tip.view()</pre>
	Output: 
[5]:	<pre># Membuat aturan dari sistem fuzzy rule1 = ctrl.Rule(quality['poor'] service['poor'], tip['low']) rule2 = ctrl.Rule(service['average'], tip['medium']) rule3 = ctrl.Rule(service['good'] quality['good'], tip['high'])</pre>
[6]:	<pre># Membuat objek untuk defuzzification tipping_ctrl = ctrl.ControlSystem([rule1, rule2, rule3]) tipping_simulation = ctrl.ControlSystemSimulation(tipping_ctrl)</pre>
[7]:	<pre># Memberikan input pada variabel Antecedent tipping_simulation.input['quality'] = 6.5 tipping_simulation.input['service'] = 9.8 # Menghitung output dari sistem fuzzy tipping_simulation.compute()</pre>
[8]:	<pre># Menampilkan hasil & visualisasi perhitungan print(tipping_simulation.output['tip']) tip.view(sim=tipping_simulation)</pre>
	Output: 19.847607361963192



13.4. Soal Latihan Pemrograman Scikit-Fuzzy

Ubah fungsi keanggotaan *quality* dan *service* pada soal *study case* 13.3. Fungsi keanggotaan *quality* dapat diubah seperti Gambar 22(a) dan fungsi keanggotaan *service* dapat diubah seperti Gambar 22(b).



Gambar 22. Grafik fungsi keanggotaan untuk soal latihan. (a) Grafik fungsi keanggotaan *quality*. (b) Grafik fungsi keanggotaan *service*.



Kontak Kami :



@dky2921g



@informaticslab



@informaticslab_telu



informaticslab@telkomuniversity.ac.id



informatics.labs.telkomuniversity.ac.id